Curso Completo de AutoGen

Table of Contents

1. Módulo 1: Introducción a AutoGen . . . 1
  1.1. 1.1. ¿Qué es AutoGen? . . . 1
    1.1.1. Definición . . . 1
    1.1.2. Propósito y Filosofía . . . 1
    1.1.3. Historia y Evolución . . . 1
    1.1.4. Casos de Uso Principales . . . 2
    1.1.5. Ventajas Detalladas . . . 3
    1.1.6. Limitaciones Honestas . . . 7
    1.1.7. Conclusión del Módulo 1 . . . 11
  1.2. 1.2. Arquitectura de AutoGen . . . 11
    1.2.1. Conceptos Fundamentales . . . 11
    1.2.2. Tipos de Agentes y sus Características . . . 13
    1.2.3. Flujos de Comunicación . . . 18
    1.2.4. Modelos de Lenguaje Soportados . . . 19
  1.3. 1.3. Instalación y Configuración . . . 20
    1.3.1. Requisitos Previos . . . 20
    1.3.2. Instalación Paso a Paso . . . 21
    1.3.3. Configuración de Proveedores . . . 21
    1.3.4. Entorno de Desarrollo Recomendado . . . 22
2. Módulo 2: Fundamentos de Agentes . . . 25
  2.1. 2.1. Agentes Básicos . . . 25
    2.1.1. AssistantAgent - Detallado . . . 25
    2.1.2. UserProxyAgent - Detallado . . . 26
    2.1.3. Agentes Personalizados . . . 28
  2.2. 2.2. Conversaciones entre Agentes . . . 29
    2.2.1. Iniciación de Conversaciones . . . 29
    2.2.2. Flujos de Comunicación Avanzados . . . 34
  2.3. 2.3. Configuración Avanzada . . . 35
    2.3.1. Parámetros de Configuración Completos . . . 35
    2.3.2. Gestión de Contexto . . . 36
3. Módulo 3: Patrones de Conversación . . . 39
  3.1. 3.1. Conversaciones Dos Agentes . . . 39
    3.1.1. Patrón Pregunta-Respuesta . . . 39
    3.1.2. Resolución de Problemas Iterativa . . . 40
    3.1.3. Validación de Respuestas . . . 41
4. Módulo 4: Capacidades Avanzadas . . . 43
  4.1. 4.1. Ejecución de Código . . . 43
    4.1.1. Code Execution Segura . . . 43

4.1.2. Manejo Avanzado de Errores en Code Execution . . . 48
4.1.3. Mejores Prácticas en Code Execution . . . 50
4.2. 4.2. Function Calling . . . 51
4.2.1. Definición y Registro de Funciones . . . 51
4.2.2. Invocación Automática de Funciones . . . 52
4.2.3. Parámetros y Esquemas . . . 53
5. Módulo 5: Agentes Especializados . . . 55
5.1. 5.1. Agentes Personalizados Avanzados . . . 55
5.1.1. Creación de Clases de Agentes . . . 55
5.1.2. Patrones de Diseño . . . 56
5.2. 5.2. Agentes Multi-Modalidad . . . 57
5.3. 5.3. Agentes de Memoria Extendida . . . 60
5.4. 5.4. Agentes de Aprendizaje Autónomo . . . 63
5.5. 5.5. Agentes de Coordinación Multi-Agente . . . 67
6. Módulo 6: Optimización y Costos . . . 73
6.1. 6.1. Gestión de Tokens . . . 73
6.2. 6.2. Caché y Reutilización . . . 76
6.3. 6.3. Modelos Locales . . . 80
6.4. 7.1. Asistente de Programación . . . 83
6.5. 7.2. Análisis de Datos . . . 87
6.6. 7.3. Automatización de Tareas . . . 90
6.7. 7.4. Investigación y Síntesis . . . 93
6.8. 8.1. Estrategias de Testing . . . 97
6.9. 8.2. Debugging . . . 101
6.10. 8.3. Evaluación de Calidad . . . 105
7. Módulo 9: Despliegue en Producción . . . 111
7.1. 9.1. Arquitectura de Producción . . . 111
7.2. 9.2. Seguridad . . . 115
7.3. 9.3. Monitorización . . . 120
7.4. 9.4. CI/CD . . . 124
8. Módulo 10: Integraciones . . . 129
8.1. 10.1. Frameworks y Librerías . . . 129
8.2. 10.2. APIs y Servicios . . . 132
8.3. 10.3. Bases de Datos . . . 137
9. Módulo 11: Proyecto Final . . . 141
9.1. 11.1. Definición del Proyecto . . . 141
9.2. 11.2. Desarrollo . . . 145
9.3. 11.3. Presentación . . . 149
10. Anexos . . . 153
10.1. A. Recursos Adicionales . . . 153
10.2. B. Glosario de Términos . . . 153

10.3. C. Mejores Prácticas 153
10.4. D. Troubleshooting 155

Chapter 1. Módulo 1: Introducción a AutoGen

1.1. 1.1. ¿Qué es AutoGen?

1.1.1. Definición

AutoGen es un framework de código abierto desarrollado por Microsoft que simplifica la creación de aplicaciones con Modelos de Lenguaje de Gran Escala (LLMs).

AutoGen proporciona una **abstracción de alto nivel** *para trabajar con sistemas multi-agente donde los agentes pueden:*

- Mantener conversaciones naturales entre sí
- Compartir información y conocimiento
- Ejecutar código de forma segura
- Acceder a herramientas y APIs externas
- Automatizar flujos de trabajo complejos
- Tomar decisiones basadas en contexto

El concepto central es que **los agentes pueden funcionar de manera autónoma o colaborativa**, permitiendo patrones sofisticados de interacción sin necesidad de escribir lógica de orquestación compleja.

1.1.2. Propósito y Filosofía

La filosofía de AutoGen se basa en tres principios clave:

1. Simplicidad: Abstraer la complejidad de los sistemas multi-agente en APIs intuitivas

2. Flexibilidad: Soportar múltiples patrones de interacción y configuraciones sin imponer estructuras rígidas

3. Extensibilidad: Permitir crear agentes personalizados y comportamientos específicos del dominio

1.1.3. Historia y Evolución

```
2023 (Q1-Q2): Presentación inicial de AutoGen
  - Enfoque: sistemas conversacionales multi-agente
  - Primeros casos de uso: debugging automático y code generation
  - Comunidad académica como early adopters

2023 (Q3-Q4): Expansión de capacidades
  - Introducción: Function Calling (invocación de funciones)
  - Introducción: Ejecución de código nativa
```

```
  - Soporte para múltiples proveedores de LLM

2024 (Presente): Consolidación y optimización
  - GroupChat para coordinación de múltiples agentes
  - Optimizaciones de rendimiento y caché
  - Mejoras en manejo de errores y recuperación
  - Integración con más proveedores (Ollama, local, etc.)
  - Comunidad activa con más de 20k stars en GitHub
```

1.1.4. Casos de Uso Principales

Caso 1: Asistente de Programación

```
Flujo:
1. Desarrollador proporciona requisito o describe error
2. Agente generador escribe código
3. Agente tester ejecuta y verifica
4. Agente revisor analiza calidad y seguridad
5. Se itera hasta solución aceptable
```

Caso 2: Análisis de Datos Automático

```
Flujo:
1. Usuario carga dataset
2. Agente explorador examina estructura y características
3. Agente visualizador genera gráficos
4. Agente estadístico calcula métricas clave
5. Agente reportero sintetiza hallazgos
```

Caso 3: Automatización Empresarial

```
Flujo:
1. Agente lector extrae información de documentos
2. Agente procesador valida y transforma datos
3. Agente integrador comunica con sistemas legacy
4. Agente reportero notifica cambios y excepciones
```

Caso 4: Investigación y Síntesis

```
Flujo:
1. Agente investigador busca en múltiples fuentes
2. Agente evaluador verifica credibilidad
3. Agente sintetizador integra hallazgos
4. Agente crítico cuestiona conclusiones
5. Resultado: reporte balanceado y documentado
```

Caso 5: Sistemas de Soporte Técnico

```
Flujo:
1. Agente clasificador entiende el problema
2. Agente diagnosticador recolecta información
3. Agente solucionador propone soluciones
4. Agente de escalación involucra expertos si necesario
```

1.1.5. Ventajas Detalladas

1. Abstracción Multi-Agente:

La ventaja principal es que AutoGen oculta mucha complejidad subyacente. Sin AutoGen, coordinar múltiples agentes requeriría:

- Escribir manualmente lógica de routing de mensajes
- Gestionar el historial de conversación
- Implementar mecanismos de término
- Sinronizar estado entre agentes

Con AutoGen, todo esto se maneja automáticamente. Ejemplo de diferencia:

```
# SIN AutoGen (código manual complicado)
class ManualMultiAgentSystem:
    def __init__(self):
        self.messages = []
        self.agent_states = {}

    def send_message(self, from_agent, to_agent, content):
        msg = {
            "from": from_agent,
            "to": to_agent,
            "content": content,
            "timestamp": datetime.now()
        }
        self.messages.append(msg)
        # Lógica manual para enrutar, procesar, etc.

# CON AutoGen (código simple y limpio)
assistant = AssistantAgent(name="Asistente", llm_config=config)
user_proxy.initiate_chat(assistant, message="Hola")  # ¡Todo maneja automáticamente!
```

Abstracción de temas:

- APIs simples y Pythónicas para conceptos complejos
- Declarativo en lugar de imperativo
- Menos código boilerplate

- Mejor mantenibilidad

2. Flexibilidad en Proveedores:

Uno de los puntos fuertes es que AutoGen soporta múltiples proveedores. Puedes cambiar de un proveedor a otro sin reescribir tu código:

```
# SIN AutoGen (dependencia bloqueada)
import openai
openai.api_key = "sk-..."  # Bloqueado a OpenAI

# CON AutoGen (intercambiable)
config_1 = {"model": "gpt-4", "api_key": "sk-..."}       # OpenAI
config_2 = {"model": "claude-3", "api_key": "sk-ant-..."} # Anthropic
config_3 = {"model": "mistral", "api_base": "http://localhost:11434"}  # Local

# El código es idéntico para todas las opciones:
assistant = AssistantAgent(
    name="Asistente",
    llm_config={"config_list": [config_1]}  # O config_2, O config_3
)
```

Ventajas prácticas:

- **Experimenta** con diferentes modelos sin cambiar código
- **Fallback automático** si un modelo falla (proporciona lista de modelos)
- **Optimiza costos** usando modelos más baratos cuando sea posible
- **Migra** a nuevos proveedores fácilmente
- **Protege** inversión en código si cambian APIs

Estrategia de fallback:

```
config_list = [
    {"model": "gpt-4", "api_key": "..."},           # Intenta primero (mejor pero
caro)
    {"model": "gpt-3.5-turbo", "api_key": "..."},   # Si falla, intenta segundo
(barato)
    {"model": "mistral", "api_base": "http://localhost:11434"}  # Fallback local
]

# Si gpt-4 falla por rate limit, automáticamente intenta gpt-3.5-turbo
assistant = AssistantAgent(llm_config={"config_list": config_list})
```

3. Ejecución de Código Nativa:

AutoGen puede generar código automáticamente y ejecutarlo. Esto es revolucionario porque:

- El agente puede validar su propio código

- Puede iterar si hay errores
- Captura automáticamente errores y retroalimenta al agente
- Crea un loop automático de prueba-error

Ejemplo del ciclo:

```
CICLO DE AUTOGEN PARA CODE GENERATION:

1. USER: "¿Cuál es la raíz cuadrada de 625?"

2. ASSISTANT (genera código):
   python
   import math
   result = math.sqrt(625)
   print(result)


3. USER PROXY (ejecuta):
   result = 25.0

4. ASSISTANT (analiza resultado):
   "La raíz cuadrada de 625 es 25"

VENTAJA: Si hubiera error, el agente vería el error y lo corregiría.
```

Opciones de sandboxing:

- Local: Rápido, acceso a archivos locales, riesgoso
- Docker: Seguro, aislado, más lento
- Remoto: Distribuido, requiere infraestructura
- Sin ejecución: Análisis de código sin ejecutar

4. Documentación y Comunidad:

AutoGen tiene:

- Documentación oficial extensa (~5000+ páginas)
- +150 ejemplos en GitHub
- Comunidad Discord activa (~5000+ miembros)
- Papers académicos (Microsoft Research)
- Contribuciones constantes (updates semanales)

Ventajas:

- Problemas comunes ya resueltos
- Ejemplos para casi cualquier caso de uso
- Soporte comunitario rápido

- Framework activamente mantenido

5. Escalabilidad:

AutoGen fue diseñado desde el inicio para escalar:

```
ESCALABILIDAD EN AUTOGEN:

Pequeño (1-2 agentes):
  - Conversación simple usuario-asistente
  - Ejemplo: Chatbot básico
  - Overhead: Mínimo
  - Complejidad: Baja

Medio (3-10 agentes):
  - GroupChat coordinado
  - Ejemplo: Revisión de código multi-persona
  - Overhead: Moderado
  - Complejidad: Media

Grande (50-500 agentes):
  - Hierarchical chats anidados
  - Comunicación especializada
  - Ejemplo: Sistema de soporte con múltiples especializaciones
  - Overhead: Significativo pero manejable

Masivo (100+):
  - Requiere optimizaciones personalizadas
  - Message queue (Redis, RabbitMQ)
  - Load balancing
  - Persistencia de estado

ARCHITECTURA ESCALABLE:

User Request
      □
   Manager 1 (Coordinador principal)
      □
    ┌──────┴──────┬──────────────────┬──────────────────┐
   □     □          □          □
 Group1 Group2   Group3    Group4
(Código)(Test) (Análisis)(Revisión)
   □     □          □          □
[A,B,C][D,E,F] [G,H,I]   [J,K,L]

Cada grupo opera independientemente
Manager1 coordina comunicación entre grupos
```

Técnicas de optimización:

- Message batching (agrupar mensajes)

- Caché de respuestas repetidas
- Async execution (ejecución paralela)
- Connection pooling (reutilizar conexiones)

1.1.6. Limitaciones Honestas

1. Costos de API:

Es importante ser honesto sobre los costos:

```
ANÁLISIS DE COSTOS (Actualizado 2024):

OpenAI GPT-4:
  - Entrada: $0.03 por 1K tokens
  - Salida: $0.06 por 1K tokens

  Ejemplos prácticos:
  • Pregunta simple: ~$0.001-0.01
  • Análisis código 100 líneas: ~$0.05-0.20
  • RAG con 10 búsquedas: ~$0.50-2.00
  • Sesión debugging completa: ~$5-50

  COSTO MENSUAL ESTIMADO:
  - Uso casual (1 hora/día): $10-30
  - Uso profesional (8 horas/día): $200-500
  - Producción de alto volumen: $1000-5000+

OpenAI GPT-3.5-turbo (más barato):
  - Entrada: $0.0005 por 1K tokens (60x menos)
  - Salida: $0.0015 por 1K tokens (40x menos)

  COSTO MENSUAL ESTIMADO:
  - Mismo uso casual: $0.50-1.50
  - Mismo uso profesional: $5-15
  - Mismo uso producción: $50-200

Anthropic Claude 3 (Premium):
  - Opus (más poderoso): $15 por 1M entrada, $75 por 1M salida
  - Sonnet (balance): $3 por 1M entrada, $15 por 1M salida
  - Haiku (rápido): $0.25 por 1M entrada, $1.25 por 1M salida

Ollama Local (sin costo):
  - Costo API: $0
  - Costo infraestructura: Hardware local
  - Velocidad: 5-10x más lento que OpenAI
  - Calidad: ~80% de GPT-4
  - Privacidad: 100% local

RECOMENDACIONES DE OPTIMIZACIÓN COSTOS:
```

```
1. Por fase del desarrollo:
   Exploración:    Use Ollama local o GPT-3.5-turbo
   Prototipo:      Use GPT-3.5-turbo con fallback a GPT-4
   Producción:     Use GPT-4 pero limite llamadas

2. Técnicas prácticas:
   - Caché de respuestas (seed = 42 para determinismo)
   - Limitar max_tokens (500 en vez de 2000)
   - Batching de operaciones
   - Usar modelos más baratos para tareas simples
   - Implementar rate limiting local

3. Monitoreo de gastos:
   import logging
   logger = logging.getLogger("cost_tracker")

   total_tokens_input = 0
   total_tokens_output = 0

   # Registrar después de cada llamada
   total_cost = (total_tokens_input * 0.03 + total_tokens_output * 0.06) / 1000
```

2. Control de Flujo:

Los agentes pueden "desviarse" de lo esperado. Esto es una característica y un desafío:

```
PROBLEMA: Falta de Control Determinista

Los agentes usan LLMs, que son estocásticos por naturaleza.
Incluso con temperature=0, pueden variar en respuestas.

EJEMPLO PROBLEMÁTICO:

Prompt: "Escribe un print statement en Python"

Respuesta posible 1: print("Hola mundo")
Respuesta posible 2: print("Hello")
Respuesta posible 3: No escribe print, explica qué es

IMPACTO EN FLUJOS COMPLEJOS:

En sistemas simples (1-2 agentes): No es problema
En sistemas complejos (10+ agentes): Impacto exponencial

ESTRATEGIAS DE MITIGACIÓN:

1. Prompt Engineering Cuidadoso (Específico y explícito)
2. Validación de Respuestas (Verificar formato y contenido)
3. Reintentos Inteligentes (Repetir hasta aceptable)
```

```
4. Structured Output (Usar JSON en lugar de texto libre)
```

3. Latencia y Rendimiento:

```
ANÁLISIS DE LATENCIA:

OpenAI GPT-4 (remoto):
  - Latencia típica: 2-10 segundos por llamada
  - En peak: Puede alcanzar 30-60 segundos
  - Causas: Network, queue en API, procesamiento

Ollama Local (Mistral en GPU):
  - Latencia típica: 5-15 segundos por llamada
  - Predictible, sin dependencia de red
  - Más rápido si está cacheado

IMPACTO EN CADENAS:

Conversación simple (3 turnos):
  Tiempo total  3 * latencia_promedio
  OpenAI: ~15-30 segundos
  Ollama: ~15-45 segundos

Conversación grupal (5 agentes, 10 turnos):
  Tiempo total: Puede ser 5-10 minutos fácilmente
    Inaceptable para interfaces interactivas
  ✓ Aceptable para procesamiento batch

OPTIMIZACIONES:

1. Paralelización de agentes:
   - Ejecutar agentes independientes en paralelo
   - Reduce tiempo exponencial a tiempo lineal

2. Modelo más rápido + más modelos lentos:
   - GPT-3.5-turbo es 10x más rápido que GPT-4
   - Reserva GPT-4 solo para decisiones críticas

3. Caché y reutilización:
   - Si pregunta es similar, reutilizar respuesta anterior
   - Implementar LRU cache de prompts

4. Async/Await:
   - No esperar respuesta antes de preparar siguiente
   - Python asyncio puede ayudar aquí
```

4. Dependencias Externas y Mantenimiento:

- AutoGen requiere Python 3.8+

- Dependencias: requests, openai/anthropic/etc
- Cambios en APIs pueden romper código

```
DEPENDENCIAS Y COMPATIBILIDAD:

AutoGen dependencies:
- Python 3.8, 3.9, 3.10, 3.11
- typing_extensions (para type hints)
- openai>=1.0.0 (si usas OpenAI)
- anthropic>=0.7.0 (si usas Claude)
- azure-openai>=1.0.0 (si usas Azure)

RIESGOS DE ROMPER CAMBIOS:

Riesgo 1: Cambios en APIs de proveedores
  OpenAI cambió su API completamente en Nov 2023
  Código antiguo dejó de funcionar

  Solución: Version pinning en requirements.txt
  openai==1.3.0  # Versión específica

Riesgo 2: Deprecación en AutoGen
  Funcionalidades marcadas como deprecated eventualmente se removerán

  Solución: Monitorear changelog y migrar proactivamente

Riesgo 3: Incompatibilidad con modelos nuevos
  Nuevos modelos pueden tener requisitos diferentes

  Solución: Abstracta la configuración del modelo

EJEMPLO DE CÓDIGO RESILIENTE:

# requirements.txt
pyautogen==0.2.5
openai==1.3.0
python-dotenv==1.0.0

# En código: usar abstractión
config = get_model_config("gpt-4")  # En lugar de hardcodear

def get_model_config(model_name):
    if model_name == "gpt-4":
        return {"model": "gpt-4", "api_key": get_key()}
    elif model_name == "gpt-3.5-turbo":
        return {"model": "gpt-3.5-turbo", "api_key": get_key()}
    # Fácil de actualizar si modelos cambian
```

1.1.7. Conclusión del Módulo 1

AutoGen es un framework poderoso que simplifica significativamente la construcción de sistemas multi-agente.

Sus fortalezas principales son:

- **Abstracción elegante** de complejidad multi-agente
- **Flexibilidad** en elección de modelos y proveedores
- **Código limpio** y facilidad de uso
- **Comunidad activa** y documentación completa
- **Casos de uso reales** y probados

Pero requiere consideración de:

- **Costos** potencialmente altos con APIs remotas
- **Latencia** puede ser problema en sistemas interactivos
- **Determinismo** limitado en LLMs
- **Dependencias externas** que pueden cambiar

En conclusión: **Ideal para prototipos, investigación, y sistemas donde el costo y latencia son aceptables. No ideal para aplicaciones críticas en tiempo real con presupuesto limitado.**

1.2. 1.2. Arquitectura de AutoGen

1.2.1. Conceptos Fundamentales

La arquitectura de AutoGen se construye sobre conceptos clave:

1. Agentes (Agents): Entidades autónomas con memoria, capacidades y personalidad. Cada agente es un "actor" en el sistema.

```
# Tipos de agentes base:
AssistantAgent        # Piensa y genera respuestas
UserProxyAgent        # Representa al usuario
GroupChatManager      # Coordina múltiples agentes
CustomAgent          # Personalizado por el usuario

# Cada agente tiene:
- name: identificador único
- system_message: instrucciones de comportamiento
- llm_config: configuración del modelo
- tools: funciones que puede ejecutar
- memory: historial de conversación
```

2. Mensajes (Messages): Unidades de comunicación estructuradas.

```
# Estructura de mensaje
{
    "role": "user|assistant",
    "content": "El contenido del mensaje",
    "name": "nombre_del_agente",
    "function_call": {...},  # Opcional: para function calling
    "tool_use": {...}        # Opcional: para herramientas
}

# El historial mantiene todo el contexto
messages = [
    {"role": "user", "content": "¿Qué es Python?"},
    {"role": "assistant", "content": "Python es un lenguaje..."},
    {"role": "user", "content": "¿Cuáles son sus usos?"},
    {"role": "assistant", "content": "Los usos principales son..."}
]
```

3. Funciones y Herramientas (Functions & Tools): Capacidades que los agentes pueden ejecutar.

```
# Ejemplo: función que un agente puede invocar
def calculate_sales_total(items: List[float], tax_rate: float = 0.08):
    """
    Calcular total de ventas con impuesto

    Args:
        items: Lista de precios
        tax_rate: Tasa de impuesto (default 8%)

    Returns:
        Total incluyendo impuesto
    """
    subtotal = sum(items)
    tax = subtotal * tax_rate
    return subtotal + tax

# Los agentes pueden:
# 1. Decidir si necesitan esta función
# 2. Proporcionarle los parámetros correctos
# 3. Usar el resultado en su respuesta
```

4. Configuración (Configuration): Define cómo se comportan los agentes.

```
# Configuración completa
llm_config = {
    # Proveedor y modelo
    "config_list": [
        {
            "model": "gpt-4",
```

```
            "api_key": "...",
            "api_base": "https://api.openai.com/v1"
        }
    ],

    # Parámetros de generación
    "temperature": 0.7,      # Creatividad (0-1)
    "top_p": 0.9,           # Nucleus sampling
    "max_tokens": 2000,     # Longitud máxima
    "timeout": 120,         # Segundos

    # Comportamiento
    "cache_seed": 42,       # Para reproducibilidad
}
```

1.2.2. Tipos de Agentes y sus Características

AssistantAgent - El "Pensador"

```
from autogen import AssistantAgent

assistant = AssistantAgent(
    name="Programador",
    system_message="""Eres un experto programador.
    Cuando se te pide escribir código:
    1. Analiza el requisito cuidadosamente
    2. Escribe código limpio y eficiente
    3. Incluye manejo de errores
    4. Documenta tu código""",
    llm_config=llm_config,
    human_input_mode="NEVER"  # No pide confirmación
)

# Características:
# - Usa LLM para generar respuestas
# - Puede ejecutar funciones
# - Mantiene contexto de conversación
# - No espera entrada del usuario
```

Parámetros Configurables de AssistantAgent:

```
AssistantAgent(
    # Identidad básica
    name="AsistenteIA",                          # Nombre único, sin espacios
    system_message="Eres un experto en Python",  # Instrucciones del comportamiento

    # Configuración del LLM
    llm_config={
```

```
        "config_list": [{"model": "gpt-4", "api_key": "..."}],
        "temperature": 0.7,                      # 0=determinista, 1=creativo
        "top_p": 0.9,                           # Nucleus sampling
        "max_tokens": 2000,                      # Máximo tokens de respuesta
        "timeout": 120,                         # Segundos antes de timeout
    },

    # Comportamiento de entrada
    human_input_mode="NEVER",                    # NEVER, ALWAYS, TERMINATE
    max_consecutive_auto_reply=10,               # Límite de respuestas consecutivas

    # Funciones que puede ejecutar
    function_map={
        "calculate": my_calculate_func,
        "search": my_search_func,
    },

    # Condición personalizada de término
    is_termination_msg=lambda x: "TERMINADO" in x.get("content", ""),

    # Parámetros opcionales
    default_auto_reply="",                       # Respuesta por defecto si sin idea
    description="Agente especializado en Python"  # Para logging/debug
)
```

Opciones de comportamiento:

```
COMPORTAMIENTO 1: Asistente determinista (reproducible)
  temperature: 0.0
  top_p: 0.1
  cache_seed: 42

  Caso de uso: Tareas que deben ser idénticas siempre
  Ejemplo: Generación de datos sintéticos con seed fijo

COMPORTAMIENTO 2: Asistente creativo (variado)
  temperature: 1.0-1.2
  top_p: 0.95

  Caso de uso: Brainstorming, generación de ideas
  Ejemplo: Sugerencias de nombres, ideas de marketing

COMPORTAMIENTO 3: Asistente técnico preciso (balanceado)
  temperature: 0.3-0.5
  top_p: 0.9

  Caso de uso: Tareas técnicas importantes
  Ejemplo: Análisis de código, debugging

COMPORTAMIENTO 4: Asistente dialógico (conversacional)
```

```
  temperature: 0.7
  top_p: 0.85
  max_tokens: 3000

  Caso de uso: Conversaciones naturales
  Ejemplo: Customer service, tutorías
```

UserProxyAgent - El "Usuario"

```
from autogen import UserProxyAgent

user_proxy = UserProxyAgent(
    name="Usuario",
    human_input_mode="TERMINATE",  # ALWAYS, NEVER, TERMINATE
    code_execution_config={
        "work_dir": "./workspace",
        "use_docker": False  # O True para aislamiento completo
    }
)

# Características:
# - Representa al usuario en la conversación
# - Puede ejecutar código generado
# - Proporciona feedback
# - Control sobre cuándo terminar
```

Parámetros completos de UserProxyAgent:

```
UserProxyAgent(
    # Identidad
    name="Usuario",
    system_message="Eres usuario prudente",  # Opcional, influye en comportamiento

    # CONTROL MÁS IMPORTANTE: Modo de entrada
    human_input_mode="TERMINATE",            # Cómo solicita confirmación

    # Ejecución de código
    code_execution_config={
        "work_dir": "./workspace",           # Directorio de trabajo
        "use_docker": False,                 # Aislamiento con Docker
        "docker_image": "python:3.9",        # Imagen si use_docker=True
        "timeout": 30,                       # Máximo segundos para ejecutar
        "last_n_messages": 3,                # Historial a ejecutar
    },

    # Parámetros de conversación
    max_consecutive_auto_reply=10,           # Cuánto puede responder solo
    llm_config=False,                        # UserProxy típicamente NO tiene LLM
    # (pero es opcional, puede tener si necesitas comportamiento especial)
```

```
    # Validación
    is_termination_msg=lambda x: "LISTO" in x.get("content", ""),
)
```

Modos de Entrada del Usuario Comparados:

```
Modo "ALWAYS":
   ┌──────────────────────────────────────────────────────────────┐
   │ Usuario: "Escribe código"       │
   ├──────────────────────────────────────────────────────────────┤
   │ Assistant: "def hola(): ..."     │
   ├──────────────────────────────────────────────────────────────┤
   │ Sistema: "Ejecutar? (y/n/s)"    │
   │ Usuario: "y"  ← SIEMPRE PREGUNTA │
   └──────────────────────────────────────────────────────────────┘

  Interacciones: 7-8 mensajes para tarea simple
  Tiempo: Lento (requiere confirmación manual)
  Control: Máximo
  Ideal para: Desarrollo, testing interactivo

  Código:
  user_proxy = UserProxyAgent(
      human_input_mode="ALWAYS"
  )

---

Modo "NEVER":
   ┌──────────────────────────────────────────────────────────────┐
   │ Usuario: "Escribe código"       │
   ├──────────────────────────────────────────────────────────────┤
   │ Assistant: "def hola(): ..."     │
   ├──────────────────────────────────────────────────────────────┤
   │ System: EJECUTA AUTOMÁTICO ← NO PREGUNTA │
   │ Output: "Éxito: resultado..."   │
   └──────────────────────────────────────────────────────────────┘

  Interacciones: 2-3 mensajes para tarea simple
  Tiempo: Rápido (automatizado)
  Control: Mínimo
  Ideal para: Producción, automatización
  Riesgo: Puede ejecutar código malicioso sin verificar

  Código:
  user_proxy = UserProxyAgent(
      human_input_mode="NEVER"
  )
```

```
---

Modo "TERMINATE":
   ┌──────────────────────────────────────────────────────────────────┐
   │ Usuario: "Escribe código"        │
   ├──────────────────────────────────────────────────────────────────┤
   │ Assistant: "def hola(): ..."      │
   │ User: "Parece bien"              │
   ├──────────────────────────────────────────────────────────────────┤
   │ System: AMBOS dicen LISTO?       │
   │ SI → EJECUTA | NO → CONTINÚA     │
   └──────────────────────────────────────────────────────────────────┘

  Interacciones: 3-4 mensajes típicamente
  Tiempo: Moderado
  Control: Balance
  Ideal para: Balance entre automación y control

  Código:
  user_proxy = UserProxyAgent(
      human_input_mode="TERMINATE"
  )

RECOMENDACIÓN:

 ├── Desarrollo local: ALWAYS (máximo control)
 ├── Sistema batch: NEVER (máxima automatización)
 └── Sistema interactivo en producción: TERMINATE (balance)
```

GroupChatManager - El "Coordinador"

```
from autogen import GroupChat, GroupChatManager

# Crear múltiples agentes especializados
coder = AssistantAgent(name="Coder", ...)
reviewer = AssistantAgent(name="Reviewer", ...)
tester = AssistantAgent(name="Tester", ...)

# Crear el chat grupal
groupchat = GroupChat(
    agents=[coder, reviewer, tester, user_proxy],
    messages=[],
    max_round=20  # Máximo turnos para evitar loops infinitos
)

# Crear el manager
manager = GroupChatManager(
    groupchat=groupchat,
    llm_config=llm_config
)
```

```
# Ahora los agentes pueden debatir entre sí
user_proxy.initiate_chat(
    manager,
    message="Necesito un módulo de validación"
)
```

1.2.3. Flujos de Comunicación

Patrón Simple (Dos Agentes):

```
Usuario (UserProxyAgent)
    □ envía: "Escribe un hola mundo"
    □
Asistente (AssistantAgent)
    □ recibe, procesa, genera código
    □ envía: "def main(): print('Hola mundo')"
    □
Usuario
    □ ejecuta código
    □ envía: "Código ejecutado exitosamente"
    □
Asistente
    □ reconoce término
    □ conversación termina
```

Patrón Complejo (Multi-Agente):

```
          ┌──────────────────────────────────┐
          │   UserProxy      │
          │  (Coordinador)   │
          └──────────────────┬───────────────┘
                   │
         ┌─────────────────┼─────────────────┐
        □         □         □
     ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
     │  Coder   │  │Reviewer│  │  Tester  │
     └──────┬───────┘  └──────┬───────┘  └──────┬───────┘
          │  debate entre sí
          │  intercambian ideas
          │  iteran hasta acuerdo
         □
    UserProxy recibe resultado
    Conversación termina
```

1.2.4. Modelos de Lenguaje Soportados

Proveedores Principales:

```
# 1. OpenAI (GPT-4, GPT-3.5-turbo)
openai_config = {
    "config_list": [{
        "model": "gpt-4",
        "api_key": "sk-...",
        "api_type": "openai"
    }]
}

# 2. Azure OpenAI
azure_config = {
    "config_list": [{
        "api_type": "azure",
        "api_key": "...",
        "api_base": "https://xxx.openai.azure.com/",
        "api_version": "2024-02-15-preview",
        "model": "gpt-4-deployment"
    }]
}

# 3. Anthropic Claude
claude_config = {
    "config_list": [{
        "model": "claude-3-opus",
        "api_key": "sk-ant-...",
        "api_type": "anthropic"
    }]
}

# 4. Ollama Local
ollama_config = {
    "config_list": [{
        "model": "mistral",
        "api_base": "http://localhost:11434/v1",
        "api_type": "ollama"
    }]
}

# 5. Modelos Open Source Locales
local_config = {
    "config_list": [{
        "model": "local-llama2",
        "api_base": "http://localhost:8000",
    }]
}
```

Comparativa de Modelos:

```
OpenAI (GPT-4):
  - Costo: ~$0.03/1K tokens entrada, $0.06/1K salida
  - Velocidad: ~2-5 segundos por respuesta
  - Calidad: Excelente, estado del arte
  - Privacidad: Datos enviados a OpenAI

GPT-3.5-turbo:
  - Costo: ~$0.0005/1K entrada, $0.0015/1K salida (mucho más barato)
  - Velocidad: ~1-3 segundos
  - Calidad: Buena, suficiente para muchos casos
  - Privacidad: Datos enviados a OpenAI

Claude (Anthropic):
  - Costo: Comparable a GPT-4
  - Velocidad: ~2-5 segundos
  - Calidad: Muy buena, excelente en razonamiento
  - Privacidad: Datos a Anthropic

Ollama Local (Mistral, Llama2):
  - Costo: $0 después de descargar
  - Velocidad: ~5-15 segundos (depende hardware)
  - Calidad: Buena pero inferior a GPT-4
  - Privacidad: Total, todo local
  - Ventaja: Sin conexión a internet después de setup
```

1.3. 1.3. Instalación y Configuración

1.3.1. Requisitos Previos

Hardware:

- CPU: Cualquiera (i5 o superior recomendado)
- RAM: 8GB mínimo, 16GB recomendado
- GPU: Opcional pero recomendado para Ollama

Software:

- Python: 3.8, 3.9, 3.10 o 3.11
- pip: Gestor de paquetes de Python
- git: Para clonar repositorios

Conocimientos:

- Python básico (variables, funciones, classes)
- APIs REST (opcional pero útil)
- Prompting para LLMs (aprenderás mientras usas)

1.3.2. Instalación Paso a Paso

Opción 1: Instalación Mínima (con OpenAI)

```
# 1. Verificar Python
python3 --version  # Debe ser 3.8+

# 2. Crear entorno virtual (recomendado)
python3 -m venv autogen_env
source autogen_env/bin/activate  # En Windows: autogen_env\Scripts\activate

# 3. Instalar AutoGen
pip install pyautogen

# 4. Instalar dependencias opcionales
pip install requests  # Para Ollama

# 5. Verificar instalación
python3 -c "import autogen; print(autogen.__version__)"
```

Opción 2: Instalación Completa (con Extras)

```
# Con todas las características
pip install "pyautogen[extra]"

# Esto incluye:
# - Soporte para Docker (ejecución segura de código)
# - Herramientas adicionales
# - Dependencias de desarrollo

# Para desarrollo (con pre-commit hooks)
pip install "pyautogen[dev]"
```

Opción 3: Desde Código Fuente

```
# Clonar repositorio
git clone https://github.com/microsoft/autogen.git
cd autogen

# Instalar en modo desarrollo
pip install -e .

# Útil para contribuir o usar versión más reciente
```

1.3.3. Configuración de Proveedores

OpenAI:

```
# 1. Crear cuenta en https://platform.openai.com
# 2. Generar API key en https://platform.openai.com/account/api-keys
# 3. Guardar en variable de entorno

export OPENAI_API_KEY="sk-..."
# O en Python:
import os
os.environ["OPENAI_API_KEY"] = "sk-..."
```

Archivo de Configuración (.env):

```
# archivo: .env
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
AZURE_OPENAI_API_KEY=...
AZURE_OPENAI_API_BASE=https://xxx.openai.azure.com

# Usar con python-dotenv:
from dotenv import load_dotenv
load_dotenv()
```

Ollama Local:

```
# 1. Instalar Ollama
curl -fsSL https://ollama.ai/install.sh | sh

# 2. Iniciar servidor
ollama serve

# 3. En otra terminal, descargar modelo
ollama pull mistral

# 4. Usar en AutoGen
config = {
    "config_list": [{
        "model": "mistral",
        "api_base": "http://localhost:11434/v1",
    }]
}
```

1.3.4. Entorno de Desarrollo Recomendado

Estructura de Proyecto:

```
mi_proyecto_autogen/
 ├─── venv/                    # Entorno virtual
```

```
├──── src/
│    ├──── agents/
│    │    ├──── assistant.py
│    │    ├──── reviewer.py
│    │    └──── __init__.py
│    ├──── tools/
│    │    ├──── code_execution.py
│    │    └──── __init__.py
│    └──── main.py
├──── examples/
│    ├──── basic_conversation.py
│    └──── multi_agent_workflow.py
├──── tests/
│    ├──── test_agents.py
│    └──── __init__.py
├──── configs/
│    ├──── openai_config.py
│    ├──── ollama_config.py
│    └──── azure_config.py
├──── .env                      # Variables de entorno
├──── .gitignore               # Git ignore patterns
├──── requirements.txt         # Dependencias
└──── README.md               # Documentación
```

IDE Recomendado:

```
VS Code (Recomendado):
  - Extensión: Python
  - Extensión: Pylance (type hints)
  - Extensión: autodocstring (generar docs)
  - Extensión: GitLens

PyCharm Community:
  - IDE especializado en Python
  - Debugging integrado
  - Refactoring automático

Jupyter Notebook:
  - Para prototipos rápidos
  - Ideal para explorar
  - Combina código y documentación
```

Herramientas de Desarrollo:

```
# Linting (verificar estilo)
pip install pylint

# Formateado automático
pip install black
```

```
# Type checking
pip install mypy

# Testing
pip install pytest

# Debugging
pip install ipdb

# Monitoreo
pip install py-spy
```

Archivo requirements.txt:

```
pyautogen>=0.2.0
python-dotenv>=1.0.0
requests>=2.31.0
openai>=1.0.0  # Para OpenAI
anthropic>=0.7.0  # Para Claude
azure-openai>=1.0.0  # Para Azure

# Desarrollo
pytest>=7.0.0
black>=23.0.0
pylint>=2.0.0
mypy>=1.0.0
ipdb>=0.13.0
```

Chapter 2. Módulo 2: Fundamentos de Agentes

2.1. 2.1. Agentes Básicos

2.1.1. AssistantAgent - Detallado

Qué es: Un agente que utiliza un LLM para generar respuestas inteligentes. Es el "cerebro" que piensa y decide.

Inicialización Básica:

```
from autogen import AssistantAgent

assistant = AssistantAgent(
    name="Asistente",
    llm_config={
        "config_list": [{"model": "gpt-4", "api_key": "..."}],
        "temperature": 0.7,
        "max_tokens": 2000
    }
)
```

Parámetros Clave:

```
AssistantAgent(
    # Identidad
    name="AsistenteIA",                    # Nombre único
    system_message="""Eres un experto    # Instrucciones base
                      en Python...""",

    # Configuración del LLM
    llm_config={
        "config_list": [...],             # Modelos disponibles
        "temperature": 0.7,               # Creatividad
        "top_p": 0.9,                     # Diversidad
        "max_tokens": 2000,               # Máximo output
    },

    # Comportamiento
    human_input_mode="NEVER",             # NEVER, ALWAYS, TERMINATE

    # Funciones que puede usar
    function_map={
        "calculate": calculate_func,
        "search": search_func
```

```
    },

    # Validación de respuestas
    is_termination_msg=lambda x:
        "TERMINAR" in x.get("content", "")
)
```

Opciones de comportamiento:

```
# 1. Assistant determinista (pocas variaciones)
assistant_deterministic = AssistantAgent(
    name="Determinista",
    llm_config={
        "temperature": 0.0,  # Siempre la misma respuesta
        "max_tokens": 500,   # Respuestas cortas
        "top_p": 0.1         # Opciones limitadas
    }
)

# 2. Assistant creativo (más variaciones)
assistant_creative = AssistentAgent(
    name="Creativo",
    llm_config={
        "temperature": 1.2,  # Muy variable
        "top_p": 0.95        # Muchas opciones posibles
    }
)

# 3. Assistant experto (completo)
assistant_expert = AssistantAgent(
    name="Experto",
    system_message="""Eres el mejor experto en Python del mundo.
    Cuando respondas:
    1. Sé muy preciso y técnico
    2. Explica el por qué de tus recomendaciones
    3. Sugiere alternativas si es relevante
    4. Menciona trade-offs
    5. Proporciona ejemplos de código cuando sea apropiado
    """,
    llm_config={
        "temperature": 0.3,  # Técnico, no creativo
        "max_tokens": 3000,
    }
)
```

2.1.2. UserProxyAgent - Detallado

Qué es: Representa al usuario en la conversación. Puede ejecutar código, proporcionar feedback, y controlar el flujo.

Inicialización:

```
from autogen import UserProxyAgent

user = UserProxyAgent(
    name="Usuario",
    human_input_mode="TERMINATE",
    code_execution_config={
        "work_dir": "./workspace",
        "use_docker": False
    }
)
```

Modos de Interacción:

```
# Modo 1: ALWAYS (Control Total)
user = UserProxyAgent(
    name="Usuario",
    human_input_mode="ALWAYS"
)

# Efecto: Después de cada respuesta del asistente,
# pide confirmación. Ideal para desarrollo.

# Ejemplo de interacción:
# Assistant: "Aquí está el código..."
# Sistema: "Proceder? (y/n/s): " <- espera entrada


# Modo 2: NEVER (Automatización Completa)
user = UserProxyAgent(
    name="Usuario",
    human_input_mode="NEVER"
)

# Efecto: Nunca pide confirmación.
# Ejecuta automáticamente. Riesgo: sin supervisión.


# Modo 3: TERMINATE (Balance)
user = UserProxyAgent(
    name="Usuario",
    human_input_mode="TERMINATE"
)

# Efecto: Solo pide si ambos agentes creen que debe terminar.
# Más seguro pero menos control.
```

Configuración de Ejecución de Código:

```
# Opción 1: Ejecutar localmente (rápido pero riesgoso)
user = UserProxyAgent(
    name="Usuario",
    code_execution_config={
        "work_dir": "./workspace",
        "use_docker": False
    }
)

# Opción 2: Ejecutar en Docker (seguro)
user = UserProxyAgent(
    name="Usuario",
    code_execution_config={
        "work_dir": "./workspace",
        "use_docker": True,
        "docker_image": "python:3.9"
    }
)

# Opción 3: Sin ejecución
user = UserProxyAgent(
    name="Usuario",
    code_execution_config=False  # No ejecutar código
)
```

2.1.3. Agentes Personalizados

Crear un Agente Especializado:

```
from autogen import AssistantAgent

class DataAnalystAgent(AssistantAgent):
    """Agente especializado en análisis de datos"""

    def __init__(self, name="DataAnalyst", **kwargs):
        system_msg = """Eres un experto analista de datos.
        Tu especialidad es:
.
        - Exploración de datos (pandas, numpy)
        - Visualización (matplotlib, seaborn)
        - Estadística descriptiva e inferencial
        - Machine learning básico

        Cuando recibas datos:
        1. Examina la estructura (shape, dtypes, missing values)
        2. Genera estadísticas descriptivas
        3. Crea visualizaciones relevantes
```

```
        4. Proporciona insights accionables"""

        super().__init__(
            name=name,
            system_message=system_msg,
            **kwargs
        )

# Uso:
analyst = DataAnalystAgent(
    llm_config={"config_list": [{"model": "gpt-4", "api_key": "..."}]}
)
```

Herencia y Composición:

```
# Opción 1: Herencia
class SecurityReviewerAgent(AssistantAgent):
    def __init__(self, **kwargs):
        super().__init__(
            system_message="Eres un experto en seguridad...",
            **kwargs
        )

# Opción 2: Composición
class PipelineAgent:
    def __init__(self):
        self.coder = AssistantAgent(...)
        self.reviewer = AssistantAgent(...)
        self.tester = AssistantAgent(...)

    def process(self, task):
        code = self.coder.generate_response(task)
        review = self.reviewer.generate_response(code)
        result = self.tester.generate_response(code)
        return {"code": code, "review": review, "test": result}
```

2.2. 2.2. Conversaciones entre Agentes

2.2.1. Iniciación de Conversaciones

Opción 1: Conversación Simple (Dos Agentes)

```
from autogen import AssistantAgent, UserProxyAgent

# Crear agentes
assistant = AssistantAgent(
    name="Asistente",
    system_message="Eres un experto en Python",
```

```
    llm_config=llm_config
)

user_proxy = UserProxyAgent(
    name="Usuario",
    human_input_mode="TERMINATE"
)

# Iniciar conversación
user_proxy.initiate_chat(
    assistant,
    message="¿Cuál es la diferencia entre map y filter?",
    max_consecutive_auto_reply=3  # Máximo 3 respuestas del assistant
)

# Flujo:
# 1. Usuario envía mensaje
# 2. Assistant responde
# 3. Usuario puede responder (si human_input_mode permite)
# 4. Itera hasta terminar
```

Opciones de initiate_chat:

```
user_proxy.initiate_chat(
    recipient=assistant,                          # A quién enviar el mensaje

    # El mensaje inicial
    message="Tu pregunta aquí",                   # String o None

    # Control de duración
    max_consecutive_auto_reply=5,                 # Máximo turnos de assistant

    # Parámetros opcionales
    clear_history=True,                          # Limpiar historial previo
    silent=False,                                # No mostrar mensajes en stdout
)

# Retorna: (último_mensaje_de_assistant, conversación_completa)
```

Ciclo de vida de una conversación:

```
INICIO:
  1. user_proxy.initiate_chat(...)

PRIMER TURNO:
  2. user_proxy → envía mensaje inicial
  3. assistant → recibe y procesa
  4. assistant → genera respuesta
  5. user_proxy ← recibe respuesta
```

```
TURNOS SUBSECUENTES (si no termina):
  6. user_proxy → analiza respuesta, decide continuar
  7. user_proxy → envía respuesta/pregunta
  8. assistant → recibe...
  [repite 3-8]

CONDICIONES DE TÉRMINO:

  a) max_consecutive_auto_reply alcanzado
  b) is_termination_msg devuelve True
  c) Error no recuperable
  d) Timeout
  e) Assistant dice "TERMINE" o similar

SALIDA:
  9. Conversación termina
  10. Ambos agentes mantienen historial en chat_history
```

Opción 2: Conversación Multigiro con Feedback Manual

```
# Conversación inicial
user_proxy.initiate_chat(
    assistant,
    message="¿Cuál es la raíz cuadrada de 625?",
    max_consecutive_auto_reply=2
)

# La conversación ya ocurrió. Ahora continuar:
user_proxy.send(
    message="¿Y cuál es el cuadrado de 25?",
    recipient=assistant,
    request_reply=True  # Solicitar respuesta inmediata
)

# Ventajas:
# - Control fino sobre cada turno
# - Puedes hacer lógica custom entre turnos
# - Acceso completo al historial

# Desventajas:
# - Más código
# - Manual, no automático
```

Opción 3: Conversación Grupal (Múltiples Agentes)

```
from autogen import GroupChat, GroupChatManager

# Crear múltiples agentes especializados
```

```
coder = AssistantAgent(
    name="Coder",
    system_message="""Eres un experto programador.
    Escribe código limpio, eficiente y bien documentado.""",
    llm_config=llm_config
)

reviewer = AssistantAgent(
    name="Reviewer",
    system_message="""Eres un revisor de código experto.
    Valida la calidad, seguridad y eficiencia del código.""",
    llm_config=llm_config
)

tester = AssistantAgent(
    name="Tester",
    system_message="""Eres un QA engineer.
    Prueba el código y reporta bugs potenciales.""",
    llm_config=llm_config
)

# Crear grupo chat
groupchat = GroupChat(
    agents=[coder, reviewer, tester, user_proxy],
    messages=[],
    max_round=10,           # Máximo turnos para evitar loops
    speaker_selection_method="round_robin"  # O "auto" o "manual"
)

# Crear manager para coordinar
manager = GroupChatManager(
    groupchat=groupchat,
    llm_config=llm_config
)

# Iniciar conversación grupal
user_proxy.initiate_chat(
    manager,
    message="Necesito una función para calcular factorial"
)

# Los agentes debaten entre sí hasta llegar a solución
```

Estrategias de Selección de Turnos (speaker_selection_method):

```
# Método 1: Round Robin (por turnos)
groupchat = GroupChat(
    agents=[coder, reviewer, tester, user_proxy],
    speaker_selection_method="round_robin"
)
```

```
# Flujo:
# Turno 1: Coder habla
# Turno 2: Reviewer habla
# Turno 3: Tester habla
# Turno 4: User habla
# [Repite]

# Ventajas:
#   ✓ Determinista y predecible
#   ✓ Todos participan equitativamente
# Desventajas:
#   ✗ No respeta contexto
#   ✗ Menos eficiente (puede hablar alguien irrelevante)


# Método 2: Automático (default)
groupchat = GroupChat(
    agents=[coder, reviewer, tester, user_proxy],
    speaker_selection_method="auto"
)

# Flujo:
# El LLM del manager decide quién debe hablar basado en contexto
# "Coder, revisa esos comentarios que hizo reviewer"
# Coder habla solo si es relevante

# Ventajas:
#   ✓ Contextualmente apropiado
#   ✓ Más natural y eficiente
# Desventajas:
#   ✗ Menos predecible
#   ✗ Puede dejar a alguien fuera


# Método 3: Manual (personalizado)
def manual_speaker_selection(last_speaker, groupchat):
    """Lógica custom para seleccionar siguiente speaker"""

    messages = groupchat.messages
    last_msg = messages[-1] if messages else None

    # Ejemplo: si fue error, siempre habla tester
    if "error" in last_msg.get("content", "").lower():
        return groupchat.agents[2]  # Tester

    # Si no, round robin
    for i, agent in enumerate(groupchat.agents):
        if agent == last_speaker:
            return groupchat.agents[(i + 1) % len(groupchat.agents)]
```

```
    return groupchat.agents[0]

groupchat = GroupChat(
    agents=[coder, reviewer, tester, user_proxy],
    speaker_selection_method=manual_speaker_selection
)

# Ventajas:
#   ✓ Control total
#   ✓ Lógica de negocio específica
# Desventajas:
#   ✗ Requiere escribir código
#   ✗ Complejidad adicional
```

2.2.2. Flujos de Comunicación Avanzados

Control de Turnos Detallado:

```
# Configuración completa de GroupChat
groupchat = GroupChat(
    agents=[coder, reviewer, tester, user_proxy],
    messages=[],

    # CONTROL DE DURACIÓN
    max_round=15,                           # Máximo turnos totales

    # SELECCIÓN DE SPEAKERS
    speaker_selection_method="auto",        # "auto", "round_robin", o función

    # MANEJO DE ERRORES
    send_introductions=True,                # Presentar agentes al inicio

    # FUNCIÓN DE TÉRMINO
    is_termination_msg=lambda x: "LISTO" in x.get("content", "")
)

# Detectar término temprano
def is_termination_msg(msg):
    content = msg.get("content", "").lower()

    # Terminar si:
    if any(word in content for word in ["TERMINADO", "COMPLETO", "LISTO", "EXITOSO"]):
        return True

    # O si usuario explícitamente rechaza continuación:
    if "no" in content and "continuar" in content:
        return True

    return False
```

```
groupchat.is_termination_msg = is_termination_msg
        return True
    return False

groupchat.is_termination_msg = is_termination_msg
```

Condiciones de Términación:

```
# Opción 1: Basada en contenido
def custom_termination(msg):
    # Terminar si alguien dice "LISTO"
    return "LISTO" in msg.get("content", "").upper()

# Opción 2: Basada en número de turnos
max_turns = 20
current_turn = 0

# Opción 3: Basada en satisfacción
class SatisfactionTerminator:
    def __init__(self, threshold=0.9):
        self.threshold = threshold

    def should_terminate(self, conversation):
        # Analizar si ambos agentes están satisfechos
        # Retornar True si satisfacción > threshold
        pass
```

2.3. 2.3. Configuración Avanzada

2.3.1. Parámetros de Configuración Completos

```
# Configuración completa y detallada
llm_config = {
    # ===== MODELOS =====
    "config_list": [
        {
            "model": "gpt-4",
            "api_key": "sk-...",
            "api_base": "https://api.openai.com/v1",
            "timeout": 120
        },
        {
            "model": "gpt-3.5-turbo",  # Fallback si GPT-4 falla
            "api_key": "sk-..."
        }
    ],

    # ===== PARÁMETROS DE GENERACIÓN =====
```

```
    "temperature": 0.7,                    # Creatividad (0-2)
    "top_p": 0.9,                         # Nucleus sampling
    "top_k": 40,                          # Opciones top-k
    "frequency_penalty": 0.0,             # Penalizar repetición
    "presence_penalty": 0.0,              # Penalizar tópicos nuevos

    # ===== LÍMITES =====
    "max_tokens": 2000,                   # Máximo output
    "max_completion_tokens": 2000,        # Alternativo

    # ===== CONTROL =====
    "timeout": 120,                       # Segundos para respuesta
    "cache_seed": 42,                     # Para reproducibilidad
    "request_timeout": 300,               # Timeout de request

    # ===== FEATURES =====
    "seed": 42,                           # Seed para determinismo
    "extra_body": {...}                   # Parámetros extra del API
}
```

2.3.2. Gestión de Contexto

Historial de Mensajes:

```
# Acceder al historial
conversation_history = assistant.chat_history

# Estructura de cada mensaje:
for msg in conversation_history:
    print(f"De: {msg['name']}")
    print(f"Rol: {msg['role']}")          # 'user' o 'assistant'
    print(f"Contenido: {msg['content']}")
    if 'function_call' in msg:
        print(f"Función: {msg['function_call']}")

# Modificar historial (cuidado)
# Útil para:
# - Guardar y cargar conversaciones
# - Analizar patrones
# - Debug
assistant.chat_history = new_history
```

Límites de Contexto:

```
# El contexto tiene límites de tokens
# GPT-4: 8K tokens (~6000 palabras) o 32K tokens
# GPT-3.5: 4K tokens
```

```
# Estrategias para largo contexto:
# 1. Resumir después de cierto número de mensajes
def summarize_context(messages, max_messages=20):
    if len(messages) > max_messages:
        # Guardar primeros X y últimos Y mensajes
        # Resumir los intermedios
        keep_first = max_messages // 2
        keep_last = max_messages // 2

        summary = create_summary(
            messages[keep_first:-keep_last]
        )

        return (messages[:keep_first] +
                [summary_msg] +
                messages[-keep_last:])
    return messages

# 2. Usar memoria externa
class ExternalMemory:
    def __init__(self):
        self.long_term = []  # Base de datos
        self.short_term = []  # Últimos N mensajes

    def add(self, message):
        self.short_term.append(message)
        if len(self.short_term) > 10:
            self.long_term.append(self.short_term.pop(0))

    def get_context(self):
        return self.short_term + self.retrieve_relevant(from_long_term=True)

# 3. Use RAG (Retrieval Augmented Generation)
# Buscar documentos relevantes en lugar de cargar todo
```

Persistencia de Conversaciones:

```
import json

# Guardar conversación
def save_conversation(agent, filename):
    with open(filename, 'w') as f:
        json.dump(agent.chat_history, f, indent=2)

# Cargar conversación
def load_conversation(agent, filename):
    with open(filename, 'r') as f:
        agent.chat_history = json.load(f)

# Usar:
```

```
save_conversation(assistant, "conversation.json")
load_conversation(assistant, "conversation.json")

# Estructura guardada:
# [
#   {
#     "role": "user",
#     "content": "¿Qué es Python?",
#     "name": "Usuario"
#   },
#   {...}
# ]
```

Chapter 3. Módulo 3: Patrones de Conversación

3.1. 3.1. Conversaciones Dos Agentes

3.1.1. Patrón Pregunta-Respuesta

Caso Simple:

```
user_proxy.initiate_chat(
    assistant,
    message="¿Cuál es la raíz cuadrada de 16?",
    max_consecutive_auto_reply=1  # Solo 1 respuesta
)

# Flujo:
# User: "¿Cuál es la raíz cuadrada de 16?"
# Assistant: "La raíz cuadrada de 16 es 4"
# [FIN]
```

Caso Iterativo:

```
# Primera pregunta
user_proxy.initiate_chat(
    assistant,
    message="¿Cuál es la raíz cuadrada de 16?",
    max_consecutive_auto_reply=5  # Permite más turnos
)

# El sistema permite:
# User -> Assistant -> User -> Assistant -> User
# hasta completar o alcanzar max_consecutive_auto_reply
```

Opciones de Configuración:

```
# control fino del diálogo
config = {
    # Parar después de X respuestas del assistant
    "max_consecutive_auto_reply": 5,

    # Parar si se alcanza número máximo de turnos
    "max_turns": 20,

    # Condición de término personalizada
    "is_termination_msg": lambda x: "LISTO" in x.get("content", ""),
```

```
    # Función para cambiar responsabilidad
    "next_turn": "auto"  # 'auto' o función personalizada
}
```

3.1.2. Resolución de Problemas Iterativa

Patrón de Refinamiento:

```
# Problema complejo que requiere iteración
user_proxy.initiate_chat(
    assistant,
    message="""Necesito optimizar esta función Python:

def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)

Mejorala en términos de velocidad.""",
    max_consecutive_auto_reply=10  # Permitir múltiples turnos
)

# Flujo esperado:
# 1. User propone problema
# 2. Assistant analiza y propone solución
# 3. User puede proporcionar feedback
# 4. Assistant refina basado en feedback
# 5. Itera hasta solución satisfactoria
```

Implementación del Feedback Loop:

```
class IterativeAgent:
    def __init__(self, assistant, user_proxy):
        self.assistant = assistant
        self.user = user_proxy
        self.iterations = 0
        self.max_iterations = 5

    def solve(self, problem):
        """Resolver problema iterativamente"""
        solution = None
        feedback = None

        for i in range(self.max_iterations):
            self.iterations += 1

            if feedback:
```

```
                message = f"{problem}\n\nFeedback previo: {feedback}"
            else:
                message = problem

            # Obtener solución
            response = self.assistant.generate_response(message)
            solution = parse_solution(response)

            # Evaluar solución
            is_good, feedback = self.evaluate(solution)

            if is_good:
                return solution, self.iterations

        return solution, self.iterations

    def evaluate(self, solution):
        """Evaluar si solución es aceptable"""
        # Implementar lógica de evaluación
        # Retornar (es_aceptable, feedback_para_mejorar)
        pass
```

3.1.3. Validación de Respuestas

Patrón de Validación:

```
class ValidatingUserProxy(UserProxyAgent):
    def __init__(self, validator_func, **kwargs):
        super().__init__(**kwargs)
        self.validator = validator_func

    def receive(self, message, sender, request_reply=None):
        """Recibir y validar mensaje"""
        # Validar antes de procesar
        if not self.validator(message):
            print("Respuesta no válida, solicitando aclaración")

            # Enviar mensaje de rechazo amable
            self.send(
                message="Tu respuesta no es válida. Por favor, asegúrate de...",
                recipient=sender
            )
        else:
            # Procesar normalmente
            super().receive(message, sender, request_reply)

# Ejemplo de validador
def validate_code(message):
    """Validar que el mensaje contiene código válido"""
    content = message.get("content", "")
```

```
    # Checks
    has_code = "```" in content or "def " in content
    has_explanation = len(content.split()) > 10

    return has_code and has_explanation

# Usar
validator_proxy = ValidatingUserProxy(
    validator_func=validate_code,
    human_input_mode="NEVER"
)
```

Chapter 4. Módulo 4: Capacidades Avanzadas

4.1. 4.1. Ejecución de Código

4.1.1. Code Execution Segura

La ejecución de código es una de las características más poderosas de AutoGen, pero también la más peligrosa. Es fundamental entender las opciones de seguridad disponibles.

Comparativa de Opciones de Ejecución:

```
OPCIÓN 1: Ejecución Local (use_docker=False)

Seguridad:      □□ BAJA
                - Código tiene acceso completo al sistema
                - Puede leer archivos sensibles
                - Puede hacer network requests
                - Puede modificar filesystem

Velocidad:      □ MUY RÁPIDA
                - 100ms por ejecución típicamente

Requisitos:     Todas las dependencias instaladas localmente

Uso recomendado: SOLO desarrollo local de confianza

CASO DE USO:
  ✓ Prototipos rápidos
  ✓ Testing durante desarrollo
  □ NUNCA en producción
  □ NUNCA con código untrusted

Ejemplo:
  user_proxy = UserProxyAgent(
      code_execution_config={
          "use_docker": False,
          "work_dir": "./workspace",
          "timeout": 30
      }
  )

RIESGOS REALES:
  - rm -rf / (borra todo el sistema)
  - import socket; socket.send_to(attacker)  (exfiltración de datos)
  - subprocess.call(['malicious_command'])

---
```

```
OPCIÓN 2: Ejecución en Docker (use_docker=True) [RECOMENDADO]

Seguridad:      □ ALTA
                - Código aislado del host
                - Acceso limitado a filesystem
                - Network sandbox
                - Recursos limitables
                - Usuario no-root por defecto

Velocidad:      □ RÁPIDA
                - ~500-2000ms por ejecución
                - Lentitud principalmente en startup de container

Requisitos:     Docker instalado y funcionando

Uso recomendado: Producción, testing, código untrusted

CASO DE USO:
  ✓ Producción (seguridad crítica)
  ✓ Ejecución de código potencialmente malicioso
  ✓ Sistemas multi-tenant
  □□ Puede ser lento para operaciones repetidas

Ejemplo recomendado para producción:
  user_proxy = UserProxyAgent(
      code_execution_config={
          "use_docker": True,
          "docker_image": "python:3.11-slim",
          "work_dir": "/workspace",
          "timeout": 60,
          "docker_memory": "2g",      # Limitar a 2GB
          "docker_cpus": "1.0"        # Limitar a 1 CPU
      }
  )

---

OPCIÓN 3: Sin Ejecución (code_execution_config=False)

Seguridad:      □□ MÁXIMA
                - No se ejecuta código en absoluto
                - Solo análisis estático

Velocidad:      □□ INSTANTÁNEO
                - Cero overhead

Requisitos:     Ninguno

Uso recomendado: Solo si realmente no necesitas ejecución
```

```
CASO DE USO:
  ✓ Solo análisis/revisión de código
  ✓ Máxima seguridad pero menos útil
  ⚠⚠ Pierdes la capacidad de validar que el código funciona

Ejemplo:
  user_proxy = UserProxyAgent(
      code_execution_config=False
  )

DESVENTAJAS:
  - El asistente no puede validar su código
  - Más probabilidad de código con bugs
  - No hay feedback de ejecución

---

COMPARATIVA RESUMIDA:

 ┌──────────────────────────────┬──────────────────────────┬──────────────────────────
─┬──────────────────────────┐
 │ Aspecto         │ Local        │ Docker     │ Sin exec.     │
 ├──────────────────────────────┼──────────────────────────┼──────────────────────────
─┼──────────────────────────┤
 │ Seguridad       │ ⚠⚠ Baja      │ ✓ Alta    │ ⚠⚠ Máxima  │
 │ Velocidad       │ ✓ Muy rápido│ ✓ Rápido   │ ⚠⚠ Instant  │
 │ Confiabilidad   │ ⚠⚠ Baja      │ ✓ Alta    │ ⚠⚠ Media     │
 │ Producción      │ ✗ NO         │ ✓ SÍ      │  ✓ Parcial    │
 │ Desarrollo      │ ✓ SÍ        │ ✓ SÍ      │  ✓ Parcial    │
 └──────────────────────────────┴──────────────────────────┴──────────────────────────
─┴──────────────────────────┘

RECOMENDACIÓN:
  - DESARROLLO LOCAL: use_docker=False (rápido)
  - TESTING AUTOMATIZADO: use_docker=True (seguro)
  - PRODUCCIÓN: use_docker=True (OBLIGATORIO)
  - MÁXIMA SEGURIDAD: code_execution_config=False
```

Opciones de Ejecución en Detalle:

```
# Opción 1: Local (Rápido, pero riesgoso)
user_proxy = UserProxyAgent(
    name="Usuario",
    code_execution_config={
        "work_dir": "./workspace",
        "use_docker": False,
        "timeout": 30
    }
)
```

```
# Opción 2: Docker (Seguro, recomendado)
user_proxy = UserProxyAgent(
    name="Usuario",
    code_execution_config={
        "work_dir": "./workspace",
        "use_docker": True,
        "docker_image": "python:3.11-slim",
        "timeout": 60
    }
)

# Opción 3: Docker con limites de recursos
user_proxy = UserProxyAgent(
    name="Usuario",
    code_execution_config={
        "work_dir": "./workspace",
        "use_docker": True,
        "docker_image": "python:3.11-slim",
        "timeout": 30,
        "docker_memory": "2g",      # Limitar memoria
        "docker_cpus": "1.0",       # Limitar CPUs
        "docker_user": "nobody"     # Usuario no-privilegiado
    }
)

# Opción 4: Remoto con Docker
user_proxy = UserProxyAgent(
    name="Usuario",
    code_execution_config={
        "work_dir": "/tmp",
        "use_docker": True,
        "docker_host": "ssh://user@remote.server"
    }
)

# Opción 5: Sin ejecución (Solo análisis)
user_proxy = UserProxyAgent(
    name="Usuario",
    code_execution_config=False
)
```

Configuración Avanzada de Sandbox:

```
                "error": "Timeout: código tardó demasiado",
                "suggestion": "Optimizar o dividir el código"
            }

        except SyntaxError as e:
            return {
                "success": False,
```

```
                "error": f"Error de sintaxis: {e}",
                "suggestion": "Verificar paréntesis y indentación"
            }
**Configuración Avanzada de Sandbox:**
```

```
# Configuración SEGURA recomendada para PRODUCCIÓN
production_config = {
    # Directorio de trabajo aislado
    "work_dir": "/sandbox/workspace",

    # AISLAMIENTO CON DOCKER (OBLIGATORIO)
    "use_docker": True,
    "docker_image": "python:3.11-slim",  # Imagen minimal

    # LÍMITES DE RECURSOS
    "docker_memory": "2g",        # Máximo 2GB RAM
    "docker_cpus": "1.0",         # Máximo 1 CPU
    "timeout": 30,                # Máximo 30 segundos por ejecución

    # SEGURIDAD ADICIONAL
    "docker_user": "nobody",      # No root
    "docker_network_disabled": True,  # No network access

    # VOLÚMENES MONTADOS (si necesario)
    "docker_volumes": {
        "/secure/input": "/input",   # Solo lectura datos de entrada
    },

    # LOGGING Y AUDITORÍA
    "log_directory": "/var/log/autogen",
    "verbose": True
}

user_proxy = UserProxyAgent(
    name="Usuario",
    code_execution_config=production_config
)

# CONFIGURACIÓN RÁPIDA para desarrollo local
dev_config = {
    "work_dir": "./workspace",
    "use_docker": False,  # Rápido para desarrollo
    "timeout": 30
}

# CONFIGURACIÓN SEGURA MÍNIMA
minimal_secure = {
    "work_dir": "./workspace",
    "use_docker": True,
    "timeout": 30
```

```
}
```

4.1.2. Manejo Avanzado de Errores en Code Execution

```
class RobustCodeExecutor(UserProxyAgent):
    """Ejecutor de código con manejo robusto de errores y recuperación"""

    def __init__(self, max_retries=3, **kwargs):
        super().__init__(**kwargs)
        self.max_retries = max_retries
        self.execution_history = []

    def execute_code(self, code):
        """Ejecutar código con recuperación automática de errores"""

        for attempt in range(self.max_retries):
            try:
                result = super().execute_code(code)
                self.execution_history.append({
                    "code": code,
                    "status": "success",
                    "result": result,
                    "attempt": attempt + 1
                })
                return {"success": True, "result": result, "attempts": attempt + 1}

            except TimeoutError as e:
                self.execution_history.append({
                    "code": code,
                    "status": "timeout",
                    "attempt": attempt + 1
                })
                if attempt == self.max_retries - 1:
                    return {
                        "success": False,
                        "error": "Timeout: el código tardó demasiado",
                        "suggestion": "Optimizar o dividir en partes más pequeñas",
                        "attempts": attempt + 1
                    }

            except SyntaxError as e:
                return {
                    "success": False,
                    "error": f"Error de sintaxis: {e}",
                    "line": e.lineno,
                    "suggestion": "Revisar paréntesis, indentación y comillas"
                }

            except Exception as e:
```

```
                error_type = type(e).__name__
                error_msg = str(e)

                self.execution_history.append({
                    "code": code,
                    "status": f"error_{error_type}",
                    "error": error_msg,
                    "attempt": attempt + 1
                })

                if attempt < self.max_retries - 1:
                    # Intentar arreglar automáticamente
                    print(f"Intento {attempt + 1} falló con {error_type},
reintentando...")
                else:
                    return {
                        "success": False,
                        "error": f"Error: {error_type}: {error_msg}",
                        "attempts": attempt + 1,
                        "suggestion": "Revisar la lógica del código"
                    }

    def auto_fix_and_retry(self, code, error_info):
        """Intentar arreglar automáticamente y reintentar"""
        fix_prompt = f"""El código falló con este error:

Error: {error_info['error']}

Código original:
```python
{code}
```

Por favor, arregla el código para que funcione correctamente.
Asegúrate de:
1. Corregir la causa del error
2. Mantener la funcionalidad original
3. Incluir validaciones de entrada"""

        # Usar LLM para generar fix (requiere acceso a LLM)
        # fixed_code = self.ask_assistant_for_fix(fix_prompt)
        # return self.execute_code(fixed_code)
        return {"status": "needs_manual_fix", "prompt": fix_prompt}

    def get_execution_summary(self):
        """Resumen de todas las ejecuciones"""
        successful = sum(1 for h in self.execution_history if h["status"] ==
"success")
        failed = len(self.execution_history) - successful

        return {
```

```
            "total_executions": len(self.execution_history),
            "successful": successful,
            "failed": failed,
            "success_rate": successful / len(self.execution_history) if
self.execution_history else 0,
            "history": self.execution_history
        }
```

4.1.3. Mejores Prácticas en Code Execution

```
MEJORES PRÁCTICAS:

1. SIEMPRE usar Docker en producción
   ✓ use_docker=True (OBLIGATORIO)
   ✓ docker_memory limite específico
   ✓ docker_user non-root

2. NUNCA confiar en código generado sin validación
   ✓ Revisar código antes de ejecutar
   ✓ Usar máximo 30-60 segundos timeout
   ✓ Ejecutar con privilegios mínimos

3. IMPLEMENTAR timeouts razonables
   ✓ 30-60 segundos para la mayoría de tareas
   ✓ Dividir tareas largas en partes
   ✓ Caché resultados para evitar repetir

4. MONITOREO Y LOGGING
   ✓ Registrar todas las ejecuciones
   ✓ Alertar en errores inesperados
   ✓ Mantener historial para auditoría

5. GESTIÓN DE DEPENDENCIAS
   ✓ Especificar versiones exactas en requirements
   ✓ Pre-instalar dependencias seguras
   ✓ Evitar instalar packages en tiempo de ejecución

RIESGOS RESIDUALES:

Incluso con Docker:
  ⚠️ Algoritmos muy complejos pueden consumir CPU
  ⚠️ Código en bucles infinitos consume timeout
  ⚠️ Side effects de librerías pueden causar problemas

Mitigación:
  ✓ Revisión de código generado por LLM
  ✓ Testing antes de ejecutar en producción
  ✓ Monitoreo de recursos
  ✓ Alertas automáticas
```

4.2. 4.2. Function Calling

4.2.1. Definición y Registro de Funciones

Concepto:

Function calling permite que los agentes LLM ejecuten funciones específicas automáticamente. El LLM decide si necesita llamar una función, qué función, y con qué parámetros.

```
# Funciones que los agentes pueden invocar
def calculate_compound_interest(
    principal: float,
    rate: float,
    years: int,
    compounds_per_year: int = 12
) -> float:
    """
    Calcular interés compuesto

    Args:
        principal: Cantidad inicial
        rate: Tasa de interés anual (ej: 0.05 para 5%)
        years: Número de años
        compounds_per_year: Frecuencia de capitalización

    Returns:
        Cantidad final incluyendo interés
    """
    return principal * (1 + rate/compounds_per_year) ** (compounds_per_year * years)

def fetch_weather(city: str) -> dict:
    """
    Obtener pronóstico del tiempo

    Args:
        city: Nombre de la ciudad

    Returns:
        dict con temperatura, condiciones, etc.
    """
    # Implementación real llamaría a API
    return {
        "city": city,
        "temp": 22,
        "condition": "Soleado"
    }

# Registrar con el agente
assistant = AssistantAgent(
    name="Asistente",
```

```
    function_map={
        "calculate_compound_interest": calculate_compound_interest,
        "fetch_weather": fetch_weather
    },
    llm_config=llm_config
)
```

4.2.2. Invocación Automática de Funciones

Cómo funciona:

```
FLUJO DE FUNCTION CALLING:

1. User: "¿Cuál será el valor de 1000€ al 5% en 10 años?"

2. LLM (Assistant) decide:
   "Necesito calculate_compound_interest"

3. LLM envía solicitud de function call:
   {
     "function": "calculate_compound_interest",
     "parameters": {
       "principal": 1000,
       "rate": 0.05,
       "years": 10
     }
   }

4. UserProxy ejecuta la función:
   resultado = calculate_compound_interest(1000, 0.05, 10)
   resultado = 1647.01

5. UserProxy devuelve resultado al LLM:
   "Resultado: 1647.01"

6. LLM interpreta resultado y responde:
   "El valor será 1647.01€"

VENTAJAS:

✓ El LLM decide automáticamente si necesita una función
✓ Extrae parámetros correctamente
✓ Maneja errores y reintentos
✓ Capaz de usar múltiples funciones en secuencia
✓ Loop automático hasta solución correcta
```

Ejemplo Completo:

El agente decide automáticamente si usar funciones:

```
user_proxy.initiate_chat(
    assistant,
    message="""¿Cuál sería el valor final de una inversión de 1000
              euros a 5% anual durante 10 años, capitalizado mensualmente?"""
)

Flujo:
1. User envía pregunta
2. Assistant reconoce que necesita calcular interés compuesto
3. Assistant llama a calculate_compound_interest(1000, 0.05, 10, 12)
4. Obtiene resultado: 1645.68
5. Proporciona respuesta: "El valor final sería 1645.68 euros"
```

4.2.3. Parámetros y Esquemas

Los esquemas se generan automáticamente de los type hints:

```
def search_products(
    query: str,              # Requerido: string
    price_max: float = None, # Opcional: float
    brand: str = None,       # Opcional: string
    in_stock: bool = True    # Opcional: bool
) -> list:
    """
    Buscar productos en el catálogo

    Args:
        query: Término de búsqueda
        price_max: Precio máximo (opcional)
        brand: Marca específica (opcional)
        in_stock: Solo en stock (default True)

    Returns:
        Lista de productos encontrados
    """
    pass

El esquema generado automáticamente:
{
  "name": "search_products",
  "description": "Buscar productos...",
  "parameters": {
    "type": "object",
    "properties": {
      "query": {"type": "string"},
      "price_max": {"type": "number"},
      "brand": {"type": "string"},
      "in_stock": {"type": "boolean"}
    },
```

```
    "required": ["query"]
  }
}
----    Args:
        query: Término de búsqueda
        price_max: Precio máximo (opcional)
        brand: Marca específica (opcional)
        in_stock: Solo en stock (default True)

    Returns:
        Lista de productos encontrados
    """
    pass

# El esquema generado automáticamente:
# {
#   "name": "search_products",
#   "description": "Buscar productos...",
#   "parameters": {
#     "type": "object",
#     "properties": {
#       "query": {"type": "string"},
#       "price_max": {"type": "number"},
#       "brand": {"type": "string"},
#       "in_stock": {"type": "boolean"}
#     },
#     "required": ["query"]
#   }
# }
```

Chapter 5. Módulo 5: Agentes Especializados

5.1. 5.1. Agentes Personalizados Avanzados

5.1.1. Creación de Clases de Agentes

```
from autogen import AssistantAgent

class CodeReviewAgent(AssistantAgent):
    """Agente especializado en revisión de código"""

    def __init__(self, name="CodeReviewer", **kwargs):
        system_message = """Eres un experto revisor de código.
        Tu trabajo es:
        1. Identificar bugs y problemas potenciales
        2. Evaluar la calidad del código
        3. Sugerir mejoras y refactoring
        4. Verificar seguridad
        5. Comprobar eficiencia

        Proporciona feedback constructivo y específico."""

        super().__init__(
            name=name,
            system_message=system_message,
            **kwargs
        )

        # Adicionar funcionalidades personalizadas
        self.review_criteria = [
            "readability",
            "efficiency",
            "security",
            "maintainability",
            "testing"
        ]

    def score_code(self, code: str) -> dict:
        """Evaluar código en múltiples dimensiones"""
        scores = {}
        for criterion in self.review_criteria:
            scores[criterion] = self.evaluate_criterion(code, criterion)
        return scores

    def evaluate_criterion(self, code: str, criterion: str) -> float:
        """Evaluar código en un criterio específico"""
        # Implementación de evaluación
        pass
```

```
# Uso
reviewer = CodeReviewAgent(
    llm_config={"config_list": [{"model": "gpt-4", "api_key": "..."}]}
)

user_proxy.initiate_chat(
    reviewer,
    message="¿Puedes revisar este código?\n" + code_to_review
)
```

5.1.2. Patrones de Diseño

Patrón 1: Herencia Simple

```
class SpecializedAssistant(AssistantAgent):
    """Base para asistentes especializados"""

    def __init__(self, specialty: str, **kwargs):
        self.specialty = specialty
        system_msg = f"Eres un experto en {specialty}"
        super().__init__(system_message=system_msg, **kwargs)

class PythonExpert(SpecializedAssistant):
    def __init__(self, **kwargs):
        super().__init__(specialty="Python", **kwargs)

class JavaScriptExpert(SpecializedAssistant):
    def __init__(self, **kwargs):
        super().__init__(specialty="JavaScript", **kwargs)
```

Patrón 2: Composición

```
class CompositePipeline:
    """Sistema que compone múltiples agentes"""

    def __init__(self):
        self.implementer = AssistantAgent(name="Implementador")
        self.reviewer = AssistantAgent(name="Revisor")
        self.tester = AssistantAgent(name="Tester")

    def process(self, request):
        # Flujo: implementador -> revisor -> tester
        impl_result = self.implementer.process(request)
        review_result = self.reviewer.process(impl_result)
        test_result = self.tester.process(review_result)
        return test_result
```

Patrón 3: Decoradores

```
def with_logging(AgentClass):
    """Decorador para añadir logging a agentes"""

    class LoggedAgent(AgentClass):
        def __init__(self, *args, **kwargs):
            super().__init__(*args, **kwargs)
            self.logs = []

        def generate_response(self, message):
            self.logs.append(f"INPUT: {message}")
            response = super().generate_response(message)
            self.logs.append(f"OUTPUT: {response}")
            return response

    return LoggedAgent

@with_logging
class LoggedAssistant(AssistantAgent):
    pass
```

Este es un documento significativamente expandido. Déjame continuar con más módulos...

5.2. 5.2. Agentes Multi-Modalidad

Concepto: Agentes que pueden procesar múltiples tipos de entrada (texto, imagen, audio) y generar salidas en diferentes formatos.

Opción 1: Agente Multi-Modal Básico

```
class MultiModalAgent(AssistantAgent):
    def __init__(self, name, llm_config):
        super().__init__(name, llm_config)
        self.supported_formats = ["text", "image", "audio"]

    def process_image(self, image_path):
        """Procesar imagen y extraer contenido"""
        # Convertir imagen a descripción textual
        description = self.describe_image(image_path)
        return self.generate_response(f"Imagen: {description}")

    def process_audio(self, audio_path):
        """Procesar audio y transcribir"""
        # Transcribir audio a texto
        transcript = self.transcribe_audio(audio_path)
        return self.generate_response(f"Audio: {transcript}")

    def generate_multimodal_output(self, prompt):
        """Generar salida en múltiples formatos"""
```

```
        text_response = self.generate_response(prompt)
        image_url = self.generate_image(prompt)
        return {
            "text": text_response,
            "image": image_url,
            "format": "multimodal"
        }
```

Opción 2: Agente Multi-Modal con Vision API

```
import anthropic

class VisionAgent(AssistantAgent):
    def __init__(self, name, api_key):
        super().__init__(name, api_key)
        self.client = anthropic.Anthropic(api_key=api_key)

    def analyze_image_with_vision(self, image_path, task):
        """Usar Vision API para análisis de imágenes"""
        with open(image_path, "rb") as img_file:
            image_data = img_file.read()

        message = self.client.messages.create(
            model="claude-3-vision-20240229",
            max_tokens=1024,
            messages=[
                {
                    "role": "user",
                    "content": [
                        {
                            "type": "image",
                            "source": {
                                "type": "base64",
                                "media_type": "image/jpeg",
                                "data": image_data
                            }
                        },
                        {
                            "type": "text",
                            "text": task
                        }
                    ]
                }
            ]
        )
        return message.content[0].text

    def extract_text_from_document(self, document_path):
        """Extraer texto de documentos con OCR"""
        response = self.analyze_image_with_vision(
```

```
            document_path,
            "Extrae todo el texto visible en esta imagen"
        )
        return response
```

Opción 3: Agente Multi-Modal Avanzado con Streaming

```
class AdvancedMultiModalAgent(AssistantAgent):
    def __init__(self, name, llm_config):
        super().__init__(name, llm_config)
        self.input_queue = []
        self.output_formats = ["text", "json", "markdown", "html"]

    def process_mixed_content(self, inputs):
        """Procesar múltiples tipos de contenido en secuencia"""
        results = {}

        for input_item in inputs:
            if input_item["type"] == "text":
                results[input_item["id"]] = self.process_text(input_item["content"])
            elif input_item["type"] == "image":
                results[input_item["id"]] = self.process_image(input_item["content"])
            elif input_item["type"] == "audio":
                results[input_item["id"]] = self.process_audio(input_item["content"])

        return results

    def generate_formatted_output(self, content, output_format):
        """Generar salida en formato especificado"""
        if output_format == "json":
            return json.dumps(content, indent=2)
        elif output_format == "markdown":
            return self.to_markdown(content)
        elif output_format == "html":
            return self.to_html(content)
        else:
            return str(content)
```

Características Clave:

- □ Procesamiento de múltiples tipos de entrada
- □ Generación de salidas diversas
- □ Integración con APIs de visión
- □ Manejo de formatos variados

Casos de Uso:

- Análisis de documentos escaneados
- Procesamiento de imágenes médicas

- Análisis de gráficos y diagramas
- Transcripción de reuniones

5.3. 5.3. Agentes de Memoria Extendida

Concepto: Agentes que mantienen contexto histórico sofisticado y pueden recordar información a largo plazo.

Opción 1: Memoria Simple con Persistencia

```
class MemoryAgent(AssistantAgent):
    def __init__(self, name, llm_config, memory_file="memory.json"):
        super().__init__(name, llm_config)
        self.memory_file = memory_file
        self.long_term_memory = self.load_memory()
        self.short_term_memory = []

    def load_memory(self):
        """Cargar memoria persistida"""
        try:
            with open(self.memory_file, 'r') as f:
                return json.load(f)
        except FileNotFoundError:
            return {"facts": [], "patterns": [], "relationships": []}

    def save_memory(self):
        """Guardar memoria a archivo"""
        with open(self.memory_file, 'w') as f:
            json.dump(self.long_term_memory, f, indent=2)

    def add_memory(self, category, content):
        """Agregar contenido a memoria"""
        if category not in self.long_term_memory:
            self.long_term_memory[category] = []

        memory_entry = {
            "content": content,
            "timestamp": datetime.now().isoformat(),
            "importance": self.calculate_importance(content)
        }
        self.long_term_memory[category].append(memory_entry)
        self.save_memory()

    def retrieve_memory(self, query, category=None):
        """Buscar en memoria"""
        if category:
            items = self.long_term_memory.get(category, [])
        else:
```

```
            items = sum(self.long_term_memory.values(), [])

        # Búsqueda simple (podría usar embeddings)
        results = [item for item in items if query.lower() in item["content"].lower()]
        return sorted(results, key=lambda x: x["importance"], reverse=True)
```

Opción 2: Memoria con Embeddings y Búsqueda Semántica

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

class SemanticMemoryAgent(AssistantAgent):
    def __init__(self, name, llm_config):
        super().__init__(name, llm_config)
        self.memory_database = []
        self.embedding_model = self.load_embedding_model()

    def load_embedding_model(self):
        """Cargar modelo de embeddings"""
        try:
            from sentence_transformers import SentenceTransformer
            return SentenceTransformer('all-MiniLM-L6-v2')
        except ImportError:
            print("Instala: pip install sentence-transformers")
            return None

    def store_memory_with_embedding(self, content):
        """Guardar contenido con embedding"""
        embedding = self.embedding_model.encode(content)
        memory_item = {
            "content": content,
            "embedding": embedding,
            "timestamp": datetime.now().isoformat(),
            "access_count": 0
        }
        self.memory_database.append(memory_item)
        return memory_item

    def semantic_search(self, query, top_k=5):
        """Búsqueda semántica en memoria"""
        query_embedding = self.embedding_model.encode(query)

        similarities = []
        for item in self.memory_database:
            similarity = cosine_similarity(
                [query_embedding],
                [item["embedding"]]
            )[0][0]
            similarities.append((item, similarity))
```

```
        # Ordenar por similitud
        top_results = sorted(similarities, key=lambda x: x[1], reverse=True)[:top_k]

        # Actualizar contador de acceso
        for item, _ in top_results:
            item["access_count"] += 1

        return [item for item, _ in top_results]

    def get_memory_summary(self):
        """Generar resumen de memoria"""
        total_items = len(self.memory_database)
        most_accessed = sorted(
            self.memory_database,
            key=lambda x: x["access_count"],
            reverse=True
        )[:5]

        return {
            "total_memories": total_items,
            "most_accessed": [item["content"] for item in most_accessed]
        }
```

Opción 3: Sistema de Memoria Jerárquico

```
class HierarchicalMemoryAgent(AssistantAgent):
    def __init__(self, name, llm_config):
        super().__init__(name, llm_config)
        self.working_memory = []      # Información actual (corto plazo)
        self.episodic_memory = []     # Eventos/conversaciones
        self.semantic_memory = {}     # Hechos/conocimiento
        self.procedural_memory = []   # Skills/procedimientos

    def categorize_and_store(self, information, info_type):
        """Almacenar información en categoría apropiada"""
        if info_type == "current":
            self.working_memory.append({
                "content": information,
                "timestamp": datetime.now().isoformat()
            })
        elif info_type == "episode":
            self.episodic_memory.append({
                "event": information,
                "timestamp": datetime.now().isoformat()
            })
        elif info_type == "semantic":
            key = information.get("key")
            self.semantic_memory[key] = information.get("value")
        elif info_type == "procedural":
            self.procedural_memory.append({
```

```
                "procedure": information,
                "learned_at": datetime.now().isoformat()
            })

    def consolidate_memory(self):
        """Consolidar memoria de trabajo a memoria de largo plazo"""
        # Mover items importantes de working memory a episodic
        important_items = [item for item in self.working_memory
                           if self.is_important(item)]

        for item in important_items:
            self.episodic_memory.append(item)

        self.working_memory = [item for item in self.working_memory
                               if not self.is_important(item)]

    def recall_context(self, query):
        """Recuperar contexto relevante para consulta"""
        relevant_memories = {
            "semantic": [v for k, v in self.semantic_memory.items()
                         if query.lower() in str(v).lower()],
            "episodic": [e for e in self.episodic_memory
                         if query.lower() in str(e).lower()],
            "procedures": [p for p in self.procedural_memory
                           if query.lower() in str(p).lower()]
        }
        return relevant_memories
```

Características Clave:

- □ Persistencia de información
- □ Búsqueda semántica
- □ Organización jerárquica
- □ Consolidación de memoria

Aplicaciones:

- Sistemas de soporte que mejoran con el tiempo
- Asistentes personales
- Análisis histórico de proyectos

5.4. 5.4. Agentes de Aprendizaje Autónomo

Concepto: Agentes que mejoran su rendimiento automáticamente a través de interacción y retroalimentación.

Opción 1: Agente con Aprendizaje por Refuerzo

```
class ReinforcementLearningAgent(AssistantAgent):
    def __init__(self, name, llm_config):
        super().__init__(name, llm_config)
        self.success_count = 0
        self.failure_count = 0
        self.strategy_effectiveness = {}
        self.learning_rate = 0.1

    def learn_from_feedback(self, action, feedback, reward):
        """Aprender de retroalimentación"""
        strategy = action.get("strategy")

        if strategy not in self.strategy_effectiveness:
            self.strategy_effectiveness[strategy] = {"score": 0, "count": 0}

        # Actualizar puntuación con rata de aprendizaje
        old_score = self.strategy_effectiveness[strategy]["score"]
        new_score = old_score + self.learning_rate * (reward - old_score)

        self.strategy_effectiveness[strategy]["score"] = new_score
        self.strategy_effectiveness[strategy]["count"] += 1

        if reward > 0:
            self.success_count += 1
        else:
            self.failure_count += 1

    def select_best_strategy(self, situation):
        """Seleccionar mejor estrategia basada en aprendizaje"""
        applicable_strategies = self.get_applicable_strategies(situation)

        if not applicable_strategies:
            return self.random_strategy()

        # Epsilon-greedy: exploración vs explotación
        epsilon = 0.1
        if np.random.random() < epsilon:
            return np.random.choice(applicable_strategies)
        else:
            return max(applicable_strategies,
                       key=lambda s: self.strategy_effectiveness.get(s,
{}).get("score", 0))

    def get_performance_metrics(self):
        """Obtener métricas de aprendizaje"""
        total = self.success_count + self.failure_count
        if total == 0:
            return {"success_rate": 0, "strategies_learned": 0}

        return {
```

```
            "success_rate": self.success_count / total,
            "total_interactions": total,
            "strategies_learned": len(self.strategy_effectiveness),
            "best_strategy": max(self.strategy_effectiveness.items(),
                                 key=lambda x: x[1]["score"])[0]
        }
```

Opción 2: Agente con Fine-Tuning Continuo

```
class ContinuousLearningAgent(AssistantAgent):
    def __init__(self, name, llm_config, fine_tune_threshold=10):
        super().__init__(name, llm_config)
        self.training_examples = []
        self.fine_tune_threshold = fine_tune_threshold
        self.fine_tune_count = 0

    def collect_training_example(self, input_text, correct_output, user_feedback):
        """Recolectar ejemplo para fine-tuning"""
        if user_feedback == "correct":
            example = {
                "input": input_text,
                "output": correct_output,
                "quality_score": 1.0,
                "timestamp": datetime.now().isoformat()
            }
            self.training_examples.append(example)

    def prepare_fine_tune_dataset(self):
        """Preparar dataset para fine-tuning"""
        if len(self.training_examples) < self.fine_tune_threshold:
            return None

        # Filtrar ejemplos de alta calidad
        high_quality = [ex for ex in self.training_examples
                        if ex.get("quality_score", 0) >= 0.8]

        return {
            "training_examples": high_quality,
            "total_examples": len(high_quality),
            "data_quality": np.mean([ex["quality_score"] for ex in high_quality])
        }

    def trigger_fine_tune(self):
        """Disparar fine-tuning si hay suficientes datos"""
        dataset = self.prepare_fine_tune_dataset()
        if dataset:
            print(f"Fine-tuning con {dataset['total_examples']} ejemplos...")
            self.fine_tune_count += 1
            # Aquí llamaría a API de fine-tuning real
            return True
```

```
        return False
```

Opción 3: Agente con Metacognición

```
class MetacognitiveAgent(AssistantAgent):
    def __init__(self, name, llm_config):
        super().__init__(name, llm_config)
        self.confidence_scores = {}
        self.error_analysis = []
        self.improvement_strategies = {}

    def reflect_on_performance(self, task, result, actual_outcome):
        """Reflexionar sobre el desempeño"""
        analysis = {
            "task": task,
            "predicted": result,
            "actual": actual_outcome,
            "correct": result == actual_outcome,
            "timestamp": datetime.now().isoformat()
        }

        if not analysis["correct"]:
            self.analyze_error(analysis)
            self.identify_improvement_strategy(analysis)

        self.error_analysis.append(analysis)
        return analysis

    def analyze_error(self, error_analysis):
        """Analizar tipos de error"""
        error_type = self.classify_error(error_analysis)

        if error_type not in self.improvement_strategies:
            self.improvement_strategies[error_type] = {
                "occurrences": 0,
                "strategies": []
            }

        self.improvement_strategies[error_type]["occurrences"] += 1

    def identify_improvement_strategy(self, error_analysis):
        """Identificar estrategia de mejora"""
        error_type = self.classify_error(error_analysis)

        strategies = {
            "insufficient_context": "Pedir más contexto antes de responder",
            "misunderstanding": "Reformular la pregunta",
            "knowledge_gap": "Admitir limitación y buscar fuente externa",
            "reasoning_error": "Explicitar el razonamiento paso a paso"
        }
```

```
        return strategies.get(error_type, "Revisar base de conocimiento")

    def get_learning_report(self):
        """Reporte de aprendizaje"""
        total_attempts = len(self.error_analysis)
        correct = sum(1 for e in self.error_analysis if e["correct"])

        return {
            "accuracy": correct / total_attempts if total_attempts > 0 else 0,
            "total_attempts": total_attempts,
            "error_types": self.improvement_strategies,
            "recommendations": self.generate_recommendations()
        }

    def generate_recommendations(self):
        """Generar recomendaciones de mejora"""
        recommendations = []

        for error_type, data in self.improvement_strategies.items():
            if data["occurrences"] > 3:
                recommendations.append(
                    f"Error frecuente ({error_type}): {data['occurrences']} veces. "
                    "Requiere atención especial."
                )

        return recommendations
```

Características Clave:

- □ Aprendizaje por refuerzo
- □ Fine-tuning continuo
- □ Autorreflexión
- □ Mejora iterativa

Beneficios:

- Mejor desempeño con el tiempo
- Adaptación a casos nuevos
- Identificación automática de debilidades

5.5. 5.5. Agentes de Coordinación Multi-Agente

Concepto: Agentes especializados que coordinan su trabajo para resolver problemas complejos de forma colaborativa.

Opción 1: Gestor de Coordinación Centralizado

```
class CentralizedCoordinatorAgent(AssistantAgent):
    def __init__(self, name, llm_config):
        super().__init__(name, llm_config)
        self.agents = {}
        self.task_queue = []
        self.result_cache = {}

    def register_agent(self, agent_id, agent, specialization):
        """Registrar agente especializado"""
        self.agents[agent_id] = {
            "agent": agent,
            "specialization": specialization,
            "status": "idle",
            "completed_tasks": 0
        }

    def decompose_task(self, task):
        """Descomponer tarea en subtareas"""
        subtasks = []

        # Analizar qué especialidades se necesitan
        required_skills = self.analyze_required_skills(task)

        for skill in required_skills:
            subtask = {
                "task": task,
                "required_skill": skill,
                "status": "pending",
                "assigned_to": None,
                "result": None
            }
            subtasks.append(subtask)

        return subtasks

    def assign_task(self, subtask):
        """Asignar subtarea al agente más apropiado"""
        required_skill = subtask["required_skill"]

        # Buscar agente con especialización
        best_agent = None
        for agent_id, agent_info in self.agents.items():
            if agent_info["specialization"] == required_skill:
                if agent_info["status"] == "idle":
                    best_agent = agent_id
                    break

        if best_agent:
            subtask["assigned_to"] = best_agent
            self.agents[best_agent]["status"] = "busy"
```

```
            return True
        return False

    def execute_subtask(self, subtask):
        """Ejecutar subtarea asignada"""
        agent_id = subtask["assigned_to"]
        agent_info = self.agents[agent_id]
        agent = agent_info["agent"]

        result = agent.generate_response(subtask["task"])
        subtask["result"] = result
        subtask["status"] = "completed"

        agent_info["status"] = "idle"
        agent_info["completed_tasks"] += 1

        return result

    def synthesize_results(self, subtasks):
        """Sintetizar resultados de subtareas"""
        results_summary = {
            "subtask_results": [st["result"] for st in subtasks],
            "synthesis": self.generate_synthesis([st["result"] for st in subtasks])
        }
        return results_summary
```

Opción 2: Coordinación Descentralizada con Protocolo

```
class DecentralizedCoordinatorAgent(AssistantAgent):
    def __init__(self, name, llm_config):
        super().__init__(name, llm_config)
        self.peers = []
        self.message_queue = []
        self.consensus_threshold = 0.7

    def add_peer(self, peer_agent):
        """Agregar agente peer"""
        self.peers.append(peer_agent)

    def broadcast_message(self, message, message_type="request"):
        """Enviar mensaje a todos los peers"""
        msg = {
            "sender": self.name,
            "content": message,
            "type": message_type,
            "timestamp": datetime.now().isoformat(),
            "responses": []
        }

        for peer in self.peers:
```

```
            response = peer.handle_message(msg)
            msg["responses"].append({
                "from": peer.name,
                "response": response
            })

        return msg

    def reach_consensus(self, topic):
        """Alcanzar consenso entre agentes"""
        votes = {}

        for peer in self.peers:
            opinion = peer.get_opinion_on(topic)
            votes[peer.name] = opinion

        # Contar votos
        vote_counts = {}
        for opinion in votes.values():
            vote_counts[opinion] = vote_counts.get(opinion, 0) + 1

        # Verificar umbral de consenso
        total_votes = len(votes)
        max_votes = max(vote_counts.values())

        if max_votes / total_votes >= self.consensus_threshold:
            consensus_decision = max(vote_counts, key=vote_counts.get)
            return {
                "consensus_reached": True,
                "decision": consensus_decision,
                "confidence": max_votes / total_votes
            }

        return {
            "consensus_reached": False,
            "votes": vote_counts,
            "confidence": max_votes / total_votes
        }

    def conflict_resolution(self, conflicting_opinions):
        """Resolver conflictos entre opiniones"""
        # Debate estructurado
        debate_rounds = []

        for round_num in range(3):
            round_arguments = []
            for opinion, agents in conflicting_opinions.items():
                for agent in agents:
                    argument = agent.present_argument(opinion)
                    round_arguments.append(argument)
```

```
            debate_rounds.append(round_arguments)

        # Seleccionar mejor argumento
        best_argument = self.evaluate_arguments(debate_rounds)
        return best_argument
```

Opción 3: Coordinación Jerárquica Flexible

```
class FlexibleHierarchyCoordinator(AssistantAgent):
    def __init__(self, name, llm_config):
        super().__init__(name, llm_config)
        self.hierarchy = {}  # {level: [agents]}
        self.current_leader = None

    def build_hierarchy(self, agents, criteria="capability"):
        """Construir jerarquía dinámica"""
        if criteria == "capability":
            sorted_agents = sorted(agents, key=lambda a: a.capability_score,
reverse=True)
        elif criteria == "experience":
            sorted_agents = sorted(agents, key=lambda a: a.experience, reverse=True)
        else:
            sorted_agents = agents

        # Distribuir en niveles
        num_levels = 3
        self.hierarchy = {}
        agents_per_level = len(sorted_agents) // num_levels

        for level in range(num_levels):
            start_idx = level * agents_per_level
            end_idx = start_idx + agents_per_level if level < num_levels - 1 else
len(sorted_agents)
            self.hierarchy[level] = sorted_agents[start_idx:end_idx]

        self.current_leader = self.hierarchy[0][0]  # Top agent es líder

    def delegate_to_appropriate_level(self, task):
        """Delegar tarea al nivel apropiado"""
        complexity = self.assess_complexity(task)

        if complexity == "simple":
            agent = self.hierarchy[2][0]  # Nivel bajo
        elif complexity == "moderate":
            agent = self.hierarchy[1][0]  # Nivel medio
        else:
            agent = self.current_leader  # Nivel alto

        return agent.execute_task(task)
```

```
    def rebalance_hierarchy(self, performance_data):
        """Rebalancear jerarquía basado en desempeño"""
        # Actualizar puntuaciones basadas en desempeño
        for agent, performance in performance_data.items():
            agent.capability_score = agent.capability_score * 0.8 + performance * 0.2

        # Reconstruir si hay cambios significativos
        current_leader_performance = performance_data.get(self.current_leader, 0)
        if current_leader_performance < 0.5:
            self.build_hierarchy(list(self.hierarchy[0]))  # Reconstruir
```

Características Clave:

- □ Coordinación centralizada y descentralizada
- □ Resolución de conflictos
- □ Consenso distribuido
- □ Jerarquías dinámicas

Ventajas:

- Mayor capacidad de resolución de problemas
- Escalabilidad
- Robustez ante fallos individuales
- Especialización eficiente

Chapter 6. Módulo 6: Optimización y Costos

6.1. 6.1. Gestión de Tokens

Concepto: Monitoreo y optimización del consumo de tokens para reducir costos y mejorar rendimiento.

Opción 1: Gestor de Tokens Básico

```
class TokenManager:
    """Gestor básico de tokens y costos"""

    def __init__(self, model="gpt-4", pricing_per_1k=0.03):
        self.model = model
        self.pricing_per_1k = pricing_per_1k
        self.tokens_used = 0
        self.cost_accumulated = 0.0

    def estimate_tokens(self, text):
        """Estimación aproximada de tokens"""
        # Aproximadamente 4 caracteres = 1 token
        return len(text) // 4

    def calculate_cost(self, tokens):
        """Calcular costo basado en tokens"""
        cost = (tokens / 1000) * self.pricing_per_1k
        self.cost_accumulated += cost
        self.tokens_used += tokens
        return cost

    def get_cost_summary(self):
        """Resumen de costos"""
        return {
            "total_tokens": self.tokens_used,
            "total_cost": self.cost_accumulated,
            "avg_token_cost": self.cost_accumulated / max(self.tokens_used, 1)
        }
```

Opción 2: Optimizador de Prompts Avanzado

```
import tiktoken

class PromptOptimizer:
    """Optimizador de prompts para reducir tokens"""

    def __init__(self, model="gpt-4"):
        self.encoding = tiktoken.encoding_for_model(model)
```

```
    def count_tokens(self, text):
        """Contar tokens exactos usando tiktoken"""
        tokens = self.encoding.encode(text)
        return len(tokens)

    def compress_prompt(self, prompt, max_tokens=2000):
        """Comprimir prompt manteniendo semántica"""
        # Eliminar espacios excesivos
        compressed = " ".join(prompt.split())

        # Si es demasiado largo, resumir
        if self.count_tokens(compressed) > max_tokens:
            # Tomar primeras líneas más importantes
            lines = compressed.split(".")
            result = []
            for line in lines:
                result.append(line.strip())
                if self.count_tokens(". ".join(result)) > max_tokens:
                    result.pop()
                    break
            compressed = ". ".join(result)

        return compressed

    def suggest_improvements(self, prompt):
        """Sugerencias para mejorar tokens"""
        suggestions = []
        token_count = self.count_tokens(prompt)

        if len(prompt) > 1000:
            suggestions.append("Considere acortar la descripción")
        if prompt.count("\n\n") > 5:
            suggestions.append("Demasiados saltos de línea")
        if len(prompt.split()) > 300:
            suggestions.append("Demasiadas palabras, intente ser más conciso")

        return {"tokens": token_count, "suggestions": suggestions}
```

Opción 3: Sistema de Monitoreo en Tiempo Real

```
from datetime import datetime
from collections import defaultdict

class RealTimeTokenMonitor:
    """Monitoreo en tiempo real de tokens y costos"""

    def __init__(self, budget_limit=100.0):
        self.budget_limit = budget_limit
        self.transactions = []
        self.usage_by_agent = defaultdict(lambda: {"tokens": 0, "cost": 0})
```

```
        self.hourly_stats = defaultdict(lambda: {"tokens": 0, "cost": 0})

    def log_transaction(self, agent_name, tokens, cost, timestamp=None):
        """Registrar transacción de tokens"""
        if timestamp is None:
            timestamp = datetime.now()

        hour_key = timestamp.strftime("%Y-%m-%d %H:00")

        self.transactions.append({
            "agent": agent_name,
            "tokens": tokens,
            "cost": cost,
            "timestamp": timestamp
        })

        self.usage_by_agent[agent_name]["tokens"] += tokens
        self.usage_by_agent[agent_name]["cost"] += cost

        self.hourly_stats[hour_key]["tokens"] += tokens
        self.hourly_stats[hour_key]["cost"] += cost

    def get_budget_status(self):
        """Estado del presupuesto"""
        total_cost = sum(u["cost"] for u in self.usage_by_agent.values())
        remaining = self.budget_limit - total_cost
        percentage = (total_cost / self.budget_limit) * 100

        return {
            "budget_limit": self.budget_limit,
            "spent": total_cost,
            "remaining": remaining,
            "percentage_used": percentage,
            "within_budget": remaining >= 0
        }

    def get_agent_ranking(self):
        """Ranking de agentes por consumo"""
        return sorted(
            self.usage_by_agent.items(),
            key=lambda x: x[1]["cost"],
            reverse=True
        )
```

Características Clave:

- □ Estimación exacta de tokens con tiktoken
- □ Cálculo preciso de costos
- □ Compresión inteligente de prompts
- □ Monitoreo en tiempo real

- □ Alertas de presupuesto

Casos de Uso:

- Control de costos en producción
- Optimización de prompts
- Facturación por agente
- Detección de anomalías en consumo

6.2. 6.2. Caché y Reutilización

Concepto: Sistema de caché para reutilizar respuestas y reducir llamadas redundantes al LLM.

Opción 1: Caché en Memoria Básico

```
import hashlib
from functools import lru_cache

class SimpleMemoryCache:
    """Caché en memoria con TTL básico"""

    def __init__(self, max_size=1000):
        self.cache = {}
        self.max_size = max_size
        self.hits = 0
        self.misses = 0

    def _hash_key(self, prompt):
        """Generar clave hash del prompt"""
        return hashlib.md5(prompt.encode()).hexdigest()

    def get(self, prompt):
        """Obtener del caché"""
        key = self._hash_key(prompt)
        if key in self.cache:
            self.hits += 1
            return self.cache[key]
        self.misses += 1
        return None

    def set(self, prompt, response):
        """Guardar en caché"""
        if len(self.cache) >= self.max_size:
            # Eliminar entrada más antigua
            oldest_key = next(iter(self.cache))
            del self.cache[oldest_key]

        key = self._hash_key(prompt)
```

```
        self.cache[key] = response

    def get_stats(self):
        """Estadísticas de caché"""
        total = self.hits + self.misses
        hit_rate = (self.hits / total * 100) if total > 0 else 0
        return {
            "hits": self.hits,
            "misses": self.misses,
            "hit_rate": hit_rate,
            "size": len(self.cache)
        }
```

Opción 2: Caché Persistente con Redis

```
import redis
import json
import pickle

class RedisCacheManager:
    """Caché persistente usando Redis"""

    def __init__(self, host='localhost', port=6379, ttl=3600):
        self.redis_client = redis.Redis(
            host=host,
            port=port,
            decode_responses=True
        )
        self.ttl = ttl

    def get(self, prompt):
        """Obtener del caché Redis"""
        key = self._hash_key(prompt)
        cached = self.redis_client.get(key)
        if cached:
            return json.loads(cached)
        return None

    def set(self, prompt, response):
        """Guardar en caché Redis"""
        key = self._hash_key(prompt)
        self.redis_client.setex(
            key,
            self.ttl,
            json.dumps(response)
        )

    def _hash_key(self, prompt):
        import hashlib
        return hashlib.md5(prompt.encode()).hexdigest()
```

```
    def clear_expired(self):
        """Limpiar entradas expiradas"""
        self.redis_client.flushdb()

    def get_cache_info(self):
        """Información del caché"""
        info = self.redis_client.info()
        return {
            "memory_used": info.get('used_memory_human'),
            "keys": self.redis_client.dbsize()
        }
```

Opción 3: Caché Inteligente con Similaridad Semántica

```
from sentence_transformers import SentenceTransformer
import numpy as np

class SemanticCache:
    """Caché que busca respuestas para prompts similares"""

    def __init__(self, similarity_threshold=0.95):
        self.model = SentenceTransformer('all-MiniLM-L6-v2')
        self.cache = []
        self.similarity_threshold = similarity_threshold

    def get(self, prompt):
        """Buscar respuesta para prompt similar"""
        if not self.cache:
            return None

        # Encodeificar prompt actual
        prompt_embedding = self.model.encode(prompt)

        # Buscar el más similar
        best_match = None
        best_similarity = 0

        for cached_prompt, cached_response, cached_embedding in self.cache:
            # Calcular similitud coseno
            similarity = np.dot(
                prompt_embedding,
                cached_embedding
            ) / (np.linalg.norm(prompt_embedding) *
                 np.linalg.norm(cached_embedding))

            if similarity > best_similarity:
                best_similarity = similarity
                best_match = cached_response
```

```
        if best_similarity >= self.similarity_threshold:
            return best_match

        return None

    def set(self, prompt, response):
        """Guardar con embedding"""
        embedding = self.model.encode(prompt)
        self.cache.append((prompt, response, embedding))

    def get_similar_prompts(self, prompt, top_k=5):
        """Obtener prompts similares"""
        prompt_embedding = self.model.encode(prompt)
        similarities = []

        for cached_prompt, _, cached_embedding in self.cache:
            similarity = np.dot(
                prompt_embedding,
                cached_embedding
            ) / (np.linalg.norm(prompt_embedding) *
                 np.linalg.norm(cached_embedding))
            similarities.append((cached_prompt, similarity))

        return sorted(
            similarities,
            key=lambda x: x[1],
            reverse=True
        )[:top_k]
```

Características Clave:

- □ Caché en memoria rápido
- □ Persistencia con Redis
- □ Búsqueda semántica de similitud
- □ TTL automático
- □ Estadísticas de eficiencia

Ventajas:

- Reducción de costo (50-80% en casos de uso repetitivos)
- Mejora de latencia
- Menor carga en LLM
- Mejor experiencia de usuario

6.3. 6.3. Modelos Locales

Concepto: Integración de modelos open-source locales para reducir costos manteniendo calidad.

Opción 1: Integración Ollama Avanzada

```
import requests
import json

class AdvancedOllamaClient:
    """Cliente Ollama con caché y fallback"""

    def __init__(self, base_url="http://localhost:11434"):
        self.base_url = base_url
        self.available_models = []
        self.model_capabilities = {}
        self.refresh_models()

    def refresh_models(self):
        """Actualizar lista de modelos disponibles"""
        try:
            response = requests.get(f"{self.base_url}/api/tags")
            data = response.json()
            self.available_models = [m['name'] for m in data.get('models', [])]
        except:
            self.available_models = []

    def select_best_model(self, task_type):
        """Seleccionar mejor modelo según tarea"""
        model_rankings = {
            "code": ["mistral", "llama2", "neural-chat"],
            "analysis": ["mistral", "orca-mini", "llama2"],
            "creative": ["mistral", "neural-chat", "zephyr"],
            "fast": ["orca-mini", "neural-chat", "mistral"]
        }

        preferred = model_rankings.get(task_type, self.available_models)
        for model in preferred:
            if model in self.available_models:
                return model

        return self.available_models[0] if self.available_models else "mistral"

    def generate_with_fallback(self, prompt, task_type="general"):
        """Generar con fallback automático"""
        model = self.select_best_model(task_type)

        try:
            return self.generate(prompt, model)
        except:
```

```
            # Fallback a otro modelo
            for alt_model in self.available_models:
                if alt_model != model:
                    try:
                        return self.generate(prompt, alt_model)
                    except:
                        continue

        return "Error: No models available"

    def generate(self, prompt, model="mistral", temperature=0.7):
        """Generar respuesta"""
        payload = {
            "model": model,
            "prompt": prompt,
            "temperature": temperature,
            "stream": False
        }

        response = requests.post(
            f"{self.base_url}/api/generate",
            json=payload
        )

        if response.status_code == 200:
            return response.json().get('response', '')
        raise Exception(f"Error: {response.status_code}")
```

Opción 2: Gestor de Modelos Multi-Proveedor

```
from enum import Enum

class LLMProvider(Enum):
    OLLAMA = "ollama"
    HUGGINGFACE = "huggingface"
    LLAMA_CPP = "llama_cpp"

class MultiProviderLLMManager:
    """Gestor de múltiples proveedores de LLM"""

    def __init__(self):
        self.providers = {}
        self.active_provider = None

    def register_provider(self, name, provider):
        """Registrar proveedor"""
        self.providers[name] = provider
        if self.active_provider is None:
            self.active_provider = name
```

```
    def switch_provider(self, name):
        """Cambiar proveedor activo"""
        if name in self.providers:
            self.active_provider = name
            return True
        return False

    def generate(self, prompt, **kwargs):
        """Generar con proveedor activo"""
        if self.active_provider and self.active_provider in self.providers:
            provider = self.providers[self.active_provider]
            return provider.generate(prompt, **kwargs)
        raise Exception("No active provider")

    def get_provider_stats(self):
        """Estadísticas de proveedores"""
        stats = {}
        for name, provider in self.providers.items():
            stats[name] = {
                "status": "active" if name == self.active_provider else "inactive",
                "available": hasattr(provider, 'is_available') and
provider.is_available()
            }
        return stats
```

Opción 3: Sistema de Caché Multi-Modelo

```
class MultiModelCacheStrategy:
    """Estrategia de caché con preferencia de modelos"""

    def __init__(self, cache_manager, ollama_client):
        self.cache = cache_manager
        self.ollama = ollama_client
        self.model_preferences = {
            "code": {"local": "mistral", "weight": 0.8},
            "analysis": {"local": "llama2", "weight": 0.7},
            "general": {"local": "mistral", "weight": 0.75}
        }

    def get_or_generate(self, prompt, task_type="general"):
        """Obtener del caché o generar"""
        # Buscar en caché
        cached = self.cache.get(prompt)
        if cached:
            return cached, "cache"

        # Generar con modelo local preferido
        model = self.model_preferences[task_type]["local"]
        response = self.ollama.generate(prompt, model)
```

```
        # Guardar en caché
        self.cache.set(prompt, response)

        return response, "generated"

    def get_cost_comparison(self, prompt):
        """Comparar costos: API vs Local"""
        api_cost = len(prompt) / 4 * 0.00002  # Estimado OpenAI
        local_cost = 0  # Después del primer entrenamiento

        return {
            "api_cost": api_cost,
            "local_cost": local_cost,
            "savings": api_cost - local_cost
        }
```

Características Clave:

- Soporte multi-modelo local
- Selección automática según tarea
- Fallback inteligente
- Gestión de múltiples proveedores
- Comparación de costos

Ventajas:

- Costo casi cero después de inversión inicial
- Privacidad total
- Funciona offline
- Control completo
- Customización completa

Casos de Uso:

- Empresas con restricciones de privacidad
- Casos de uso de alto volumen
- Desarrollo local
- Sistemas críticos que requieren independencia

6.4. 7.1. Asistente de Programación

Concepto: Sistema multi-agente para generación, revisión y documentación de código.

Opción 1: Stack Básico de Desarrollo

```
class CodeGeneratorAgent(AssistantAgent):
```

```
    """Agente especializado en generación de código"""

    def __init__(self, name, llm_config):
        super().__init__(name, llm_config)
        self.language = "python"
        self.frameworks = []

    def generate_function(self, requirements):
        """Generar función basada en requisitos"""
        prompt = f"""
        Genera una función Python que cumpla estos requisitos:
        {requirements}

        - Incluye docstring
        - Manejo de errores
        - Type hints
        """
        return self.generate_response(prompt)

    def generate_class(self, class_name, methods):
        """Generar clase con métodos"""
        methods_str = ", ".join(methods)
        prompt = f"""
        Crea una clase {class_name} con estos métodos: {methods_str}
.
        - Usa type hints
        - Docstrings claros
        - Patrón de diseño SOLID
        """
        return self.generate_response(prompt)

class CodeReviewerAgent(AssistantAgent):
    """Agente especializado en revisión de código"""

    def review_code(self, code):
        """Revisar código y sugerir mejoras"""
        prompt = f"""
        Revisa este código y sugiere mejoras:
        ```python
        {code}
        ```

        Evalúa:
.
        - Legibilidad
        - Performance
        - Seguridad
        - Type hints
        """
        return self.generate_response(prompt)
```

Opción 2: Sistema de Desarrollo Colaborativo

```
class ProgrammingTeam:
    """Equipo de desarrollo con múltiples especialistas"""

    def __init__(self, llm_config):
        self.generator = CodeGeneratorAgent("CodeGen", llm_config)
        self.reviewer = CodeReviewerAgent("CodeReviewer", llm_config)
        self.documenter = DocumenterAgent("Documenter", llm_config)
        self.tester = TestWriterAgent("TestWriter", llm_config)

    def develop_feature(self, feature_spec):
        """Desarrollo completo de feature"""
        # 1. Generar código
        code = self.generator.generate_function(feature_spec)

        # 2. Revisar
        review = self.reviewer.review_code(code)

        # 3. Escribir tests
        tests = self.tester.generate_tests(code)

        # 4. Documentar
        docs = self.documenter.generate_docs(code)

        return {
            "code": code,
            "review": review,
            "tests": tests,
            "docs": docs
        }

    def debug_code(self, code, error_message):
        """Debugging colaborativo"""
        analysis = self.analyzer.analyze_error(code, error_message)
        fix = self.generator.generate_fix(code, analysis)
        review = self.reviewer.review_fix(fix)
        return {"fix": fix, "review": review}
```

Opción 3: IDE Inteligente con AutoGen

```
class IntelligentCodeEditor:
    """Editor de código mejorado con IA"""

    def __init__(self, llm_config):
        self.code_assistant = CodeGeneratorAgent("Assistant", llm_config)
        self.context = []
        self.suggestions_cache = {}
```

```
    def suggest_completion(self, code_prefix):
        """Completación inteligente de código"""
        if code_prefix in self.suggestions_cache:
            return self.suggestions_cache[code_prefix]

        prompt = f"""
        Completa este código Python de forma inteligente:
        {code_prefix}

        - Continúa de forma natural
        - Mantén consistencia de estilo
        - Sigue el contexto
        """
        completion = self.code_assistant.generate_response(prompt)
        self.suggestions_cache[code_prefix] = completion
        return completion

    def explain_code(self, code_snippet):
        """Explicar código seleccionado"""
        prompt = f"""
        Explica este código Python de forma clara y educativa:
        ```python
        {code_snippet}
        ```
        """
        return self.code_assistant.generate_response(prompt)

    def refactor_code(self, code):
        """Refactorización automática"""
        prompt = f"""
        Refactoriza este código para mejorar:
.
        - Legibilidad
        - Mantenibilidad
        - Performance
        ```python
        {code}
        ```
        """
        return self.code_assistant.generate_response(prompt)
```

Características Clave:

- □ Generación de código funcional
- □ Revisión automática de calidad
- □ Testing automático
- □ Documentación inteligente
- □ Debugging colaborativo

6.5. 7.2. Análisis de Datos

Concepto: Sistema para procesamiento, análisis y visualización automática de datos.

Opción 1: Pipeline de Análisis Básico

```
import pandas as pd
import numpy as np

class DataAnalystAgent(AssistantAgent):
    """Agente especializado en análisis de datos"""

    def __init__(self, name, llm_config):
        super().__init__(name, llm_config)
        self.data = None
        self.analysis_history = []

    def load_data(self, file_path):
        """Cargar datos"""
        self.data = pd.read_csv(file_path)
        return self.get_data_summary()

    def get_data_summary(self):
        """Resumen de datos"""
        return {
            "shape": self.data.shape,
            "columns": self.data.columns.tolist(),
            "dtypes": self.data.dtypes.to_dict(),
            "missing": self.data.isnull().sum().to_dict(),
            "stats": self.data.describe().to_dict()
        }

    def explore_data(self):
        """Exploración automática"""
        prompt = f"""
        Analiza estos datos y sugiere insights:

        Forma: {self.data.shape}
        Columnas: {self.data.columns.tolist()}
        Resumen estadístico:
        {self.data.describe()}
        """
        analysis = self.generate_response(prompt)
        self.analysis_history.append(analysis)
        return analysis

    def answer_question(self, question):
        """Responder preguntas sobre datos"""
        # Generar consulta SQL/pandas automáticamente
```

```
        prompt = f"""
        Basándote en estos datos:
        {self.data.head()}

        Pregunta: {question}

        Genera una respuesta análitica y visualización sugerida
        """
        return self.generate_response(prompt)
```

Opción 2: Sistema de Reporting Automático

```
class AutomaticReportGenerator:
    """Generador de reportes automáticos"""

    def __init__(self, llm_config):
        self.analyst = DataAnalystAgent("Analyst", llm_config)
        self.visualizer = VisualizationAgent("Visualizer", llm_config)

    def generate_executive_summary(self, data):
        """Resumen ejecutivo automático"""
        summary = self.analyst.get_data_summary()

        prompt = f"""
        Crea un resumen ejecutivo de estos datos:
        {summary}

        Incluye:
        - 3 Key Performance Indicators (KPIs)
        - 3 Insights principales
        - 3 Recomendaciones
        """

        return self.analyst.generate_response(prompt)

    def generate_full_report(self, data, analysis_type="comprehensive"):
        """Reporte completo con secciones"""
        sections = {
            "executive_summary": self.generate_executive_summary(data),
            "data_overview": self.analyst.get_data_summary(),
            "detailed_analysis": self.analyst.explore_data(),
            "visualizations": self.visualizer.suggest_charts(data),
            "recommendations": self.generate_recommendations(data)
        }

        return sections

    def generate_recommendations(self, data):
        """Recomendaciones basadas en análisis"""
        prompt = f"""
```

```
        Basándote en estos datos, sugiere acciones:
        {data.describe()}

        Incluye:
        - Acciones inmediatas
        - Mejoras de medio plazo
        - Estrategia a largo plazo
        """
        return self.analyst.generate_response(prompt)
```

Opción 3: BI Conversacional

```
class ConversationalBI:
    """Business Intelligence conversacional"""

    def __init__(self, llm_config):
        self.analyst = DataAnalystAgent("BiAgent", llm_config)
        self.conversation_history = []
        self.data = None

    def chat(self, user_question):
        """Chat conversacional sobre datos"""
        # Guardar en historial
        self.conversation_history.append({
            "user": user_question,
            "timestamp": datetime.now()
        })

        # Generar respuesta considerando contexto
        context = self._get_conversation_context()

        prompt = f"""
        Contexto de conversación:
        {context}

        Pregunta del usuario: {user_question}

        Datos disponibles:
        {self.data.head() if self.data is not None else "No data loaded"}

        Responde de forma natural y proporciona insights
        """

        response = self.analyst.generate_response(prompt)

        self.conversation_history[-1]["response"] = response

        return response

    def _get_conversation_context(self):
```

```
        """Contexto de conversación anterior"""
        if not self.conversation_history:
            return "Inicio de conversación"

        recent = self.conversation_history[-5:]  # Últimas 5 interacciones
        return "\n".join([
            f"Usuario: {h['user']}\nAsistente: {h.get('response', '')}"
            for h in recent
        ])
```

Características Clave:

- □ Exploración automática de datos
- □ Generación de reportes
- □ Visualizaciones inteligentes
- □ Q&A conversacional
- □ Recomendaciones automáticas

6.6. 7.3. Automatización de Tareas

Concepto: Flujos de trabajo automatizados para tareas empresariales repetitivas.

Opción 1: Orquestador de Tareas Simple

```
from enum import Enum
from datetime import datetime

class TaskStatus(Enum):
    PENDING = "pending"
    IN_PROGRESS = "in_progress"
    COMPLETED = "completed"
    FAILED = "failed"

class TaskOrchestrator:
    """Orquestador de tareas automático"""

    def __init__(self, llm_config):
        self.tasks = []
        self.agent = AssistantAgent("TaskOrchestrator", llm_config)

    def create_workflow(self, workflow_name, steps):
        """Crear flujo de trabajo"""
        workflow = {
            "name": workflow_name,
            "steps": steps,
            "created_at": datetime.now(),
```

```
            "status": TaskStatus.PENDING
        }
        self.tasks.append(workflow)
        return workflow

    def execute_workflow(self, workflow_name):
        """Ejecutar flujo de trabajo"""
        workflow = next(w for w in self.tasks if w["name"] == workflow_name)

        results = []
        for step in workflow["steps"]:
            result = self._execute_step(step)
            results.append(result)

        workflow["status"] = TaskStatus.COMPLETED
        return results

    def _execute_step(self, step):
        """Ejecutar paso individual"""
        prompt = f"""
        Ejecuta esta tarea:
        Tipo: {step.get('type')}
        Descripción: {step.get('description')}
        Parámetros: {step.get('params')}

        Proporciona resultado y estado
        """
        return self.agent.generate_response(prompt)
```

Opción 2: Gestor de Procesos Inteligente

```
class IntelligentProcessManager:
    """Gestor de procesos con IA"""

    def __init__(self, llm_config):
        self.processes = {}
        self.optimizer = ProcessOptimizer()
        self.monitor = ProcessMonitor()

    def automate_process(self, process_name, process_def):
        """Automatizar proceso"""
        # Analizar y optimizar
        optimized = self.optimizer.analyze(process_def)

        # Crear ejecutable
        self.processes[process_name] = {
            "definition": process_def,
            "optimized": optimized,
            "status": "ready"
        }
```

```
    def execute_process(self, process_name, inputs):
        """Ejecutar proceso"""
        process = self.processes[process_name]

        # Monitorear durante ejecución
        execution = {
            "process": process_name,
            "inputs": inputs,
            "start_time": datetime.now(),
            "status": "running"
        }

        # Ejecutar pasos
        for step in process["optimized"]["steps"]:
            execution["outputs"] = self._run_step(step, inputs)

        execution["end_time"] = datetime.now()
        execution["status"] = "completed"

        return execution

    def get_performance_report(self, process_name):
        """Reporte de performance"""
        executions = self.monitor.get_executions(process_name)

        return {
            "total_runs": len(executions),
            "avg_duration": np.mean([e["duration"] for e in executions]),
            "success_rate": sum(1 for e in executions if e["status"] == "completed") /
len(executions),
            "bottlenecks": self.optimizer.identify_bottlenecks(executions)
        }
```

Opción 3: RPA Inteligente

```
class IntelligentRPA:
    """Automatización inteligente de procesos"""

    def __init__(self, llm_config):
        self.bots = {}
        self.scheduler = BackgroundScheduler()

    def create_bot(self, bot_name, trigger_rules, actions):
        """Crear bot RPA"""
        self.bots[bot_name] = {
            "triggers": trigger_rules,
            "actions": actions,
            "active": True
        }
```

```
    def schedule_bot(self, bot_name, cron_expression):
        """Agendar bot"""
        def run_bot():
            return self._execute_bot(bot_name)

        self.scheduler.add_job(
            run_bot,
            trigger="cron",
            **self._parse_cron(cron_expression),
            id=bot_name
        )

    def _execute_bot(self, bot_name):
        """Ejecutar bot"""
        bot = self.bots[bot_name]

        for action in bot["actions"]:
            if action["type"] == "read_email":
                self._handle_email(action)
            elif action["type"] == "process_document":
                self._process_document(action)
            elif action["type"] == "update_system":
                self._update_system(action)

        return {"status": "completed", "bot": bot_name}
```

Características Clave:

- □ Automatización de flujos
- □ Monitoreo en tiempo real
- □ Optimización de procesos
- □ RPA inteligente
- □ Reportes de performance

6.7. 7.4. Investigación y Síntesis

Concepto: Búsqueda inteligente de información y síntesis de conocimiento.

Opción 1: Investigador Automático Básico

```
class ResearchAgent(AssistantAgent):
    """Agente de investigación automatizada"""

    def __init__(self, name, llm_config):
        super().__init__(name, llm_config)
```

```
        self.sources = []
        self.findings = []

    def research_topic(self, topic, num_sources=5):
        """Investigar tema"""
        prompt = f"""
        Investiga el tema: {topic}

        Proporciona:
        1. Definición clara
        2. Historia y evolución
        3. Actores principales
        4. Tendencias actuales
        5. Desafíos
        6. Oportunidades
        """

        findings = self.generate_response(prompt)
        self.findings.append({
            "topic": topic,
            "findings": findings,
            "timestamp": datetime.now()
        })

        return findings

    def compare_topics(self, topic1, topic2):
        """Comparar dos temas"""
        prompt = f"""
        Compara {topic1} vs {topic2}

        Incluye:
        - Similitudes
        - Diferencias
        - Ventajas/desventajas
        - Casos de uso
        """
        return self.generate_response(prompt)

    def verify_facts(self, claims):
        """Verificar hechos"""
        prompt = f"""
        Verifica estos claims:
        {claims}

        Para cada uno:
        - Confirma o refuta
        - Proporciona evidencia
        - Indica nivel de certeza
        """
```

```
        return self.generate_response(prompt)
```

Opción 2: Sistema de Síntesis Avanzado

```
class KnowledgeSynthesizer:
    """Sintetizador de conocimiento"""

    def __init__(self, llm_config):
        self.researcher = ResearchAgent("Researcher", llm_config)
        self.knowledge_base = {}

    def synthesize_knowledge(self, sources):
        """Sintetizar múltiples fuentes"""
        synthesis = {
            "sources": sources,
            "key_concepts": self._extract_concepts(sources),
            "connections": self._find_connections(sources),
            "summary": self._generate_summary(sources),
            "gaps": self._identify_gaps(sources),
            "next_steps": self._suggest_next_steps(sources)
        }

        return synthesis

    def create_knowledge_graph(self, topic):
        """Crear grafo de conocimiento"""
        research = self.researcher.research_topic(topic)

        # Extraer entidades y relaciones
        prompt = f"""
        Crea un grafo de conocimiento para:
        {research}

        Identifica:
        - Nodos principales
        - Relaciones entre nodos
        - Jerarquía conceptual
        """

        return self.researcher.generate_response(prompt)

    def generate_literature_review(self, topic, num_papers=10):
        """Generar revisión de literatura"""
        research = self.researcher.research_topic(topic)

        prompt = f"""
        Basándote en esta investigación:
        {research}

        Genera una revisión de literatura que incluya:
```

```
        - Estado del arte
        - Clasificación de trabajos
        - Vacíos de investigación
        - Oportunidades futuras
        """

        return self.researcher.generate_response(prompt)
```

Opción 3: Analista Inteligente de Información

```
class IntelligentInformationAnalyzer:
    """Analizador inteligente de información"""

    def __init__(self, llm_config):
        self.researcher = ResearchAgent("Analyzer", llm_config)
        self.analysis_cache = {}

    def analyze_document(self, document):
        """Analizar documento"""
        if document in self.analysis_cache:
            return self.analysis_cache[document]

        prompt = f"""
        Analiza este documento:
        {document}

        Extrae:
        - Idea principal
        - Puntos clave
        - Conclusiones
        - Limitaciones
        - Validez de argumentos
        """

        analysis = self.researcher.generate_response(prompt)
        self.analysis_cache[document] = analysis
        return analysis

    def identify_misinformation(self, claim):
        """Identificar desinformación"""
        prompt = f"""
        Analiza si este claim es verdadero o falso:
        {claim}

        Proporciona:
        - Evaluación (verdadero/falso/parcial)
        - Evidencia
        - Fuentes confiables
        - Contexto completo
        """
```

```
        return self.researcher.generate_response(prompt)

    def synthesize_debate(self, topic, perspectives):
        """Sintetizar debate"""
        prompt = f"""
        Sintetiza este debate sobre {topic}:

        Perspectiva A: {perspectives['a']}
        Perspectiva B: {perspectives['b']}

        Proporciona:
        - Puntos comunes
        - Puntos de divergencia
        - Posibles consensos
        - Preguntas sin resolver
        """

        return self.researcher.generate_response(prompt)
```

Características Clave:

- □ Investigación automatizada
- □ Síntesis de múltiples fuentes
- □ Verificación de hechos
- □ Análisis inteligente
- □ Detección de desinformación

6.8. 8.1. Estrategias de Testing

Concepto: Frameworks de testing para validar comportamiento y calidad de agentes.

Opción 1: Unit Testing Básico

```
import unittest
from unittest.mock import Mock, patch

class AgentTestCase(unittest.TestCase):
    """Casos de prueba para agentes"""

    def setUp(self):
        """Configuración de pruebas"""
        self.agent = AssistantAgent("TestAgent", llm_config)
        self.mock_llm = Mock()

    def test_agent_initialization(self):
        """Prueba inicialización de agente"""
        self.assertIsNotNone(self.agent.name)
```

```
        self.assertEqual(self.agent.name, "TestAgent")

    def test_generate_response(self):
        """Prueba generación de respuesta"""
        with patch.object(self.agent, 'generate_response') as mock_gen:
            mock_gen.return_value = "Test response"
            response = self.agent.generate_response("Test prompt")
            self.assertEqual(response, "Test response")
            mock_gen.assert_called_once()

    def test_response_format(self):
        """Prueba formato de respuesta"""
        response = self.agent.generate_response("Hola")
        self.assertIsInstance(response, str)
        self.assertGreater(len(response), 0)

    def test_error_handling(self):
        """Prueba manejo de errores"""
        with self.assertRaises(ValueError):
            self.agent.generate_response("")
```

Opción 2: Testing de Conversaciones

```
class ConversationTestSuite:
    """Suite de pruebas para conversaciones multi-agente"""

    def __init__(self):
        self.test_conversations = []
        self.results = []

    def test_two_agent_conversation(self):
        """Prueba conversación de dos agentes"""
        agent1 = AssistantAgent("Alice", llm_config)
        agent2 = AssistantAgent("Bob", llm_config)

        # Simular conversación
        message1 = agent1.generate_response("Hola Bob")
        message2 = agent2.generate_response(message1)
        message3 = agent1.generate_response(message2)

        # Validaciones
        assert len(message1) > 0, "Agent 1 should produce output"
        assert len(message2) > 0, "Agent 2 should produce output"
        assert len(message3) > 0, "Agent 1 should respond again"

        return {
            "passed": True,
            "messages": [message1, message2, message3]
        }
```

```
    def test_conversation_coherence(self, conversation):
        """Prueba coherencia de conversación"""
        # Verificar que los temas se mantienen
        for i in range(len(conversation) - 1):
            current = conversation[i]
            next_msg = conversation[i + 1]

            # Validar continuidad temática
            assert self._are_related(current, next_msg), \
                f"Messages {i} and {i+1} are not thematically related"

    def test_output_validation(self, response):
        """Validar output de agente"""
        validations = {
            "non_empty": len(response) > 0,
            "valid_json": self._is_valid_json(response),
            "contains_key_terms": any(term in response for term in ["hello", "hola"]),
            "language_appropriate": self._check_language(response)
        }
        return validations

    def _are_related(self, text1, text2):
        """Verificar relación temática entre textos"""
        # Implementación simplificada
        return True

    def _is_valid_json(self, text):
        """Verificar si es JSON válido"""
        try:
            json.loads(text)
            return True
        except:
            return False

    def _check_language(self, text):
        """Verificar idioma apropiado"""
        return len(text) > 10  # Simplificado
```

Opción 3: Framework Avanzado de Testing

```
class AdvancedTestFramework:
    """Framework avanzado para testing de agentes"""

    def __init__(self):
        self.test_suites = {}
        self.metrics = {}
        self.coverage = {}

    def create_test_suite(self, name, test_cases):
        """Crear suite de pruebas"""
```

```
        self.test_suites[name] = {
            "cases": test_cases,
            "results": [],
            "passed": 0,
            "failed": 0
        }

    def run_test_suite(self, suite_name):
        """Ejecutar suite completa"""
        suite = self.test_suites[suite_name]

        for test_case in suite["cases"]:
            try:
                result = test_case()
                suite["results"].append({
                    "name": test_case.__name__,
                    "status": "passed",
                    "result": result
                })
                suite["passed"] += 1
            except Exception as e:
                suite["results"].append({
                    "name": test_case.__name__,
                    "status": "failed",
                    "error": str(e)
                })
                suite["failed"] += 1

        return self._generate_report(suite)

    def measure_code_coverage(self, agents):
        """Medir cobertura de código"""
        coverage_data = {}

        for agent in agents:
            methods = [m for m in dir(agent) if not m.startswith('_')]
            tested_methods = self._get_tested_methods(agent.__class__)

            coverage_data[agent.name] = {
                "total_methods": len(methods),
                "tested_methods": len(tested_methods),
                "coverage_percentage": (len(tested_methods) / len(methods)) * 100
            }

        return coverage_data

    def _generate_report(self, suite):
        """Generar reporte de pruebas"""
        total = suite["passed"] + suite["failed"]
        pass_rate = (suite["passed"] / total * 100) if total > 0 else 0
```

```
        return {
            "total_tests": total,
            "passed": suite["passed"],
            "failed": suite["failed"],
            "pass_rate": pass_rate,
            "details": suite["results"]
        }

    def _get_tested_methods(self, agent_class):
        """Obtener métodos testeados"""
        # Implementación simplificada
        return []
```

Características Clave:

- Unit testing completo
- Testing de conversaciones
- Validación de outputs
- Medición de cobertura
- Reportes detallados

6.9. 8.2. Debugging

Concepto: Herramientas y estrategias para identificar y resolver problemas.

Opción 1: Sistema de Logging Básico

```
import logging
from datetime import datetime

class AgentLogger:
    """Sistema de logging para agentes"""

    def __init__(self, agent_name):
        self.agent_name = agent_name
        self.logs = []

        # Configurar logging estándar
        self.logger = logging.getLogger(agent_name)
        self.logger.setLevel(logging.DEBUG)

        # Crear handler
        handler = logging.FileHandler(f"{agent_name}.log")
        formatter = logging.Formatter(
            '%(asctime)s - %(levelname)s - %(message)s'
        )
```

```
        handler.setFormatter(formatter)
        self.logger.addHandler(handler)

    def log_action(self, action, details=None):
        """Registrar acción"""
        log_entry = {
            "timestamp": datetime.now(),
            "action": action,
            "details": details
        }
        self.logs.append(log_entry)
        self.logger.info(f"{action}: {details}")

    def log_error(self, error, context=None):
        """Registrar error"""
        error_entry = {
            "timestamp": datetime.now(),
            "error": str(error),
            "context": context
        }
        self.logs.append(error_entry)
        self.logger.error(f"Error: {error} | Context: {context}")

    def get_trace(self, action_filter=None):
        """Obtener trazabilidad completa"""
        if action_filter:
            return [log for log in self.logs if action_filter in log["action"]]
        return self.logs

    def generate_debug_report(self):
        """Generar reporte de debug"""
        return {
            "agent": self.agent_name,
            "total_actions": len(self.logs),
            "errors": len([l for l in self.logs if "error" in l]),
            "timeline": self.logs
        }
```

Opción 2: Debugger Interactivo

```
class InteractiveDebugger:
    """Debugger interactivo para agentes"""

    def __init__(self, agent):
        self.agent = agent
        self.breakpoints = {}
        self.watched_variables = {}
        self.execution_history = []

    def set_breakpoint(self, method_name):
```

```
        """Establecer breakpoint"""
        self.breakpoints[method_name] = True
        print(f"Breakpoint set at {method_name}")

    def watch_variable(self, var_name):
        """Monitorear variable"""
        self.watched_variables[var_name] = None
        print(f"Watching {var_name}")

    def step_through(self, prompt):
        """Ejecutar paso a paso"""
        # Capturar estado inicial
        initial_state = self._capture_state()

        # Ejecutar
        result = self.agent.generate_response(prompt)

        # Capturar estado final
        final_state = self._capture_state()

        # Registrar cambios
        step_info = {
            "prompt": prompt,
            "result": result,
            "state_changes": self._compare_states(initial_state, final_state)
        }
        self.execution_history.append(step_info)

        return step_info

    def inspect_state(self):
        """Inspeccionar estado actual"""
        return {
            "agent_name": self.agent.name,
            "variables": self.watched_variables,
            "recent_actions": self.execution_history[-5:]
        }

    def _capture_state(self):
        """Capturar estado del agente"""
        return {
            "timestamp": datetime.now(),
            "variables": dict(self.watched_variables)
        }

    def _compare_states(self, state1, state2):
        """Comparar estados"""
        changes = {}
        for var in self.watched_variables:
            if var in state1["variables"] and var in state2["variables"]:
                if state1["variables"][var] != state2["variables"][var]:
```

```
                    changes[var] = {
                        "before": state1["variables"][var],
                        "after": state2["variables"][var]
                    }
        return changes
```

Opción 3: Error Recovery y Auto-Healing

```
class ErrorRecoverySystem:
    """Sistema automático de recuperación de errores"""

    def __init__(self, agent):
        self.agent = agent
        self.error_patterns = {}
        self.recovery_strategies = {}
        self.error_history = []

    def register_error_pattern(self, error_type, detection_fn):
        """Registrar patrón de error"""
        self.error_patterns[error_type] = detection_fn

    def register_recovery_strategy(self, error_type, strategy_fn):
        """Registrar estrategia de recuperación"""
        self.recovery_strategies[error_type] = strategy_fn

    def execute_with_recovery(self, prompt, max_retries=3):
        """Ejecutar con recuperación automática"""
        for attempt in range(max_retries):
            try:
                result = self.agent.generate_response(prompt)
                return {"success": True, "result": result, "attempts": attempt + 1}

            except Exception as e:
                error_type = type(e).__name__

                # Detectar patrón de error
                detected_pattern = self._detect_error_pattern(e)

                if detected_pattern and detected_pattern in self.recovery_strategies:
                    # Aplicar estrategia de recuperación
                    recovery_fn = self.recovery_strategies[detected_pattern]
                    self.agent = recovery_fn(self.agent, e)
                    print(f"Applied recovery strategy: {detected_pattern}")
                else:
                    if attempt == max_retries - 1:
                        raise
                    print(f"Retry {attempt + 1}/{max_retries}...")

                self.error_history.append({
                    "attempt": attempt + 1,
```

```
                    "error": str(e),
                    "pattern": detected_pattern
                })

        return {"success": False, "error": "Max retries exceeded"}

    def _detect_error_pattern(self, error):
        """Detectar patrón de error"""
        for pattern_name, detection_fn in self.error_patterns.items():
            if detection_fn(error):
                return pattern_name
        return None

    def get_error_statistics(self):
        """Estadísticas de errores"""
        from collections import Counter
        error_types = Counter([e["pattern"] for e in self.error_history])
        return {
            "total_errors": len(self.error_history),
            "error_breakdown": dict(error_types),
            "recent_errors": self.error_history[-10:]
        }
```

Características Clave:

- Logging detallado
- Debugging interactivo
- Rastreo de ejecución
- Recuperación automática
- Análisis de errores

6.10. 8.3. Evaluación de Calidad

Concepto: Métricas y herramientas para medir y mejorar la calidad de agentes.

Opción 1: Framework de Métricas Básico

```
class QualityMetrics:
    """Métricas de calidad para agentes"""

    def __init__(self):
        self.metrics = {}

    def measure_response_quality(self, response, expected=None):
        """Medir calidad de respuesta"""
        quality_score = {
```

```
            "length": self._score_length(response),
            "coherence": self._score_coherence(response),
            "completeness": self._score_completeness(response),
            "accuracy": self._score_accuracy(response, expected) if expected else None
        }

        overall_score = sum(
            v for v in quality_score.values() if v is not None
        ) / len([v for v in quality_score.values() if v is not None])

        return {
            "scores": quality_score,
            "overall": overall_score,
            "grade": self._calculate_grade(overall_score)
        }

    def benchmark_agents(self, agents, test_prompts):
        """Comparar rendimiento de agentes"""
        results = {}

        for agent in agents:
            scores = []
            for prompt in test_prompts:
                response = agent.generate_response(prompt)
                quality = self.measure_response_quality(response)
                scores.append(quality["overall"])

            results[agent.name] = {
                "avg_score": sum(scores) / len(scores),
                "scores": scores
            }

        # Ranking
        ranking = sorted(
            results.items(),
            key=lambda x: x[1]["avg_score"],
            reverse=True
        )

        return {
            "results": results,
            "ranking": ranking,
            "best_agent": ranking[0][0] if ranking else None
        }

    def _score_length(self, response):
        """Puntuación de longitud"""
        word_count = len(response.split())
        if 50 < word_count < 500:
            return 1.0
        elif 20 < word_count < 1000:
```

```
            return 0.8
        else:
            return 0.5

    def _score_coherence(self, response):
        """Puntuación de coherencia"""
        # Simplificado: verificar que tiene puntuación
        has_punctuation = any(p in response for p in ".!?")
        return 0.9 if has_punctuation else 0.5

    def _score_completeness(self, response):
        """Puntuación de completitud"""
        sentences = response.split(".")
        return min(len(sentences) / 3, 1.0)

    def _score_accuracy(self, response, expected):
        """Puntuación de precisión"""
        if expected.lower() in response.lower():
            return 1.0
        return 0.5

    def _calculate_grade(self, score):
        """Calcular calificación"""
        if score >= 0.9:
            return "A"
        elif score >= 0.8:
            return "B"
        elif score >= 0.7:
            return "C"
        else:
            return "D"
```

Opción 2: Evaluación Continua

```
class ContinuousEvaluation:
    """Evaluación continua de agentes"""

    def __init__(self, agent):
        self.agent = agent
        self.evaluation_history = []
        self.performance_trends = {}

    def evaluate(self, test_set):
        """Evaluar agente con test set"""
        results = []

        for test_case in test_set:
            prompt = test_case["prompt"]
            expected = test_case.get("expected")
```

```
            response = self.agent.generate_response(prompt)

            # Evaluar respuesta
            score = self._evaluate_response(response, expected)
            results.append({
                "prompt": prompt,
                "response": response,
                "score": score
            })

        # Guardar evaluación
        evaluation = {
            "timestamp": datetime.now(),
            "agent": self.agent.name,
            "results": results,
            "avg_score": sum(r["score"] for r in results) / len(results)
        }

        self.evaluation_history.append(evaluation)
        self._update_trends()

        return evaluation

    def identify_weak_areas(self):
        """Identificar áreas débiles"""
        if not self.evaluation_history:
            return {}

        latest = self.evaluation_history[-1]
        weak_prompts = [
            r for r in latest["results"]
            if r["score"] < 0.7
        ]

        return {
            "weak_areas": weak_prompts,
            "improvement_needed": len(weak_prompts) > 0
        }

    def _evaluate_response(self, response, expected):
        """Evaluar respuesta individual"""
        if not expected:
            return 0.5  # Sin expectativa

        # Comparación simple de similitud
        response_lower = response.lower()
        expected_lower = expected.lower()

        similarity = sum(
            1 for word in expected_lower.split()
            if word in response_lower
```

```
        ) / len(expected_lower.split())

        return min(similarity, 1.0)

    def _update_trends(self):
        """Actualizar tendencias de rendimiento"""
        if len(self.evaluation_history) >= 2:
            recent = self.evaluation_history[-2:]
            trend = recent[-1]["avg_score"] - recent[-2]["avg_score"]
            self.performance_trends["latest"] = trend

    def get_performance_report(self):
        """Reporte de rendimiento"""
        if not self.evaluation_history:
            return {}

        scores = [e["avg_score"] for e in self.evaluation_history]

        return {
            "evaluations": len(self.evaluation_history),
            "current_score": scores[-1],
            "avg_score": sum(scores) / len(scores),
            "best_score": max(scores),
            "trend": self.performance_trends.get("latest", 0)
        }
```

Opción 3: Quality Assurance Avanzado

```
class AdvancedQualityAssurance:
    """QA avanzado con múltiples validadores"""

    def __init__(self):
        self.validators = {}
        self.thresholds = {}
        self.quality_rules = []

    def register_validator(self, name, validator_fn, threshold=0.7):
        """Registrar validador"""
        self.validators[name] = validator_fn
        self.thresholds[name] = threshold

    def add_quality_rule(self, rule):
        """Agregar regla de calidad"""
        self.quality_rules.append(rule)

    def validate_response(self, response):
        """Validar respuesta contra todos los validadores"""
        validation_results = {}

        for name, validator_fn in self.validators.items():
```

```
            score = validator_fn(response)
            threshold = self.thresholds[name]
            validation_results[name] = {
                "score": score,
                "threshold": threshold,
                "passed": score >= threshold
            }

        # Aplicar reglas
        rule_results = []
        for rule in self.quality_rules:
            passed = rule(response, validation_results)
            rule_results.append(passed)

        # Resultado general
        all_passed = all(validation_results[k]["passed"] for k in validation_results)
        all_rules_passed = all(rule_results) if rule_results else True

        return {
            "validators": validation_results,
            "rules": rule_results,
            "overall_quality": all_passed and all_rules_passed,
            "quality_score": sum(v["score"] for v in validation_results.values()) /
len(validation_results) if validation_results else 0
        }
```

Características Clave:

- □ Múltiples métricas de calidad
- □ Benchmarking comparativo
- □ Evaluación continua
- □ Identificación de debilidades
- □ Reportes de rendimiento

Chapter 7. Módulo 9: Despliegue en Producción

7.1. 9.1. Arquitectura de Producción

Concepto: Diseño escalable y resiliente de sistemas de agentes en producción.

Opción 1: Arquitectura Microservicios Básica

```
from fastapi import FastAPI
from pydantic import BaseModel
import uvicorn

class AgentRequest(BaseModel):
    prompt: str
    agent_id: str

class AgentResponse(BaseModel):
    response: str
    agent_id: str
    timestamp: str

app = FastAPI()

class ProductionAgent:
    """Agente preparado para producción"""

    def __init__(self, agent_id, llm_config):
        self.agent_id = agent_id
        self.agent = AssistantAgent(agent_id, llm_config)
        self.request_queue = []
        self.response_cache = {}

    @app.post("/query")
    async def query(self, request: AgentRequest) -> AgentResponse:
        """Endpoint de consulta"""
        try:
            response = self.agent.generate_response(request.prompt)

            return AgentResponse(
                response=response,
                agent_id=self.agent_id,
                timestamp=datetime.now().isoformat()
            )
        except Exception as e:
            return {"error": str(e)}

    @app.get("/health")
```

```
    async def health_check():
        """Health check"""
        return {
            "status": "healthy",
            "agent_id": self.agent_id,
            "timestamp": datetime.now().isoformat()
        }

    @app.get("/metrics")
    async def get_metrics():
        """Métricas del agente"""
        return {
            "total_requests": len(self.request_queue),
            "cache_size": len(self.response_cache)
        }
```

Opción 2: Sistema Distribuido Escalable

```
from kubernetes import client, config
import docker

class DistributedAgentManager:
    """Gestor de agentes distribuidos"""

    def __init__(self, deployment_config):
        self.config = deployment_config
        self.agents = {}
        self.load_balancer = LoadBalancer()

    def deploy_agent(self, agent_config):
        """Desplegar agente en contenedor"""
        # Crear imagen Docker
        dockerfile_content = f"""
        FROM python:3.9
        RUN pip install autogen ollama
        COPY agent.py /app/
        WORKDIR /app
        CMD ["python", "agent.py"]
        """

        # Construir imagen
        docker_client = docker.from_env()
        image = docker_client.images.build(
            fileobj=dockerfile_content.encode(),
            tag=f"agent-{agent_config['name']}:latest"
        )

        # Desplegar en Kubernetes
        self._deploy_kubernetes(agent_config, image[0].id)
```

```
    def scale_agent(self, agent_name, replicas):
        """Escalar agente a N réplicas"""
        config.load_incluster_config()
        v1 = client.AppsV1Api()

        deployment = v1.read_namespaced_deployment(
            agent_name,
            "default"
        )

        deployment.spec.replicas = replicas
        v1.patch_namespaced_deployment(
            agent_name,
            "default",
            deployment
        )

    def route_request(self, request):
        """Enrutar solicitud a agente disponible"""
        available_agents = self._get_healthy_agents()
        selected_agent = self.load_balancer.select(available_agents)
        return selected_agent.handle_request(request)

    def _get_healthy_agents(self):
        """Obtener agentes saludables"""
        healthy = []
        for agent_name, agent in self.agents.items():
            if agent.health_check():
                healthy.append(agent)
        return healthy
```

Opción 3: Estado Persistente y Recuperación

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
import json

class PersistentAgentState:
    """Persistencia de estado de agente"""

    def __init__(self, db_url="postgresql://user:pass@localhost/agents"):
        self.engine = create_engine(db_url)
        self.Session = sessionmaker(bind=self.engine)

    def save_conversation(self, agent_id, conversation):
        """Guardar conversación"""
        session = self.Session()
        try:
            db_conversation = ConversationRecord(
                agent_id=agent_id,
```

```
                content=json.dumps(conversation),
                created_at=datetime.now()
            )
            session.add(db_conversation)
            session.commit()
        finally:
            session.close()

    def load_conversation(self, agent_id, conversation_id):
        """Cargar conversación anterior"""
        session = self.Session()
        try:
            record = session.query(ConversationRecord).filter(
                ConversationRecord.agent_id == agent_id,
                ConversationRecord.id == conversation_id
            ).first()

            if record:
                return json.loads(record.content)
            return None
        finally:
            session.close()

    def save_agent_state(self, agent_id, state):
        """Guardar estado del agente"""
        session = self.Session()
        try:
            state_record = AgentStateRecord(
                agent_id=agent_id,
                state_data=json.dumps(state),
                saved_at=datetime.now()
            )
            session.add(state_record)
            session.commit()
            return state_record.id
        finally:
            session.close()

    def restore_agent(self, agent_id):
        """Restaurar agente desde estado guardado"""
        session = self.Session()
        try:
            latest_state = session.query(AgentStateRecord).filter(
                AgentStateRecord.agent_id == agent_id
            ).order_by(AgentStateRecord.saved_at.desc()).first()

            if latest_state:
                return json.loads(latest_state.state_data)
            return None
        finally:
```

```
            session.close()
```

Características Clave:

- □ Microservicios escalables
- □ Deploymentc con contenedores
- □ Load balancing
- □ Persistencia de estado
- □ Alta disponibilidad

7.2. 9.2. Seguridad

Concepto: Medidas de seguridad para proteger agentes en producción.

Opción 1: Sanitización de Entrada

```
import re
from html import escape
import bleach

class InputSanitizer:
    """Sanitizador de inputs"""

    def __init__(self):
        self.max_length = 10000
        self.forbidden_patterns = [
            r'<script.*?</script>',  # Script tags
            r'javascript:',           # JavaScript protocol
            r'on\w+\s*=',            # Event handlers
            r'DROP\s+TABLE',         # SQL injection
            r'UNION\s+SELECT',       # SQL injection
        ]

    def sanitize(self, user_input):
        """Sanitizar entrada del usuario"""
        if not isinstance(user_input, str):
            raise ValueError("Input must be string")

        # Verificar longitud
        if len(user_input) > self.max_length:
            raise ValueError(f"Input exceeds max length {self.max_length}")

        # Escapar HTML
        sanitized = escape(user_input)

        # Verificar patrones peligrosos
```

```
        for pattern in self.forbidden_patterns:
            if re.search(pattern, sanitized, re.IGNORECASE):
                raise ValueError(f"Dangerous pattern detected: {pattern}")

        # Usar bleach para limpieza adicional
        sanitized = bleach.clean(sanitized, tags=[], strip=True)

        return sanitized

    def validate_prompt(self, prompt):
        """Validar y sanitizar prompt"""
        sanitized = self.sanitize(prompt)

        # Validaciones adicionales
        if len(sanitized) < 1:
            raise ValueError("Prompt cannot be empty")

        if self._has_excessive_repetition(sanitized):
            raise ValueError("Excessive character repetition detected")

        return sanitized

    def _has_excessive_repetition(self, text, max_repeat=10):
        """Detectar repetición excesiva"""
        for char in set(text):
            if char * max_repeat in text:
                return True
        return False
```

Opción 2: Control de Ejecución

```
from RestrictedPython import compile_restricted_exec
import ast

class ExecutionController:
    """Controlador de ejecución segura"""

    def __init__(self):
        self.allowed_modules = ['math', 'json', 'datetime']
        self.execution_timeout = 30
        self.memory_limit = 512  # MB

    def execute_code_safely(self, code):
        """Ejecutar código de forma segura"""
        # Compilar con restricciones
        compiled = compile_restricted_exec(code)

        if compiled.errors:
            return {"error": "Código no permitido", "details": compiled.errors}
```

```
        # Crear entorno restringido
        safe_globals = self._create_safe_globals()
        safe_locals = {}

        try:
            # Ejecutar con timeout
            exec_with_timeout(
                compiled.code,
                safe_globals,
                safe_locals,
                timeout=self.execution_timeout
            )

            return {
                "status": "success",
                "output": safe_locals.get("result")
            }

        except TimeoutError:
            return {"error": "Ejecución excedió tiempo límite"}
        except Exception as e:
            return {"error": f"Error en ejecución: {str(e)}"}

    def _create_safe_globals(self):
        """Crear globals seguros"""
        safe_globals = {
            '__builtins__': {
                'abs': abs,
                'len': len,
                'sum': sum,
                'print': print,
                'range': range,
            },
            '__name__': 'restricted_module'
        }

        return safe_globals

    def validate_code(self, code):
        """Validar código antes de ejecutar"""
        try:
            tree = ast.parse(code)

            # Verificar nodos peligrosos
            dangerous_nodes = [
                ast.Import,
                ast.ImportFrom,
                ast.Open,
                ast.Eval,
                ast.Exec
            ]
```

```
            for node in ast.walk(tree):
                if type(node) in dangerous_nodes:
                    return False

            return True
        except:
            return False
```

Opción 3: Gestión de Secretos y Auditoría

```
import os
from hashlib import sha256
from cryptography.fernet import Fernet
import logging

class SecretManager:
    """Gestor de secretos y credenciales"""

    def __init__(self):
        self.cipher = self._init_cipher()
        self.audit_log = logging.getLogger("security_audit")
        self.secrets = {}

    def store_secret(self, key, value, secret_type="api_key"):
        """Almacenar secreto encriptado"""
        encrypted = self.cipher.encrypt(value.encode())

        self.secrets[key] = {
            "value": encrypted,
            "type": secret_type,
            "created_at": datetime.now(),
            "last_accessed": None,
            "access_count": 0
        }

        self._audit_log(f"SECRET_STORED", {"key": key, "type": secret_type})

    def retrieve_secret(self, key, requester_id):
        """Recuperar secreto con auditoría"""
        if key not in self.secrets:
            self._audit_log("SECRET_NOT_FOUND", {"key": key, "requester":
requester_id})
            raise ValueError(f"Secret '{key}' not found")

        secret_data = self.secrets[key]

        # Actualizar auditoría
        secret_data["last_accessed"] = datetime.now()
        secret_data["access_count"] += 1
```

```
        self._audit_log("SECRET_ACCESSED", {
            "key": key,
            "requester": requester_id,
            "access_count": secret_data["access_count"]
        })

        # Desencriptar y retornar
        decrypted = self.cipher.decrypt(secret_data["value"]).decode()
        return decrypted

    def rotate_secret(self, key):
        """Rotar secreto"""
        if key not in self.secrets:
            raise ValueError(f"Secret '{key}' not found")

        # Crear nuevo secret
        new_secret_value = os.urandom(32)

        # Actualizar
        self.secrets[key]["value"] = self.cipher.encrypt(new_secret_value)
        self.secrets[key]["rotated_at"] = datetime.now()

        self._audit_log("SECRET_ROTATED", {"key": key})

    def _init_cipher(self):
        """Inicializar cipher"""
        key = os.getenv("ENCRYPTION_KEY")
        if not key:
            raise ValueError("ENCRYPTION_KEY environment variable not set")
        return Fernet(key.encode())

    def _audit_log(self, action, details):
        """Registrar en auditoría"""
        log_entry = {
            "timestamp": datetime.now().isoformat(),
            "action": action,
            "details": details
        }
        self.audit_log.info(json.dumps(log_entry))
```

Características Clave:

- □ Sanitización de inputs
- □ Ejecución controlada
- □ Encriptación de secretos
- □ Auditoría completa
- □ Detección de anomalías

7.3. 9.3. Monitorización

Concepto: Monitoreo en tiempo real de agentes y sistemas.

Opción 1: Logging Estructurado en Producción

```
import logging
import json
from pythonjsonlogger import jsonlogger

class ProductionLogger:
    """Logger estructurado para producción"""

    def __init__(self, agent_name):
        self.agent_name = agent_name
        self.logger = logging.getLogger(agent_name)

        # Handler JSON
        logHandler = logging.FileHandler(f"{agent_name}.json")
        formatter = jsonlogger.JsonFormatter()
        logHandler.setFormatter(formatter)
        self.logger.addHandler(logHandler)

        self.logger.setLevel(logging.INFO)

    def log_request(self, request_id, prompt, agent_id):
        """Registrar solicitud"""
        self.logger.info(
            "REQUEST",
            extra={
                "request_id": request_id,
                "agent_id": agent_id,
                "prompt_length": len(prompt),
                "timestamp": datetime.now().isoformat()
            }
        )

    def log_response(self, request_id, response, latency_ms):
        """Registrar respuesta"""
        self.logger.info(
            "RESPONSE",
            extra={
                "request_id": request_id,
                "response_length": len(response),
                "latency_ms": latency_ms,
                "timestamp": datetime.now().isoformat()
            }
        )
```

```
    def log_error(self, request_id, error, error_type):
        """Registrar error"""
        self.logger.error(
            "ERROR",
            extra={
                "request_id": request_id,
                "error": str(error),
                "error_type": error_type,
                "timestamp": datetime.now().isoformat()
            }
        )
```

Opción 2: Métricas y Alertas

```
from prometheus_client import Counter, Histogram, Gauge

class MetricsCollector:
    """Colector de métricas con Prometheus"""

    def __init__(self):
        # Contadores
        self.requests_total = Counter(
            'agent_requests_total',
            'Total de solicitudes',
            ['agent_id', 'status']
        )

        self.tokens_used = Counter(
            'agent_tokens_used_total',
            'Total de tokens usados',
            ['agent_id']
        )

        # Histogramas
        self.response_time = Histogram(
            'agent_response_time_seconds',
            'Tiempo de respuesta',
            ['agent_id'],
            buckets=(0.1, 0.5, 1.0, 2.0, 5.0)
        )

        # Gauges
        self.active_conversations = Gauge(
            'agent_active_conversations',
            'Conversaciones activas',
            ['agent_id']
        )

    def record_request(self, agent_id, status, duration_seconds, tokens):
        """Registrar métrica de solicitud"""
```

```
        self.requests_total.labels(
            agent_id=agent_id,
            status=status
        ).inc()

        self.response_time.labels(
            agent_id=agent_id
        ).observe(duration_seconds)

        if tokens:
            self.tokens_used.labels(
                agent_id=agent_id
            ).inc(tokens)

class AlertManager:
    """Gestor de alertas"""

    def __init__(self, thresholds):
        self.thresholds = thresholds
        self.alerts = []

    def check_metrics(self, metrics):
        """Verificar métricas y generar alertas"""
        if metrics['response_time_ms'] > self.thresholds['max_latency']:
            self.raise_alert("HIGH_LATENCY", metrics)

        if metrics['error_rate'] > self.thresholds['max_error_rate']:
            self.raise_alert("HIGH_ERROR_RATE", metrics)

        if metrics['tokens_used'] > self.thresholds['max_tokens']:
            self.raise_alert("HIGH_TOKEN_USAGE", metrics)

    def raise_alert(self, alert_type, details):
        """Generar alerta"""
        alert = {
            "type": alert_type,
            "details": details,
            "timestamp": datetime.now(),
            "severity": "warning"
        }

        self.alerts.append(alert)
        self._send_notification(alert)

    def _send_notification(self, alert):
        """Enviar notificación (Slack, PagerDuty, etc)"""
        # Implementación
        pass
```

Opción 3: Dashboard de Monitoreo

```
from streamlit import streamlit as st
import pandas as pd

class MonitoringDashboard:
    """Dashboard de monitoreo con Streamlit"""

    def __init__(self, metrics_source):
        self.metrics = metrics_source

    def render(self):
        """Renderizar dashboard"""
        st.title("AutoGen Monitoring Dashboard")

        # Métricas principales
        col1, col2, col3, col4 = st.columns(4)

        with col1:
            st.metric("Total Requests", self.metrics.total_requests)

        with col2:
            st.metric("Success Rate", f"{self.metrics.success_rate}%")

        with col3:
            st.metric("Avg Latency", f"{self.metrics.avg_latency_ms}ms")

        with col4:
            st.metric("Active Agents", self.metrics.active_agents_count)

        # Gráficos
        st.subheader("Request Timeline")
        df = pd.DataFrame(self.metrics.requests_timeline)
        st.line_chart(df)

        st.subheader("Error Rate by Agent")
        error_data = pd.DataFrame(self.metrics.error_rates_by_agent)
        st.bar_chart(error_data)

        st.subheader("Token Usage")
        token_data = pd.DataFrame(self.metrics.token_usage_by_agent)
        st.area_chart(token_data)
```

Características Clave:

- □ Logging estructurado JSON
- □ Métricas con Prometheus
- □ Alertas automáticas
- □ Dashboard visual
- □ Análisis en tiempo real

7.4. 9.4. CI/CD

Concepto: Pipeline automático de integración y despliegue continuo.

Opción 1: Pipeline GitHub Actions

```
name: Deploy Agent

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
.
    - uses: actions/checkout@v2

    - name: Set up Python
      uses: actions/setup-python@v2
      with:
        python-version: '3.9'

    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install -r requirements.txt

    - name: Run tests
      run: |
        python -m pytest tests/ -v

    - name: Code coverage
      run: |
        pytest --cov=agents tests/
        codecov

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
.
    - uses: actions/checkout@v2

    - name: Build Docker image
      run: |
```

```
        docker build -t agent:latest .

    - name: Push to registry
      run: |
        docker tag agent:latest ${{ secrets.REGISTRY }}/agent:${{ github.sha }}
        docker push ${{ secrets.REGISTRY }}/agent:${{ github.sha }}

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
.
    - uses: actions/checkout@v2

    - name: Deploy to Kubernetes
      run: |
        kubectl set image deployment/agent \
          agent=${{ secrets.REGISTRY }}/agent:${{ github.sha }}
        kubectl rollout status deployment/agent
```

Opción 2: Sistema de Versionado de Agentes

```
from dataclasses import dataclass
import semver

@dataclass
class AgentVersion:
    major: int
    minor: int
    patch: int
    agent_id: str
    timestamp: str
    changes: list

class VersionManager:
    """Gestor de versiones de agentes"""

    def __init__(self):
        self.versions = {}

    def create_version(self, agent_id, changes, version_type="patch"):
        """Crear nueva versión"""
        latest = self._get_latest_version(agent_id)

        if version_type == "major":
            latest.major += 1
            latest.minor = 0
            latest.patch = 0
        elif version_type == "minor":
            latest.minor += 1
```

```
            latest.patch = 0
        else:  # patch
            latest.patch += 1

        new_version = AgentVersion(
            major=latest.major,
            minor=latest.minor,
            patch=latest.patch,
            agent_id=agent_id,
            timestamp=datetime.now().isoformat(),
            changes=changes
        )

        if agent_id not in self.versions:
            self.versions[agent_id] = []

        self.versions[agent_id].append(new_version)
        return new_version

    def get_version_history(self, agent_id):
        """Obtener historial de versiones"""
        return self.versions.get(agent_id, [])

    def rollback_to_version(self, agent_id, version_number):
        """Revertir a versión anterior"""
        versions = self.versions.get(agent_id, [])
        if version_number < len(versions):
            return versions[version_number]
        return None

    def _get_latest_version(self, agent_id):
        """Obtener versión más reciente"""
        versions = self.versions.get(agent_id, [])
        return versions[-1] if versions else AgentVersion(0, 0, 1, agent_id, "", [])
```

Opción 3: Testing y Rollback Automático

```
class AutomatedTestingPipeline:
    """Pipeline de testing automático"""

    def __init__(self):
        self.test_results = {}

    def run_test_suite(self, agent, test_cases):
        """Ejecutar suite de tests"""
        results = {
            "agent_id": agent.name,
            "tests": [],
            "passed": 0,
            "failed": 0,
```

```
            "timestamp": datetime.now()
        }

        for test in test_cases:
            try:
                result = test.execute(agent)
                results["tests"].append({
                    "name": test.name,
                    "status": "passed",
                    "duration": result.duration
                })
                results["passed"] += 1
            except Exception as e:
                results["tests"].append({
                    "name": test.name,
                    "status": "failed",
                    "error": str(e)
                })
                results["failed"] += 1

        return results

    def should_rollback(self, test_results):
        """Determinar si hacer rollback"""
        fail_rate = test_results["failed"] / (test_results["passed"] +
test_results["failed"])
        return fail_rate > 0.1  # Más del 10% de fallos

    def automated_rollback(self, agent_id, previous_version):
        """Rollback automático"""
        print(f"Rolling back {agent_id} to {previous_version}")

        # Ejecutar rollback
        self._execute_rollback(agent_id, previous_version)

        # Verificar salud
        if self._verify_health(agent_id):
            print(f"Rollback successful for {agent_id}")
            return True
        else:
            print(f"Rollback failed for {agent_id}")
            return False

    def _execute_rollback(self, agent_id, version):
        """Ejecutar rollback"""
        # Implementación específica
        pass

    def _verify_health(self, agent_id):
        """Verificar salud post-rollback"""
        # Implementación específica
```

```
        return True
```

Características Clave:

- 🔄 Pipelines automáticos
- 🧪 Testing continuo
- 📦 Versionado semántico
- 🔙 Rollback automático
- 📊 Monitoreo post-deploy

Chapter 8. Módulo 10: Integraciones

8.1. 10.1. Frameworks y Librerías

Concepto: Integración con frameworks y librerías populares en el ecosistema de IA.

Opción 1: Integración con LangChain

```
from langchain.agents import initialize_agent, Tool
from langchain.llms import OpenAI
from langchain.memory import ConversationBufferMemory

class LangChainIntegration:
    """Integración con LangChain"""

    def __init__(self):
        self.llm = OpenAI(temperature=0.7)
        self.memory = ConversationBufferMemory()
        self.tools = self._setup_tools()

    def _setup_tools(self):
        """Configurar herramientas de LangChain"""
        tools = [
            Tool(
                name="Calculator",
                func=self._calculate,
                description="Útil para matemáticas"
            ),
            Tool(
                name="Search",
                func=self._search,
                description="Buscar información"
            )
        ]
        return tools

    def create_agent(self):
        """Crear agente con LangChain"""
        agent = initialize_agent(
            self.tools,
            self.llm,
            agent="zero-shot-react-description",
            memory=self.memory,
            verbose=True
        )
        return agent

    def _calculate(self, expression):
        """Función de cálculo"""
```

```
        try:
            return str(eval(expression))
        except:
            return "Error in calculation"

    def _search(self, query):
        """Función de búsqueda"""
        # Implementación con API de búsqueda
        return f"Results for: {query}"
```

Opción 2: Integración con LlamaIndex

```
from llama_index import GPTVectorStoreIndex, Document
from llama_index.llms import OpenAI

class LlamaIndexIntegration:
    """Integración con LlamaIndex"""

    def __init__(self):
        self.llm = OpenAI(temperature=0.7)
        self.indices = {}

    def create_index_from_documents(self, documents, index_name):
        """Crear índice a partir de documentos"""
        doc_objects = [
            Document(text=doc["content"], metadata=doc.get("metadata", {}))
            for doc in documents
        ]

        index = GPTVectorStoreIndex.from_documents(doc_objects)
        self.indices[index_name] = index
        return index

    def query_index(self, index_name, query):
        """Consultar índice"""
        if index_name not in self.indices:
            raise ValueError(f"Index {index_name} not found")

        index = self.indices[index_name]
        response = index.as_query_engine().query(query)
        return response

    def hybrid_search(self, index_name, query, top_k=5):
        """Búsqueda híbrida"""
        index = self.indices[index_name]

        # Vector search
        vector_results = index.as_query_engine().query(query)

        # BM25 search
```

```
        bm25_results = self._bm25_search(index, query, top_k)

        return {
            "vector_results": vector_results,
            "bm25_results": bm25_results
        }

    def _bm25_search(self, index, query, top_k):
        """Implementar BM25"""
        # Implementación simplificada
        return []
```

Opción 3: Arquitectura Multi-Framework

```
class MultiFrameworkAgent:
    """Agente que integra múltiples frameworks"""

    def __init__(self):
        self.langchain_agent = None
        self.llamaindex_client = None
        self.autogen_agent = None

    def setup(self, config):
        """Configurar integraciones"""
        # LangChain setup
        self.langchain_agent = LangChainIntegration()

        # LlamaIndex setup
        self.llamaindex_client = LlamaIndexIntegration()

        # AutoGen setup
        self.autogen_agent = AssistantAgent("Agent", config)

    def query(self, prompt, strategy="auto"):
        """Realizar query con framework óptimo"""
        if strategy == "auto":
            strategy = self._select_best_framework(prompt)

        if strategy == "langchain":
            return self.langchain_agent.create_agent().run(prompt)
        elif strategy == "llamaindex":
            return self.llamaindex_client.query_index("default", prompt)
        elif strategy == "autogen":
            return self.autogen_agent.generate_response(prompt)

    def _select_best_framework(self, prompt):
        """Seleccionar mejor framework automáticamente"""
        if "search" in prompt.lower():
            return "langchain"
        elif "document" in prompt.lower():
```

```
            return "llamaindex"
        else:
            return "autogen"
```

Características Clave:

- □ Integración con LangChain
- □ Integración con LlamaIndex
- □ Arquitectura multi-framework
- □ Selección automática de herramientas
- □ Composición flexible

8.2. 10.2. APIs y Servicios

Concepto: Integración con proveedores de LLM y servicios de IA.

Opción 1: Adaptadores de Múltiples LLMs

```
from abc import ABC, abstractmethod
import openai
import anthropic

class LLMAdapter(ABC):
    """Adaptador base para LLMs"""

    @abstractmethod
    def generate(self, prompt, **kwargs):
        pass

class OpenAIAdapter(LLMAdapter):
    """Adaptador para OpenAI"""

    def __init__(self, api_key, model="gpt-4"):
        self.api_key = api_key
        self.model = model
        openai.api_key = api_key

    def generate(self, prompt, temperature=0.7, max_tokens=2000):
        """Generar con OpenAI"""
        response = openai.ChatCompletion.create(
            model=self.model,
            messages=[{"role": "user", "content": prompt}],
            temperature=temperature,
            max_tokens=max_tokens
        )
        return response.choices[0].message.content
```

```
class AnthropicAdapter(LLMAdapter):
    """Adaptador para Anthropic Claude"""

    def __init__(self, api_key, model="claude-3-opus"):
        self.api_key = api_key
        self.model = model
        self.client = anthropic.Anthropic(api_key=api_key)

    def generate(self, prompt, temperature=0.7, max_tokens=2000):
        """Generar con Claude"""
        message = self.client.messages.create(
            model=self.model,
            max_tokens=max_tokens,
            temperature=temperature,
            messages=[{"role": "user", "content": prompt}]
        )
        return message.content[0].text

class UnifiedLLMClient:
    """Cliente unificado para múltiples LLMs"""

    def __init__(self):
        self.adapters = {}
        self.primary_adapter = None

    def register_adapter(self, name, adapter):
        """Registrar adaptador"""
        self.adapters[name] = adapter
        if self.primary_adapter is None:
            self.primary_adapter = name

    def generate(self, prompt, adapter_name=None, **kwargs):
        """Generar con adaptador especificado o primario"""
        if adapter_name is None:
            adapter_name = self.primary_adapter

        if adapter_name not in self.adapters:
            raise ValueError(f"Adapter {adapter_name} not found")

        return self.adapters[adapter_name].generate(prompt, **kwargs)

    def generate_with_fallback(self, prompt, **kwargs):
        """Generar con fallback automático"""
        for adapter_name, adapter in self.adapters.items():
            try:
                return adapter.generate(prompt, **kwargs)
            except Exception as e:
                print(f"Error con {adapter_name}: {e}")
                continue
```

```
        raise Exception("All adapters failed")
```

Opción 2: Gestión de Cuotas y Costos

```
from datetime import datetime, timedelta

class APIQuotaManager:
    """Gestor de cuotas de API"""

    def __init__(self):
        self.quotas = {}
        self.usage = {}
        self.costs = {}

    def set_quota(self, service, limit, period="day"):
        """Establecer cuota para servicio"""
        self.quotas[service] = {
            "limit": limit,
            "period": period,
            "reset_at": datetime.now() + timedelta(days=1)
        }
        self.usage[service] = 0

    def record_usage(self, service, tokens=1, cost=0.0):
        """Registrar uso"""
        if service not in self.usage:
            self.usage[service] = 0

        self.usage[service] += tokens

        if service not in self.costs:
            self.costs[service] = 0.0

        self.costs[service] += cost

    def check_quota(self, service):
        """Verificar si hay cuota disponible"""
        if service not in self.quotas:
            return True

        quota = self.quotas[service]

        # Resetear si pasó el período
        if datetime.now() > quota["reset_at"]:
            self.usage[service] = 0
            quota["reset_at"] = datetime.now() + timedelta(days=1)

        return self.usage[service] < quota["limit"]

    def get_usage_report(self):
```

```
        """Obtener reporte de uso"""
        report = {}
        for service in self.usage:
            quota = self.quotas.get(service, {})
            report[service] = {
                "used": self.usage[service],
                "limit": quota.get("limit"),
                "cost": self.costs.get(service, 0.0),
                "percentage": (self.usage[service] / quota.get("limit", 1)) * 100 if
"limit" in quota else 0
            }
        return report
```

Opción 3: Servicio de Recomendación de LLM

```
class LLMRecommender:
    """Sistema de recomendación de LLM"""

    def __init__(self):
        self.models_db = self._init_models()

    def recommend(self, task_type, constraints=None):
        """Recomendar modelo para tarea"""
        if constraints is None:
            constraints = {}

        candidates = self._filter_models(task_type, constraints)
        scored = self._score_models(candidates, constraints)

        return sorted(scored, key=lambda x: x["score"], reverse=True)

    def _filter_models(self, task_type, constraints):
        """Filtrar modelos por tipo de tarea"""
        suitable = []
        for model_name, model_info in self.models_db.items():
            if task_type in model_info["capabilities"]:
                # Verificar restricciones
                if self._satisfies_constraints(model_info, constraints):
                    suitable.append((model_name, model_info))

        return suitable

    def _score_models(self, candidates, constraints):
        """Puntuar modelos"""
        scores = []
        for model_name, model_info in candidates:
            score = 100

            # Ajustar por costo
            if "max_cost" in constraints:
```

```
                if model_info.get("cost") > constraints["max_cost"]:
                    score -= 50

            # Ajustar por latencia
            if "max_latency" in constraints:
                if model_info.get("latency") > constraints["max_latency"]:
                    score -= 30

            # Ajustar por capacidad
            score += model_info.get("capability_score", 0)

            scores.append({
                "model": model_name,
                "score": score,
                "info": model_info
            })

        return scores

    def _satisfies_constraints(self, model_info, constraints):
        """Verificar si modelo satisface restricciones"""
        for constraint, value in constraints.items():
            if constraint == "min_context_length":
                if model_info.get("context_length", 0) < value:
                    return False
            elif constraint == "language":
                if value not in model_info.get("languages", []):
                    return False

        return True

    def _init_models(self):
        """Inicializar base de datos de modelos"""
        return {
            "gpt-4": {
                "capabilities": ["text", "vision", "code"],
                "cost": 0.03,
                "latency": 1000,
                "context_length": 8000,
                "capability_score": 95
            },
            "claude-3-opus": {
                "capabilities": ["text", "vision"],
                "cost": 0.025,
                "latency": 800,
                "context_length": 200000,
                "capability_score": 90
            },
            "mistral": {
                "capabilities": ["text"],
                "cost": 0.0,
```

```
                "latency": 500,
                "context_length": 32000,
                "capability_score": 75
            }
        }
```

Características Clave:

- □ Múltiples adaptadores de LLM
- □ Gestión de cuotas
- □ Fallback automático
- □ Recomendación de modelo
- □ Análisis de costos

8.3. 10.3. Bases de Datos

Concepto: Integración con bases de datos para almacenamiento y búsqueda.

Opción 1: Almacenamiento Vectorial

```
import numpy as np
from sentence_transformers import SentenceTransformer

class VectorStoreAdapter:
    """Adaptador para bases de datos vectoriales"""

    def __init__(self, provider="chroma", collection_name="default"):
        self.provider = provider
        self.collection_name = collection_name
        self.embedder = SentenceTransformer('all-MiniLM-L6-v2')

        if provider == "chroma":
            import chromadb
            self.client = chromadb.Client()
            self.collection = self.client.get_or_create_collection(collection_name)
        elif provider == "pinecone":
            import pinecone
            self.index = pinecone.Index("autogen-index")

    def add_documents(self, documents):
        """Agregar documentos"""
        embeddings = self.embedder.encode([d["text"] for d in documents])

        if self.provider == "chroma":
            self.collection.upsert(
                ids=[d["id"] for d in documents],
```

```
                embeddings=embeddings.tolist(),
                documents=[d["text"] for d in documents],
                metadatas=[d.get("metadata", {}) for d in documents]
            )
        elif self.provider == "pinecone":
            vectors = [
                (documents[i]["id"], embeddings[i], documents[i].get("metadata", {}))
                for i in range(len(documents))
            ]
            self.index.upsert(vectors=vectors)

    def search(self, query, top_k=5):
        """Búsqueda semántica"""
        query_embedding = self.embedder.encode(query)

        if self.provider == "chroma":
            results = self.collection.query(
                query_embeddings=query_embedding.tolist(),
                n_results=top_k
            )
            return results

        elif self.provider == "pinecone":
            results = self.index.query(
                vector=query_embedding.tolist(),
                top_k=top_k,
                include_metadata=True
            )
            return results
```

Opción 2: Almacenamiento Relacional

```
from sqlalchemy import create_engine, Column, String, Integer, Float
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

Base = declarative_base()

class AgentInteraction(Base):
    """Modelo de interacción"""
    __tablename__ = "agent_interactions"

    id = Column(Integer, primary_key=True)
    agent_id = Column(String)
    prompt = Column(String)
    response = Column(String)
    timestamp = Column(String)
    tokens_used = Column(Integer)
    latency_ms = Column(Float)
```

```
class RelationalDataStore:
    """Almacén de datos relacional"""

    def __init__(self, db_url="sqlite:///autogen.db"):
        self.engine = create_engine(db_url)
        Base.metadata.create_all(self.engine)
        self.Session = sessionmaker(bind=self.engine)

    def save_interaction(self, agent_id, prompt, response, tokens, latency):
        """Guardar interacción"""
        session = self.Session()
        try:
            interaction = AgentInteraction(
                agent_id=agent_id,
                prompt=prompt,
                response=response,
                timestamp=datetime.now().isoformat(),
                tokens_used=tokens,
                latency_ms=latency
            )
            session.add(interaction)
            session.commit()
        finally:
            session.close()

    def query_interactions(self, agent_id, limit=100):
        """Consultar interacciones"""
        session = self.Session()
        try:
            query = session.query(AgentInteraction).filter(
                AgentInteraction.agent_id == agent_id
            ).order_by(AgentInteraction.timestamp.desc()).limit(limit)
            return query.all()
        finally:
            session.close()
```

Opción 3: Data Lake y Analytics

```
import pandas as pd

class DataLakeManager:
    """Gestor de Data Lake"""

    def __init__(self, data_dir="/data/autogen"):
        self.data_dir = data_dir

    def export_interactions(self, agent_id, format="parquet"):
        """Exportar interacciones a Data Lake"""
        interactions = self._get_interactions(agent_id)
```

```
        df = pd.DataFrame([
            {
                "agent_id": i.agent_id,
                "prompt": i.prompt,
                "response": i.response,
                "tokens": i.tokens_used,
                "latency": i.latency_ms
            }
            for i in interactions
        ])

        if format == "parquet":
            df.to_parquet(f"{self.data_dir}/{agent_id}/data.parquet")
        elif format == "csv":
            df.to_csv(f"{self.data_dir}/{agent_id}/data.csv")

        return df

    def analyze_performance(self, agent_id):
        """Analizar rendimiento"""
        df = self.export_interactions(agent_id)

        analysis = {
            "total_interactions": len(df),
            "avg_tokens": df["tokens"].mean(),
            "avg_latency": df["latency"].mean(),
            "p95_latency": df["latency"].quantile(0.95),
            "p99_latency": df["latency"].quantile(0.99)
        }

        return analysis
```

Características Clave:

- □ Almacenamiento vectorial escalable
- □ Datos relacionales con SQL
- □ Data Lake para análisis
- □ Búsqueda semántica
- □ Analytics y reportes

Chapter 9. Módulo 11: Proyecto Final

9.1. 11.1. Definición del Proyecto

Concepto: Metodología para definir, planificar e implementar un proyecto completo con AutoGen.

Opción 1: Análisis de Requisitos

```
class ProjectRequirementsAnalyzer:
    """Analizador de requisitos de proyecto"""

    def __init__(self):
        self.requirements = {}
        self.constraints = {}
        self.success_criteria = {}

    def analyze_use_case(self, use_case_description):
        """Analizar caso de uso"""
        analysis = {
            "agents_needed": self._identify_agents(use_case_description),
            "data_requirements": self._identify_data(use_case_description),
            "integration_needs": self._identify_integrations(use_case_description),
            "performance_targets":
self._set_performance_targets(use_case_description),
            "timeline": self._estimate_timeline(use_case_description)
        }

        return analysis

    def create_architecture_plan(self, analysis):
        """Crear plan de arquitectura"""
        plan = {
            "agent_roles": [
                {"name": agent, "responsibility": self._get_responsibility(agent)}
                for agent in analysis["agents_needed"]
            ],
            "data_flow": self._create_data_flow(analysis),
            "integration_points": analysis["integration_needs"],
            "deployment_strategy": self._select_deployment_strategy(analysis)
        }

        return plan

    def _identify_agents(self, description):
        """Identificar agentes necesarios"""
        # Implementación simplificada
        return ["coordinator", "data_processor", "response_formatter"]

    def _identify_data(self, description):
```

```
        """Identificar requisitos de datos"""
        return {"input": "text", "output": "json"}

    def _identify_integrations(self, description):
        """Identificar integraciones necesarias"""
        return ["database", "api", "logging"]

    def _set_performance_targets(self, description):
        """Establecer objetivos de rendimiento"""
        return {
            "max_latency_ms": 5000,
            "max_cost_per_request": 0.50,
            "success_rate": 0.95
        }

    def _estimate_timeline(self, description):
        """Estimar cronograma"""
        return {
            "phase_1_analysis": "1 week",
            "phase_2_development": "2 weeks",
            "phase_3_testing": "1 week",
            "phase_4_deployment": "1 week"
        }

    def _get_responsibility(self, agent):
        """Obtener responsabilidad de agente"""
        responsibilities = {
            "coordinator": "Orquestar flujo de trabajo",
            "data_processor": "Procesar datos de entrada",
            "response_formatter": "Formatear respuesta final"
        }
        return responsibilities.get(agent, "General task")

    def _create_data_flow(self, analysis):
        """Crear diagrama de flujo de datos"""
        return {
            "input": analysis["data_requirements"]["input"],
            "processing": "Multi-step transformation",
            "output": analysis["data_requirements"]["output"]
        }

    def _select_deployment_strategy(self, analysis):
        """Seleccionar estrategia de despliegue"""
        if "high" in str(analysis):
            return "kubernetes"
        return "docker"
```

Opción 2: Plan de Implementación

```
from datetime import datetime, timedelta
```

```
class ImplementationPlanner:
    """Planificador de implementación"""

    def __init__(self, project_name):
        self.project_name = project_name
        self.milestones = []
        self.tasks = []
        self.dependencies = {}

    def create_implementation_plan(self, architecture):
        """Crear plan detallado"""
        plan = {
            "milestone_1_agent_development": {
                "duration_days": 7,
                "tasks": [
                    "Design agent interfaces",
                    "Implement agent logic",
                    "Add logging and monitoring"
                ]
            },
            "milestone_2_integration": {
                "duration_days": 5,
                "tasks": [
                    "Integrate with databases",
                    "Connect to external APIs",
                    "Setup event handlers"
                ]
            },
            "milestone_3_testing": {
                "duration_days": 5,
                "tasks": [
                    "Unit testing",
                    "Integration testing",
                    "Load testing"
                ]
            },
            "milestone_4_deployment": {
                "duration_days": 3,
                "tasks": [
                    "Pre-production validation",
                    "Deployment",
                    "Post-deployment verification"
                ]
            }
        }

        return plan

    def track_progress(self):
        """Rastrear progreso"""
```

```
        progress = {
            "completed": len([t for t in self.tasks if t.get("status") == "done"]),
            "in_progress": len([t for t in self.tasks if t.get("status") ==
"in_progress"]),
            "pending": len([t for t in self.tasks if t.get("status") == "pending"]),
            "total": len(self.tasks),
            "percentage": (len([t for t in self.tasks if t.get("status") == "done"]) /
len(self.tasks)) * 100 if self.tasks else 0
        }

        return progress
```

Opción 3: Especificaciones Técnicas

```
class TechnicalSpecification:
    """Generador de especificaciones técnicas"""

    def __init__(self, project_architecture):
        self.architecture = project_architecture

    def generate_api_spec(self):
        """Generar especificación de API"""
        return {
            "endpoints": [
                {
                    "path": "/query",
                    "method": "POST",
                    "request": {"prompt": "string"},
                    "response": {"result": "string", "latency": "integer"}
                },
                {
                    "path": "/health",
                    "method": "GET",
                    "response": {"status": "string"}
                }
            ],
            "authentication": "API_KEY",
            "rate_limit": "100 req/minute"
        }

    def generate_db_spec(self):
        """Generar especificación de base de datos"""
        return {
            "tables": [
                {
                    "name": "agent_interactions",
                    "columns": [
                        {"name": "id", "type": "integer", "primary_key": True},
                        {"name": "agent_id", "type": "string"},
                        {"name": "prompt", "type": "text"},
```

```
                        {"name": "response", "type": "text"},
                        {"name": "timestamp", "type": "datetime"}
                    ]
                }
            ],
            "indexes": [
                {"table": "agent_interactions", "columns": ["agent_id", "timestamp"]}
            ]
        }

    def generate_security_spec(self):
        """Generar especificación de seguridad"""
        return {
            "authentication": "OAuth2",
            "encryption": "TLS 1.3",
            "input_validation": "Strict",
            "rate_limiting": "Enabled",
            "audit_logging": "All requests",
            "secret_management": "Vault"
        }
```

9.2. 11.2. Desarrollo

Concepto: Proceso iterativo de desarrollo, testing y refinamiento.

Opción 1: Desarrollo Iterativo

```
class IterativeDevelopmentCycle:
    """Ciclo iterativo de desarrollo"""

    def __init__(self, sprint_length_days=7):
        self.sprint_length = sprint_length_days
        self.sprints = []
        self.current_sprint = None

    def start_sprint(self, sprint_number, objectives):
        """Iniciar sprint"""
        sprint = {
            "number": sprint_number,
            "objectives": objectives,
            "start_date": datetime.now(),
            "end_date": datetime.now() + timedelta(days=self.sprint_length),
            "tasks": [],
            "status": "in_progress"
        }

        self.current_sprint = sprint
        self.sprints.append(sprint)
```

```
        return sprint

    def add_task(self, task_name, story_points, assignee):
        """Agregar tarea al sprint"""
        task = {
            "name": task_name,
            "story_points": story_points,
            "assignee": assignee,
            "status": "todo",
            "created_at": datetime.now()
        }

        if self.current_sprint:
            self.current_sprint["tasks"].append(task)
        return task

    def complete_sprint(self):
        """Completar sprint"""
        if self.current_sprint:
            self.current_sprint["status"] = "completed"
            self.current_sprint["end_date"] = datetime.now()

    def get_velocity(self):
        """Obtener velocidad del equipo"""
        completed_sprints = [s for s in self.sprints if s["status"] == "completed"]
        if not completed_sprints:
            return 0

        total_points = sum(
            sum(t.get("story_points", 0) for t in s.get("tasks", []))
            for s in completed_sprints
        )

        return total_points / len(completed_sprints)
```

Opción 2: Testing Integrado

```
class IntegratedTestingSuite:
    """Suite de testing integrada"""

    def __init__(self, project):
        self.project = project
        self.test_results = []

    def run_unit_tests(self, agent):
        """Ejecutar unit tests"""
        tests = {
            "test_agent_initialization": self._test_init(agent),
            "test_generate_response": self._test_generate(agent),
            "test_error_handling": self._test_errors(agent)
```

```
        }

        return {"unit_tests": tests}

    def run_integration_tests(self):
        """Ejecutar integration tests"""
        tests = {
            "test_agent_communication": self._test_communication(),
            "test_database_integration": self._test_database(),
            "test_api_endpoints": self._test_api()
        }

        return {"integration_tests": tests}

    def run_performance_tests(self):
        """Ejecutar performance tests"""
        metrics = {
            "latency": self._measure_latency(),
            "throughput": self._measure_throughput(),
            "memory_usage": self._measure_memory()
        }

        return {"performance_metrics": metrics}

    def _test_init(self, agent):
        return {"passed": agent is not None}

    def _test_generate(self, agent):
        response = agent.generate_response("Test")
        return {"passed": len(response) > 0}

    def _test_errors(self, agent):
        return {"passed": True}

    def _test_communication(self):
        return {"passed": True}

    def _test_database(self):
        return {"passed": True}

    def _test_api(self):
        return {"passed": True}

    def _measure_latency(self):
        return {"avg_ms": 150, "p95_ms": 300}

    def _measure_throughput(self):
        return {"requests_per_second": 100}

    def _measure_memory(self):
```

```
        return {"peak_mb": 256}
```

Opción 3: Refinamiento Continuo

```
class ContinuousRefinement:
    """Sistema de refinamiento continuo"""

    def __init__(self, agent):
        self.agent = agent
        self.improvements = []
        self.feedback_log = []

    def collect_feedback(self, feedback_item):
        """Recopilar feedback"""
        self.feedback_log.append({
            "timestamp": datetime.now(),
            "feedback": feedback_item,
            "addressed": False
        })

    def analyze_feedback(self):
        """Analizar feedback recibido"""
        patterns = {}
        for feedback in self.feedback_log:
            if not feedback["addressed"]:
                category = self._categorize_feedback(feedback["feedback"])
                patterns[category] = patterns.get(category, 0) + 1

        return patterns

    def implement_improvement(self, improvement_description):
        """Implementar mejora"""
        improvement = {
            "description": improvement_description,
            "implemented_at": datetime.now(),
            "impact": "pending"
        }

        self.improvements.append(improvement)
        return improvement

    def measure_improvement_impact(self, improvement):
        """Medir impacto de mejora"""
        # Comparar métricas antes y después
        impact = {
            "latency_reduction": 10,
            "accuracy_improvement": 5,
            "cost_reduction": 2
        }
```

```
        improvement["impact"] = impact
        return impact

    def _categorize_feedback(self, feedback):
        """Categorizar feedback"""
        if "slow" in feedback.lower():
            return "performance"
        elif "error" in feedback.lower():
            return "reliability"
        else:
            return "feature"
```

9.3. 11.3. Presentación

Concepto: Documentación, demostración y análisis de resultados del proyecto.

Opción 1: Documentación Técnica

```
class TechnicalDocumentationGenerator:
    """Generador de documentación técnica"""

    def __init__(self, project):
        self.project = project

    def generate_architecture_doc(self):
        """Generar documento de arquitectura"""
        doc = f"""
# Arquitectura del Proyecto {self.project.name}

## Componentes Principales
- **Agentes**: Componentes autónomos que toman decisiones
- **LLM**: Motor de razonamiento basado en modelos de lenguaje
- **Almacenamiento**: Persistencia de datos y estado
- **APIs**: Interfaz de comunicación externa

## Flujo de Datos
[Diagrama ASCII de flujo]

## Decisiones Técnicas
- Framework: AutoGen
- LLM: Ollama local (Mistral)
- Base de datos: PostgreSQL + ChromaDB
        """
        return doc

    def generate_agent_doc(self, agent):
        """Generar documentación de agente"""
        doc = f"""
```

```
# Agente: {agent.name}

## Responsabilidades
{agent.responsibilities}

## Interfaces
.
- Input: {agent.input_format}
- Output: {agent.output_format}

## Métodos Principales
{self._list_methods(agent)}
        """
        return doc

    def _list_methods(self, agent):
        methods = [m for m in dir(agent) if not m.startswith('_')]
        return "\n".join([f"- {m}" for m in methods[:5]])
```

Opción 2: Demostración del Sistema

```
class SystemDemonstration:
    """Demostración del sistema"""

    def __init__(self, project):
        self.project = project
        self.demo_scenarios = []

    def create_demo_scenario(self, name, description, steps):
        """Crear escenario de demostración"""
        scenario = {
            "name": name,
            "description": description,
            "steps": steps,
            "results": []
        }

        self.demo_scenarios.append(scenario)
        return scenario

    def execute_demo(self, scenario_name):
        """Ejecutar demostración"""
        scenario = next(s for s in self.demo_scenarios if s["name"] == scenario_name)

        results = []
        for step in scenario["steps"]:
            result = self._execute_step(step)
            results.append(result)
            scenario["results"].append(result)
```

```
        return results

    def _execute_step(self, step):
        """Ejecutar paso de demostración"""
        return {
            "step": step,
            "output": f"Output for: {step}",
            "duration_ms": 100
        }
```

Opción 3: Análisis y Reportes

```
class ProjectAnalysisReporter:
    """Reportero de análisis del proyecto"""

    def __init__(self, project):
        self.project = project

    def generate_final_report(self):
        """Generar reporte final"""
        report = {
            "project_name": self.project.name,
            "timeline": self._analyze_timeline(),
            "budget": self._analyze_budget(),
            "performance": self._analyze_performance(),
            "lessons_learned": self._extract_lessons(),
            "recommendations": self._generate_recommendations()
        }

        return report

    def _analyze_timeline(self):
        return {
            "planned_duration": "4 weeks",
            "actual_duration": "4.2 weeks",
            "variance": "+2%"
        }

    def _analyze_budget(self):
        return {
            "planned_cost": "$10,000",
            "actual_cost": "$9,800",
            "variance": "-2%"
        }

    def _analyze_performance(self):
        return {
            "success_rate": "98%",
            "avg_latency": "150ms",
            "throughput": "100 req/s"
```

```
        }

    def _extract_lessons(self):
        return [
            "AutoGen provides excellent abstraction for multi-agent systems",
            "Proper monitoring is critical in production",
            "Testing early prevents major refactoring late"
        ]

    def _generate_recommendations(self):
        return [
            "Consider scaling horizontally with Kubernetes",
            "Implement more sophisticated caching strategies",
            "Expand agent capabilities with domain-specific knowledge"
        ]
```

Chapter 10. Anexos

10.1. A. Recursos Adicionales

Documentación Oficial: - AutoGen Documentation: https://microsoft.github.io/autogen/ - Ollama Documentation: https://github.com/ollama/ollama - LangChain Documentation: https://python.langchain.com/ - LlamaIndex Documentation: https://docs.llamaindex.ai/

Repositorios de Ejemplo: - AutoGen Examples: https://github.com/microsoft/autogen/tree/main/examples - Ollama Examples: https://github.com/ollama/examples - Community Projects: https://github.com/topics/autogen

Comunidad y Foros: - AutoGen GitHub Discussions - Stack Overflow: Tags autogen, langchain - Reddit: r/LanguageModels

Artículos y Papers: - "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation" - Papers on arxiv.org related to agent systems - Blog posts on Medium and Dev.to

---

10.2. B. Glosario de Términos

Terminología de AutoGen: - **Agent**: Entidad autónoma que interactúa con otros agentes - **Conversation**: Intercambio de mensajes entre agentes - **UserProxyAgent**: Agente que representa al usuario - **AssistantAgent**: Agente que responde usando un LLM

Conceptos de LLM: - **Token**: Unidad atómica de texto (palabra o subpalabra) - **Prompt**: Entrada de texto para el LLM - **Temperature**: Parámetro que controla aleatoriedad (0-1) - **Context Window**: Número máximo de tokens en una conversación

Arquitectura de Agentes: - **Reasoning**: Proceso de deducción del agente - **Acting**: Ejecución de acciones del agente - **Learning**: Mejora del agente con experiencia - **Coordination**: Sincronización entre múltiples agentes

Referencias Técnicas: - **Ollama**: Herramienta para ejecutar LLMs localmente - **Mistral**: Modelo de lenguaje abierto - **Embeddings**: Representaciones vectoriales de texto - **Vector Database**: BD especializada en búsqueda vectorial

---

10.3. C. Mejores Prácticas

Patrones de Diseño:

1. **Single Responsibility Pattern** .
    - Cada agente tiene una responsabilidad clara
    - Facilita testing y mantenimiento

2. **Observer Pattern** .
    - Agentes observan cambios en el estado
    - Comunicación desacoplada
3. **Factory Pattern** .
    - Crear agentes de forma programática
    - Configuración centralizada

Anti-patrones a Evitar:

1. **Monolithic Agent** .
    - Todos los agentes en uno
    - Difícil de probar y mantener
2. **No Error Handling** .
    - Sin manejo de excepciones
    - Sistema frágil
3. **Hardcoded Configuration** .
    - Sin separación de configuración
    - Difícil de desplegar

Optimización de Rendimiento:

1. Implementar caché para respuestas frecuentes
2. Usar embeddings para búsqueda rápida
3. Monitorear latencia y tokens continuamente
4. Implementar rate limiting
5. Usar connection pooling para BD

Guías de Estilo:

```
# Bueno
class DataProcessingAgent(AssistantAgent):
    """Agente especializado en procesamiento de datos"""

    def process_data(self, data: dict) -> dict:
        """Procesar datos de entrada"""
        # Implementación clara
        pass

# Malo
class A(AssistantAgent):
    def p(self, d):  # Nombres poco claros
        pass
```

10.4. D. Troubleshooting

Problemas Comunes:

1. **"Ollama connection refused"** .
   - Solución: Verificar que Ollama esté ejecutándose
   - Comando: `ollama serve`
2. **"Model not found"** .
   - Solución: Descargar el modelo
   - Comando: `ollama pull mistral`
3. **"Token limit exceeded"** .
   - Solución: Usar modelo con mayor context window
   - O: Implementar compresión de contexto
4. **"Memory usage too high"** .
   - Solución: Reducir batch size
   - O: Implementar paginación

Soluciones Rápidas:

```
# Debug mode
import logging
logging.basicConfig(level=logging.DEBUG)

# Health check
from ollama_utils import test_ollama_connection
if test_ollama_connection():
    print("Ollama is working!")

# Monitor resources
import psutil
print(f"Memory: {psutil.virtual_memory().percent}%")
```

FAQ:

P: ¿Cuál es la diferencia entre AssistantAgent y UserProxyAgent? R: AssistantAgent usa LLM para responder, UserProxyAgent representa al usuario.

P: ¿Puedo usar AutoGen sin Ollama? R: Sí, pero requieres una API key de OpenAI o similar.

P: ¿Cómo escalalo a producción? R: Usar Kubernetes con múltiples réplicas de agentes.

P: ¿Cuál es el costo típico? R: Depende del proveedor. Ollama local = $0. OpenAI = $0.001-0.1 por request.

Soporte y Ayuda:

- GitHub Issues: https://github.com/microsoft/autogen/issues
- Discussions: https://github.com/microsoft/autogen/discussions
- Email: autogen@microsoft.com (si aplica)